DISUSUN UNTUK MEMENUHI TUGAS MATA KULIAH APLIKASI MIKROKONTROLER

2022

BUKU BESAR APLIKASI MIKROKONTROLER 3-TNK

DISUSUN OLEH:

LUTHFI NUR FADHILLAH (191344015)
MUTIA TSANI AZ ZAHRA (191344020)
VINZA KAUZAR JANZA (191344029)

KATA PENGANTAR

Puji syukur atas kehadirat Tuhan Yang Maha Esa karena rahmatnya sehingga penulis dapat menyelesaikan tugas buku besar mata kuliah Aplikasi Mikrokontroler dengan dilimpahkan nikmat iman dan kesehatan.

Penulis mengucapkan terima kasih kepada Bapak Ferry Satria, BSEE., M.T. dan Rahmawati Hasanah, S.ST., M.T. selaku dosen mata kuliah Aplikasi Mikrokontoler yang telah memberikan kepercayaan kepada penulis untuk menyelesaikan tugas buku besar ini dengan tepat waktu. Penulis berharap penulisan buku besar ini dapat bermanfaat dalam meningkatkan pengetahuan dan wawasan pembaca mengenai Aplikasi Mikrokontroler.

Penulis mengakui bahwa terdapat banyak kekurangan pada karya ilmiah ini. Oleh karena itu, kritik dan saran dari seluruh pihak senantiasa penulis harapkan demi kesempurnaan karya penulis. Semoga karya ilmiah ini dapat membawa pemahaman dan pengetahuan bagi pembaca dan penulis mengenai Aplikasi Mikrokontroler.

Bandung, 9 Januari 2022

Penulis

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB 1 PENJUMLAHAN BILANGAN HEKSADESIMAL PADA ATMEGA8535 . 1
1.1 Pendahuluan ATmega8535 1
1.2 Langkah Pemrograman Atmega8535 pada AVRstudio 3
1.3 Penjumlahan Bilangan Heksadesimal pada Atmega8535 5
1.4 Contoh Program Penjumlahan Bilangan Heksadesimal 6
BAB 2 PENJUMLAHAN BILANGAN DESIMAL PADA ATMEGA8535 10
2.1 Teori Penjumlahan Bilangan Desimal pada Atmega8535 10
2.2 Contoh Program Penjumlahan Bilangan Desimal 11
BAB 3 PENGISIAN DATA MENGGUNAKAN LOOP PADA ATMEGA8535 13
3.1 Teori Pengisian Data pada Atmega8535 13
3.2 Contoh Program Pengisian Data 14
BAB 4 KONVERSI BILANGAN PADA ATMEGA8535 17
4.1 Konversi Desimal ke Heksadesimal 17
4.2 Konversi Heksadesimal ke Desimal 19
4.3 Konversi Oktal ke Desimal 25
BAB 5 DATA TERKECIL DAN TERBESAR PADA ATMEGA8535 31
5.1 Teori Pendukung Mencari Data Terkecil dan Terbesar 31
5.2 Contoh Program Mencari Data Terkecil dan Terbesar 31
BAB 6 PENGGUNAAN PROTEUS UNTUK SIMULASI ATMEGA8535 41
6.1 Langkah-langkah Menggunakan Proteus untuk Simulasi 41
6.2 Contoh Program Penggunaan Proteus untuk Simulasi Atmega8535 43
BAB 7 BINARY COUNTER PADA ATMEGA 8535 46
7.1 Teori Binary Counter 46
7.2 Contoh Program Binary Counter 46
BAB 8 INTERRUPT PADA ATMEGA 8535 51
8.1 Teori Interrupt 51

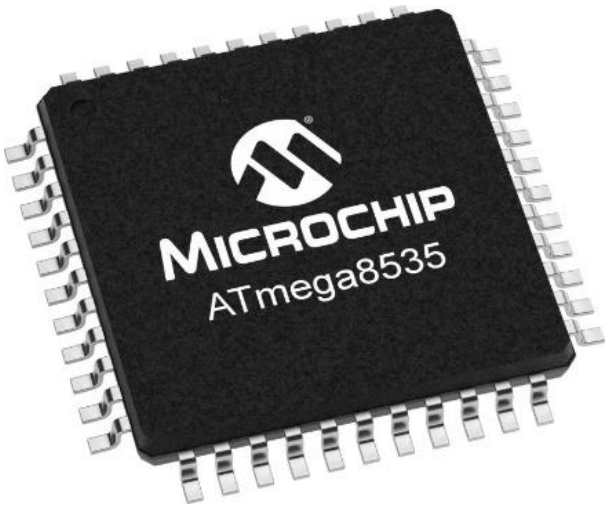
8.2 Contoh Program Interrupt 53
BAB 9 POLLING INTERRUPT PADA ATMEGA 8535 63
9.1 Teori Sistem Polling dan Interrupt Polling 63
9.2 Contoh Program Sistem Polling dan Interrupt Polling 63
BAB 10 DASAR ARDUINO 73
10.1 Pengertian Arduino 73
10.2 Pin-pin pada Arduino 74
10.3 Jenis-jenis Arduino 78
10.4 Skema Proyek Arduino 78
10.5 Input dan Output Modul 79
10.6 Pemrograman Software Arduino IDE 79
10.7 Contoh Program Arduino 81
BAB 11 PWM, PUSH BUTTON, DAN POTENSIOMETER PADA ARDUINO 85
11.1 Teori PWM 85
11.2 Teori dan Contoh Program Push Button 85
11.3 Teori dan Contoh Program Potensiometer 87
BAB 12 PENGGUNAAN SENSOR PADA ARDUINO 91
12.1 Teori Sensor 91
12.2 Sensor Cahaya (LDR) / Photoresistor 91
12.3 Sensor Ultrasonik (HC-SR204) 93
12.4 Sensor Suhu LM35 97
12.5 Sensor Suhu dan Kelembapan DHT11 99
BAB 13 PENGGUNAAN LCD PADA ARDUINO 101
13.1 Teori LCD pada Arduino 101
13.2 Contoh Program LCD pada Arduino 103
BAB 14 AKTUATOR (RELAY DAN MOTOR DC) PADA ARDUINO 107
14.1 Pengertian Relay 107
14.2 Fungsi Relay 107
14.3 Cara Kerja Relay 108
BAB 15 WSN DAN IOT 118
15.1 Teori WSN 118

15.2 Teori IOT 123
DAFTAR PUSTAKA 128

BAB 1
PENJUMLAHAN BILANGAN HEKSADESIMAL PADA ATMEGA8535

1.1 Pendahuluan ATmega8535

ATmega8385 merupakan mikrokontroler CMOS 8 bit berbasis AVR yang menggunakan arsitektur RISC (Reduced Instruction Set Computing) atau komputasi set instruksi yang disederhanakan. Pengisian program pada mikrokontroler AVR dilakukan dengan menggunakan software AVRStudio yang menggunakan bahasa assembly [1].



Berikut adalah beberapa karakteristik dari ATmega8385 [1]:

1. Sistem mikrokontroler 8 bit berbasis RISC dgn kecepatan maksimal 16 MHz.
2. Memiliki memori flash 8KB, SRAM 512 byte, dann EEPROM 512 byte.
3. Memiliki ADC (converter analog ke digital) internal dgn ketelitian 10 bit sebanyak 8 saluran.
4. Memilki PWM (Pulse Width MOdulaton/Gelombang Sinyal Termodulasi) internalsebanyak 4 saluran.

5. Port komuninkasi serial (USART) dengan kecepatan maksimal 2,5 Mbps.
6. Memiliki 6 pilihan mode sleep untuk menghemat penggunaan daya listrik.

AVR ATmega8535 memiliki 32 General Purpose Register (GPR) yang digunakan untuk menyimpan data yang di mana register tersebut adalah R0 sampai dengan R1 yang berada di lokasi memori terendah (00H sampai dengan 1FH). Register ini digunakan untuk operasi logka dan operasi aritmatika [1].

Berikut adalah beberapa Flag Register pada AVR ATmega8535 [2]:

1. Z (Zero Flag): Jika operasi logis atau aritmatika menghasilkan bilangan nol, maka Z = 1. Jika hasilnya bukan nol maka Z = 0. Zero flag pada umumnya digunakan pada operasi loop atau perulangan.
2. N (Negative Flag): Merupakan representasi biner dari signed numbers. Jika hasil dari operasi aritmatika adalah bilangan negatif maka N = 1. Jika hasilnya positif amka N = 0.
3. V (Overflow Flag): JIka hasil dari operasi bilangan bulat melampaui bit penanda, maka V = 1. Overflow flag pada umumnya digunakan untuk mendeteksi kesalahan pada operasi signed number.
4. C (Carry Flag): Jika ada carry yang keluar dari registernya maka C = 1. Carry flag pada umumnya digunakan untuk mendeteksi kesalahan pada operasi aritmatika unsigned number.
5. H (Half Carry Flag): Jika terdapat carry dari D3 ke D4 selama operasi ADD atau SUB, maka H = 1.

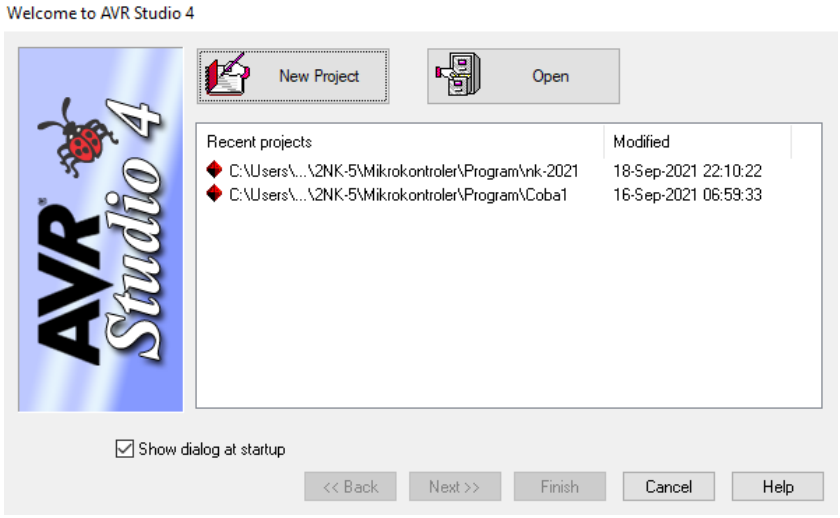
Berikut adalah Flags pada I/O Registers [2]:

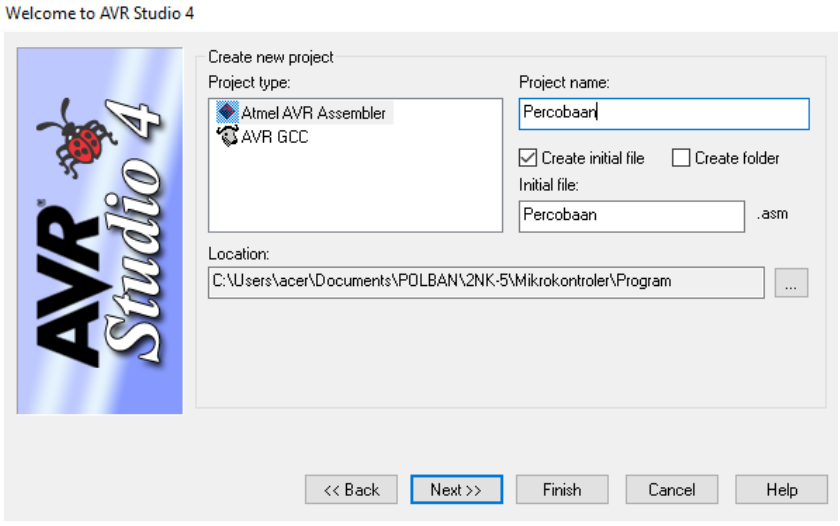
1. ⇔ : Flag yang dipengaruhi oleh perintah
2. **0** : Flag yang dihapus oleh perintah
3. **1** : Flag yang di-set oleh perintah
4. **-** : Flag yang tidak terpengaruh oleh perintah

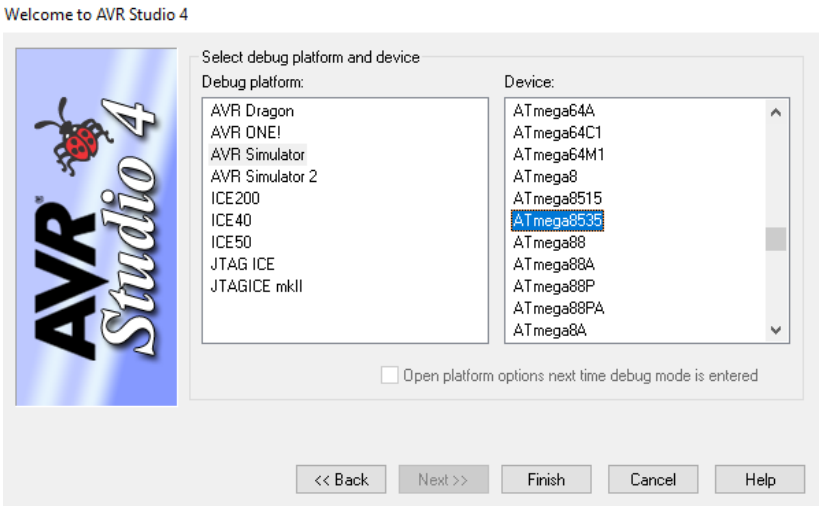
1.2 Langkah Pemrograman Atmega8535 pada AVRstudio

Berikut adalah langkah-langkah untuk melakukan pemrograman Atmega8535 pada AVRStudio:

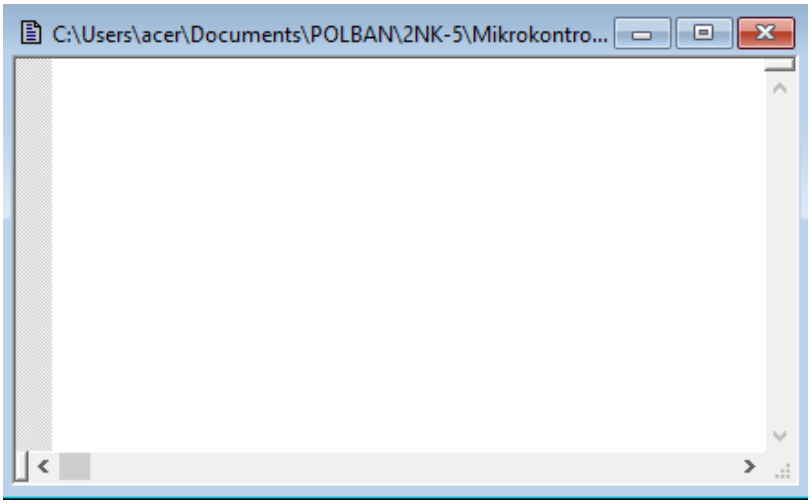
1. *Install* AVRStudio.
2. Klik New Project > Pilih Atmel AVR Assembler > Isi project name > Tentukan lokasi penyimpanan > Klik Next > Pada Debug Platform pilih AVR Simulator > Pada Device pilih ATmega8535 > Klik Finish.



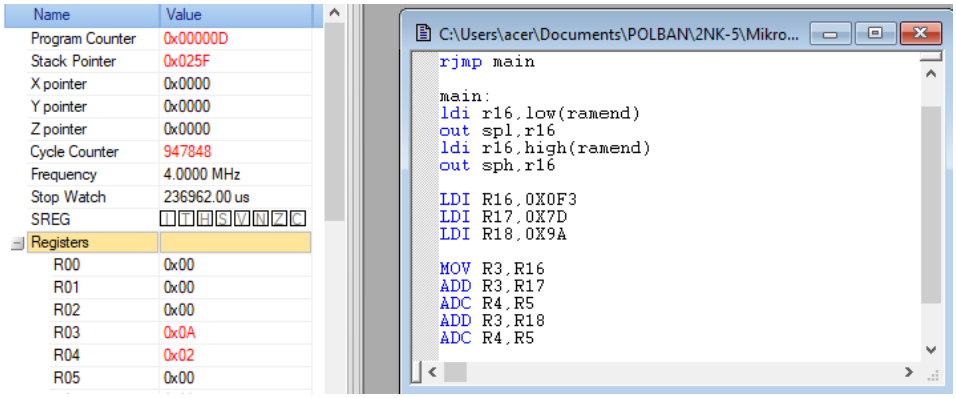




3. Tulis pemrograman pada Workspace berikut.



4. Setelah selesai menuliskan program, klik Build (F7) > Klik Run (F5) > Klik Break (Ctrl + F5). Hasil dari program akan ditampilkan pada bagian kiri layar.



1.3 Penjumlahan Bilangan Heksadesimal pada Atmega8535

Pada penulisan pemrograman penjumlahan bilangan heksadesimal pada AVR ATmega8583, digunakan instruksi ADD (Add without Carry) dan ADC (Add with Carry).

Instruksi ADD akan menjumlahkan isi dari dua register tanpa carry dan menempatkan hasil pada register Rd tujuan.

Contoh:

ADD R1, R2

Perintah di atas menjumlahkan isi register R2 ke register R1 (R1 = R1 + R2).

Sedangkan instruksi ADC akan menjumlahkan dua register bersamaan dengan carry dan turut menempatkan hasil pada register Rd tujuan.

Contoh:

ADD R2, R0

ADC RC, R1

Perintah di atas menjumlahkan R1:R0 ke R3:R2, menjumlahkan low byte, dan menjumlahkannya dengan carry high bite.

Selain itu, turut digunakan beberapa perintah berikut:

MOV (Copy Register): Memindahkan atau menyalin muatan pada satu register ke register lainnya.

LDI (Load Immediate): Memuat nilai 8 bit konstan secara langsung ke register yang telah ditentukan. Contohnya, bilangan 0X0F3 dapat dimuat ke register R16 dengan perintah ldi R16, 0X0F3.

Dalam melakukan penulisan pemrograman untuk menjumlahkan bilangan heksadesimal pada Atmega8535, pada umumnya terdapat register yang menjadi register penyimpan hasil dari setiap proses penjumlahan yang menggunakan perintah

ADD dan terdapat register lainnya yang menjadi penyimpan hasil penjumlahan dan turut menyimpan hasil carry-nya yang menggunakan perintah ADC. Proses penjumlahan menggunakan perintah ADD dan ADC ini terus berlanjut sampai batas yang ditentukan.

1.4 Contoh Program Penjumlahan Bilangan Heksadesimal

1. Buat program untuk menjumlahkan bilangan heksadesimal 4C + 7F + D4 + A5 + FF.

   Program:

```
.include"m8535def.inc"
.org 0x0000
rjmp main

main:
ldi r16,low(ramend)
out spl,r16
ldi r16,high(ramend)
out sph,r16

ldi r16,0x4c
ldi r17,0x7f
ldi r18,0x0d4
ldi r19,0x0a5
ldi r20,0x0ff

mov r3,r16
add r3,r17
adc r4,r5

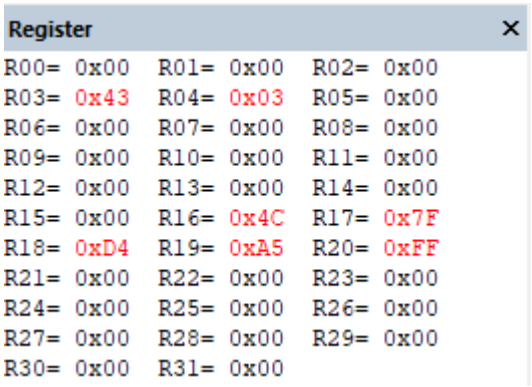
add r3,r18
adc r4,r5

add r3,r19
adc r4,r5

add r3,r20
adc r4,r5

loop: rjmp loop
```

   Hasil:



   Program diawali dengan memuat register r16 sampai dengan r20 dengan bilangan heksadesimal menggunakan perintah LDI (Load Immediate). Register r3 menjadi register yang menyimpan hasil dari setiap proses

penjumlahan bilangan 8 bit menggunakan perintah ADD. Register r4 menjadi register yang menyimpan hasil penjumlahan dan turut menyimpan hasil carry-nya menggunakan perintah ADC. Proses penjumlahan menggunakan perintah ADD dan ADC terus berlanjut sampai menjumlahkan 5 bilangan heksadesimal. Hasil disimpan dengan low bit pada r3 dan high bit pada r4.

2. Buat program untuk menjumlahkan bilangan heksadesimal C59A + 5DFC. Simpan hasil pada R22 : R21 : R20.

Program:

```
.include"m8535def.inc"
.org 0x0000
rjmp main

main:
ldi r16,low(ramend)
out spl,r16
ldi r16,high(ramend)
out sph,r16

ldi r16,0x9a
ldi r17,0x0c5
ldi r18,0x0fc
ldi r19,0x5d

mov r20,r16
mov r21,r17
add r20,r18
adc r21,r5

add r21,r19
adc r22,r5

loop: rjmp loop
```

Hasil:

Register

R00= 0x00	R01= 0x00	R02= 0x00
R03= 0x00	R04= 0x00	R05= 0x00
R06= 0x00	R07= 0x00	R08= 0x00
R09= 0x00	R10= 0x00	R11= 0x00
R12= 0x00	R13= 0x00	R14= 0x00
R15= 0x00	R16= 0x9A	R17= 0xC5
R18= 0xFC	R19= 0x5D	R20= 0x96
R21= 0x23	R22= 0x01	R23= 0x00
R24= 0x00	R25= 0x00	R26= 0x00
R27= 0x00	R28= 0x00	R29= 0x00
R30= 0x00	R31= 0x00	

Program ini dibuat untuk menjumlahkan 2 bilangan heksadesimal 16 bit. Setelah memuat bilangan-bilangan ke register-register yang ditentukan,

bilangan C59Ah dipindahkan dengan dibagi dua yaitu low bit ke r20 dan high bit ke r21. Bilangan low bit tersebut dijumlahkan dengan bilangan low bit pada bilangan 5DFCh yang hasilnya akan tersimpan di register r20 dengan menggunakan perintah ADD dan carry-nya dsimpan di register r21 dengan menggunakan perintah ADC. Isi carry di register r21 dijumlahkan dengan high bit dari 5DFCh yang hasilnya disimpan pada register r21 dan carry-nya disimpan pada register r22.

3. Buat program untuk menjumlahkan data biner 16 bit: 1234, A2C5, FFFF, D5C7, ABCD, BECA, CABE, DEDE. Simpan hasil penjumlahan pada register R20:R19:R18.

Program:

```
.include"m8535def.inc"
.org 0x0000
rjmp main

main:
ldi r16,low(ramend)
out spl,r16
ldi r16,high(ramend)
out sph,r16

ldi r18,0x34
ldi r19,0x12

ldi r17,0x0c5
ldi r16,0x0a2
rcall tmbh

ldi r17,0x0ff
ldi r16,0x0ff
rcall tmbh

ldi r17,0x0c7
ldi r16,0x0d5
rcall tmbh

ldi r17,0x0cd
ldi r16,0x0ab
rcall tmbh

ldi r17,0x0ca
ldi r16,0x0be
rcall tmbh

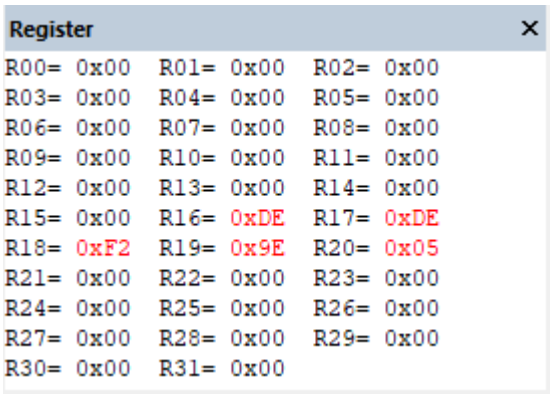
ldi r17,0x0be
ldi r16,0x0ca
rcall tmbh

ldi r17,0x0de
ldi r16,0x0de
rcall tmbh

loop: rjmp loop

tmbh:
add r18,r17
adc r19,r16
adc r20,r1
ret
```

Hasil:



Pada penulisan program, register r16 dan r17 menjadi penyimpan nilai yang akan menjadi penjumlah sehingga nilainya selalu berubah-ubah setelah dilakukan proses penjumlahan, terkecuali bilangan 1234h yang menjadi nilai awal sehingga disimpan di register r19:r18. Kemudian, pada program digunakan subrutin tmbh sebagai perintah untuk menjumlahkan register r18 dengan r17 yang di mana jika terdapat carry maka akan turut dijumlahkan saat terjadi penjumlahan r19 dengan r16. Namun, untuk register r20 hanya akan digunakan pada perintah ADC karena hanya digunakan untuk menyimpan carry dari hasil penjumlahan r19 dengan r16.

BAB 2

PENJUMLAHAN BILANGAN DESIMAL PADA ATMEGA8535

2.1 Teori Penjumlahan Bilangan Desimal pada Atmega8535

Untuk melakukan operasi perhitungan bilangan desimal maka bilangan harus diberi faktor koreksi. Faktor koreksi tersebut merupakan subrutin bernama daa yang digunakan untuk koreksi hasil penjumlahan menjadi BCD 2 digit yang selalu dilaksanakan melalui register r19. Subrutin ini dibuat oleh Bapak Ferry Satria, BSEE, M.T. Berikut adalah subrutin tersebut:

```
.include"m8535def.inc"
.org 0x0000
rjmp main

main:
                ldi r16,low(ramend)
                out spl,r16
                ldi r16,high(ramend)
                out sph,r16

                LDI R19,0X90
                ADD R19,R19
                RCALL DAA

loop: rjmp loop


; subrutin unt koreksi hasil penjumlahan mjd. BCD.2 dgt.
; yang dilaksanakan mll. r19 sbg. acc.:
daa: push r16;amankan isi reg. ke stack
                push r17
                push r21
                push r24
                in r16,sreg; amankan sreg
                in r17,sreg
                ldi r24,0 ; r24 hanya sbg status dr carry flag
                adc r24,r24; status carry flag sebelum koreksi
; ambil, amankan kembali sreg, uji half carry
                out sreg,r17
                brhc lup4; lompat ke lup4 jika H=0
                rcall fk_ln; satuan + 06 jika H=1
        ;       rjmp lup5
lup4:           mov r21,r19
                andi r21,$0f; ambil dgt.satuan
                cpi r21,$0a ; > 9
                brcs lup5; lompat ke lup5 jika < 9
                rcall fk_ln; satuan + 06 jika > 9
lup5:           out sreg,r17; ambil, amankan kembali sreg, uji carry
                brcc lup6; lompat ke lup6 jika C=0
```

```
                rcall fk_hn; puluhan + 60 jika C=1
        ;       rjmp lup7
lup6:           mov r21,r19
                andi r21,0xf0; ambil dgt.puluhan
                cpi r21,0xa0
                brcs lup7; lompat ke lup7 jika pul < a
                rcall fk_hn; pul + 60 jika > 9
lup7:           out sreg,r16; mengatur carry flag sblm keluar:
                clc; C = 0
                in r16,sreg ;amankan sreg stlh c=0
                cpi r24,0 ;menguji status
                breq lup8
                out sreg,r16;ambil sreg, set c
                sec; C = 1
                in r16,sreg; amankan sreg stlh c=1
lup8:
                out sreg,r16;status akhir
                pop r24;kembalikan isi reg. dr stack
                pop r21
                pop r17
                pop r16
                ret
fk_ln:          ldi r21,06; koreksi dgt.satuan
                out sreg,r16
                add r19,r21
                in r16,sreg
                brcc tuh
                ldi r21,0x01
                add r17,r21
tuh:  out sreg,r16
                adc r24,r24
                ret
fk_hn:          ldi r21,$60; koreksi dgt.puluhan
                out sreg,r16
                add r19,r21
                in r16,sreg
                adc r24,r24; amankan carry pd r24
                ret ; akhir dari prog. subrutin.
```

Faktor koreksi ini diimplementasikan dengan memeriksa terlebih dahulu setiap 4 bit nya. Jika terdeteksi bahwa nilainya lebih dari 9, maka faktor koreksi akan diberlakukan dengan melakukan penjumlahan dengan 6 atau 0110.

2.2 Contoh Program Penjumlahan Bilangan Desimal

Buat program untuk menjumlahkan 5 data Desimal 5 digit berikut, dan simpan hasil penjumlahan pada Register R25:R24:R23. [67895, 55667, 89459, 92467, 87996].

Program:

```
.include"m8535def.inc"
.org 0x0000
rjmp main

main:
        ldi r16,low(ramend)
        out spl,r16
        ldi r16,high(ramend)
        out sph,r16
        ;67895,55667,89459,92467,87996
        ;25;24;23

        ldi r25,0x6
        ldi r24,0x78
        ldi r23,0x95

        ldi r19,0x67
        add r19,r23
        rcall daa
        mov r23,r19

        ldi r19,0x56
        adc r19,r24
        rcall daa
        mov r24,r19

        ldi r19,0x5
        adc r19,r25
        rcall daa
        mov r25,r19

        ldi r19,0x59
        add r19,r23
        rcall daa
        mov r23,r19

        ldi r19,0x94
        adc r19,r24
        rcall daa
        mov r24,r19

        ldi r19,0x8
        adc r19,r25
        rcall daa
        mov r25,r19

        ldi r19,0x67
        add r19,r23
        rcall daa
        mov r23,r19

        ldi r19,0x24
        adc r19,r24
        rcall daa
        rcall daa
        mov r24,r19

        ldi r19,0x9
        adc r19,r25
        rcall daa
        mov r25,r19

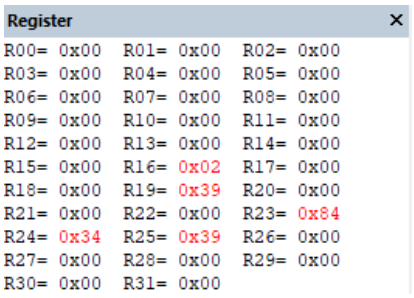
        ldi r19,0x96
        add r19,r23
        rcall daa
        mov r23,r19

        ldi r19,0x79
        adc r19,r24
        rcall daa
        mov r24,r19

        ldi r19,0x8
        adc r19,r25
        rcall daa
        mov r25,r19

loop:   rjmp loop
```

Hasil:



Dalam melakukan operasi perhitungan bilangan desimal, karena default data yang di-input pada AVR ATmega8535 merupakan bilangan heksadesimal, maka bilangan heksadesimal tersebut harus diberi faktor koreksi. Faktor koreksi tersebut merupakan subrutin bernama daa yang digunakan untuk koreksi hasil penjumlahan menjadi BCD 2 digit yang selalu dilaksanakan melalui register r19. Subrutin ini sudah dibuat oleh Bapak Ferry Satria, BSEE, M.T. Penulisan program diawali dengan mengisi register r25:r24:r23 dengan data awal. Kemudian, untuk proses penjumlahannya, setiap 2 digit dalam satu buah data 7 digit tersebut harus diberi terlebih dahulu faktor koreksi sekaligus menambahkannya dengan nilai yang sudah terdapat pada register sebelumnya. Konsep penjumlahan yang sama terus berlanjut sampai kelima data berhasil dijumlahkan.

BAB 3
PENGISIAN DATA MENGGUNAKAN LOOP PADA ATMEGA8535

3.1 Teori Pengisian Data pada Atmega8535

Pada penulisan program pengisian data pada ATmega8535 dilakukan dengan menggunakan pointer X, Y, maupun Z. Pointer ini digunakan sebagai penunjuk alamat memori program dan untuk melewatkan data menggunakan suatu referensi yang digunakan pada program dengan menggunakan konsep loop sehingga dapat menyederhanakan penulisan program [3].

Pointer merupakan penunjuk alamat memori program dan digunakan untuk melewatkan data menggunakan suatu referensi, karena tidak diperlukan men-copy data. Pada pemrograman ATmega8535, terdapat 3 pointer yaitu [3]:

1. Pointer X: R27:R26
2. Pointer Y: R29:R28
3. Pointer Z: R31:R30

Setiap pointer dapat digunakan sebagai indirect pointer ke SRAM

Contoh:

LD R16, X

Dengan menggunakan post-increment atau pre-decrement

Contoh:

LD R16, -Y

ST Z+, R16

Pada penulisan program pengisian data pada ATmega8535, digunakan beberapa instruksi berikut [4]:

1. ST (Store Indirect): Merupakan kebalikan dari LD. Perintah ST menyimpan data dari register disimpan ke memori.

   Operasi:

   1. (X) ← Rr ; X: Tetap
   2. (X) ← Rr, X ← X+1 ; X: Post incremented
   3. X ← X - 1, (X) ← Rr ; X: Pre decremented

2. CPI (Compare with Immediate): Membandingkan antara register Rd dengan sebuah kosntanta. Semua kondisional instruksi percabngan dapat digunakan setelah instruksi CPI.

   Operasi:

   Rd – K

3. BRNE (Branch if Not Equal): Instruksi ini akan lompat ke alamat yang ditunjuk jika di antara dua register atau di antara register dan konstanta yang dibandingkan tidak sama atau dengan kata lain akan lompat jika flag Z = 0. Instruksi ini pada umumnya dipasangkan dengan instruksi CP, CPI, SUB, dan SUBI.

   Operasi:

   Jika Rd ≠ Rr (Z = 0) maka PC ← PC + k + 1, dan PC ← PC + 1

3.2 Contoh Program Pengisian Data

1. Buat program untuk mengisi R0 s/d R24 dengan data 25, 24, 23, 22, …, 01, 00 dengan menggunakan loop.

   Program:

```
.include"m8535def.inc"
.org 0x0000
rjmp main

main:
        ldi r16,low(ramend)
        out spl,r16
        ldi r16,high(ramend)
        out sph,r16
        ;
        ldi r28,25
        ldi r29,0x6
        ldi r26,0

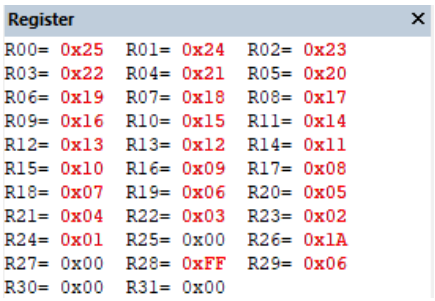
y1:     mov r31,r28
        cpi r28,20
        brlo y2
        add r28,r29

y2:     cpi r28,10
        brlo y3
        add r28,r29

y3:     st x+,r28
        mov r28,r31
        dec r28
        cpi r26,26
        brne y1

loop:   rjmp loop
```

Hasil:



Pada program, karena data yang diisikan merupakan bilangan desimal maka harus dilakukan koreksi terlebih dahulu pada bilangan heksadesimal dalam program. Proses konversinya atau factor koreksinya adalah jika nilai bilangan lebih besar dari 9 dan berada pada rentang belasan maka bilangan harus ditambah dengan 6 yang disimpan di register r29. Kemudian, jika nilai bilangan pada rentang dua puluhan maka factor koreksinya diberikan sebanyak dua kali sehingga penambahan angka 6 nya dua kali, dan begitu seterusnya pada bilangan dalam rentang lain. Lalu, setelah diberi factor koreksi, data bilangan akan dipindahkan ke register Rd.

2. Buat program untuk mengisi R0 s/d R20 dengan data 8 bit: 00, 03, 06, 09, 0C, 0F, … dengan menggunakan loop.

Program:

```
.include"m8535def.inc"
.org 0x0000
rjmp main

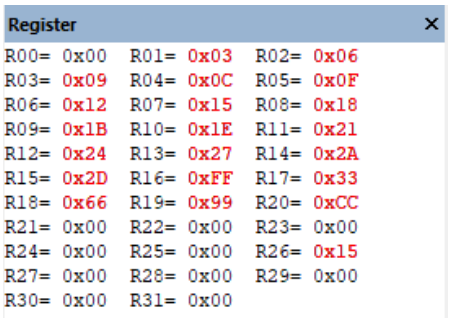
main:
        ldi r16,low(ramend)
        out spl,r16
        ldi r16,high(ramend)
        out sph,r16

        ldi r16,0x0
        ldi r17,0x3
        ldi r26,0

y1:     st x+,r16
        add r16,r17
        cpi r26,21
        brne y1

loop:   rjmp loop
```

Hasil:



Pada program, register r16 diisi dengan nilai awal 0 dan menjadi register yang selalu akan diisikan ke index x. Sedangkan register r17 menjadi register yang selalu menjumlahkan register r16 dengan 0x3. Register r26 diisi dengan alamat awal dari register yang akan diisi dengan 0x55. Perintah st akan menyimpan data dari register ke memori dengan alamat awal yaitu 0 yang kemudian akan bertambah 1 indexnya dalam setiap loop. Perintah ADD pada setiap loopnya akan menjumlahkan isi register r17 dengan r16. Perintah cpi membandingkan immediate data yaitu 21 sebagai batas alamat pengisian data dengan isi dari register r26 yaitu 0. Berdasarkan perintah brne (branch if not equal), jika hasil perbandingan oleh cpi belum sama maka program akan terus mengulang dimulai dari label y1 sehingga dengan loop tersebut akan membatasi berjalannya program sampai r26 berisi 21 yang menandakan data sudah terisi dari register r0 sampai dengan r20.

BAB 4
KONVERSI BILANGAN PADA ATMEGA8535

Untuk program konversi bilangan dari heksadesimal ke desimal, desimal ke heksadesimal, maupun oktal ke desimal digunakan konsep yang sama dengan penghitungan yang dapat dilakukan secara matematis yaitu menggunakan persamaan yang telah ditentukan sebagai berikut.

4.1 Konversi Desimal ke Heksadesimal

Untuk program mengkonversi bilangan desimal ke heksadesimal digunakan konsep penghitungan secara matematis dimana persamaannya adalah sebagai berikut:

$ABCDEF_{(10)}$

$$\left(\left(\left(\left(\left(\left(\left((A \times 10) + B\right) \times 10\right) + C\right) \times 10\right) + D\right) \times 10\right) + E\right) \times 10\Big) + F$$

Contoh program:

Buatlah program untuk mengubah data Desimal 789456 menjadi Heksadesimal. Simpan hasil konversi pada Register R23:R22:R21.

Code:

```
.include "m8535def.inc"
.org 0x0000
rjmp main

main:   ldi r16,low(ramend)
        out spl,r16
        ldi r16,high(ramend)
        out sph,r16

        ldi ZL,LOW(DAT<<1)
        ldi ZH,HIGH(DAT<<1)
        lpm r18,Z+
        mov r19,r18
        andi r19,0x0f
        mov r21,r19
        mov r19,r18
        andi r19,0xf0
        lsr r19 ; dikali 8
        mov r24,r19
        lsr r19
        lsr r19 ; dikali 2
        add r24,r19
        add r21,r24
```

```
hush:   rcall kali10
        lpm r18,Z+
        mov r19,r18
        andi r19,0xf0
        lsr r19
        lsr r19
        lsr r19
        lsr r19
        add r21,r19
        adc r22,r25
        adc r23,r26
        rcall kali10
        mov r19,r18
        andi r19,0x0f
        add r21,r19
        adc r22,r25
        adc r23,r26
        inc r29

        cpi r29,2
        brne hush

loop:rjmp loop

DAT:
.DB 0x78,0x94,0x56 ; 789456 = C0BD0

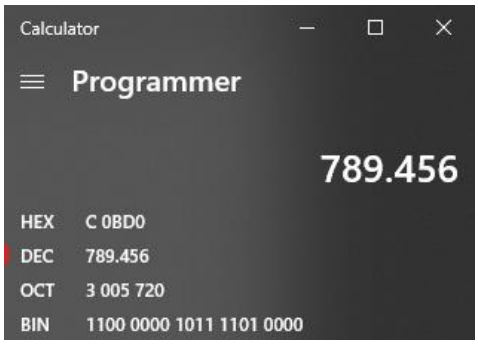
kali10: push r24
        push r25
        push r26
        lsl r21
        rol r22
        rol r23
        mov r24,r21
        mov r25,r22
        mov r26,r23
        lsl r21
        rol r22
        rol r23
        lsl r21
        rol r22
        rol r23
        add r21,r24
        adc r22,r25
        adc r23,r26
        pop r26
        pop r25
        pop r24
        ret
```

Hasil:

Register

R00=	0x00	R01=	0x00
R02=	0x00	R03=	0x00
R04=	0x00	R05=	0x00
R06=	0x00	R07=	0x00
R08=	0x00	R09=	0x00
R10=	0x00	R11=	0x00
R12=	0x00	R13=	0x00
R14=	0x00	R15=	0x00
R16=	0x02	R17=	0x00
R18=	0x56	R19=	0x06
R20=	0x00	R21=	0xD0
R22=	0x0B	R23=	0x0C
R24=	0x46	R25=	0x00
R26=	0x00	R27=	0x00
R28=	0x00	R29=	0x02
R30=	0x53	R31=	0x00

Pembuktian:



4.2 Konversi Heksadesimal ke Desimal

Untuk program mengkonversi bilangan heksadesimal ke desimal digunakan konsep penghitungan secara matematis dimana persamaannya adalah sebagai berikut:

$ABCDEFG_{(16)}$

$$\Big(\Big(\Big(\Big(\Big(\Big(\Big(\Big(\big((A \times 16) + B\big) \times 16\big) + C\Big) \times 16\Big) + D\Big) \times 16\Big) + E\Big) \times 16\Big) + F\Big) \times 16\Big) + G$$

Contoh program:

Buatlah program untuk mengubah data Heksadesimal E5D78C9 menjadi Desimal. Simpan hasil konversi pada Register R25:R24:R23:R22:R21.

Code:

```
.include"m8535def.inc"
.org 0x0000
rjmp main
main:
        ldi r16,low(ramend)
        out spl,r16
        ldi r16,high(ramend)
        out sph,r16

        ldi ZL,LOW(DAT<<1)
        ldi ZH,HIGH(DAT<<1)
```

```
        lpm r18,Z+
        mov r19,r18
        andi r19,0xf0
        lsr r19
        lsr r19
        lsr r19
        lsr r19
        rcall daa
        lsl r19
        rcall daa
        lsl r19
        rcall daa
        lsl r19
        rcall daa
        adc r22,r5
        add r19,r19
        rcall daa
        adc r22,r22
        rcall daa
        mov r21,r19
        andi r18,0x0f
        mov r19,r18
        rcall daa
        add r21,r19
        adc r22,r5

hush:     rcall kali16
        lpm r18,Z+
        mov r19,r18
        andi r19,0xf0
        lsr r19
        lsr r19
        lsr r19
        lsr r19
        add r21,r19
        mov r19,r21
        rcall daa
        mov r21,r19
        adc r22,r5
        mov r19,r22
        rcall daa
        mov r22,r19
        adc r23,r5
        mov r19,r23
```

```
                rcall daa
                mov r23,r19
                adc r24,r5
                mov r19,r24
                rcall daa
                mov r24,r19
                adc r25,r5
                mov r19,r25
                rcall daa
                mov r25,r19

                rcall kali16
                mov r19,r18
                andi r19,0x0f
                add r21,r19
                mov r19,r21
                rcall daa
                mov r21,r19
                adc r22,r5
                mov r19,r22
                rcall daa
                mov r22,r19
                adc r23,r5
                mov r19,r23
                rcall daa
                mov r23,r19
                adc r24,r5
                mov r19,r24
                rcall daa
                mov r24,r19
                adc r25,r5
                mov r19,r25
                rcall daa
                mov r25,r19
                inc r29
                cpi r29,3
                brne hush

loop:rjmp loop
DAT:
.DB 0x0E,0x5D,0x78,0x0C9 ;E5D78C9 = 241006793

kali16:         push r5
                add r21,r21
```

```
mov r19,r21
rcall daa
mov r21,r19
adc r22,r22
mov r19,r22
rcall daa
mov r22,r19
adc r23,r23
mov r19,r23
rcall daa
mov r23,r19
adc r24,r24
mov r19,r24
rcall daa
mov r24,r19
adc r25,r25
mov r19,r25
rcall daa
mov r25,r19

add r21,r21
mov r19,r21
rcall daa
mov r21,r19
adc r22,r22
mov r19,r22
rcall daa
mov r22,r19
adc r23,r23
mov r19,r23
rcall daa
mov r23,r19
adc r24,r24
mov r19,r24
rcall daa
mov r24,r19
adc r25,r25
mov r19,r25
rcall daa
mov r25,r19

add r21,r21
mov r19,r21
rcall daa
```

```
                mov r21,r19
                adc r22,r22
                mov r19,r22
                rcall daa
                mov r22,r19
                adc r23,r23
                mov r19,r23
                rcall daa
                mov r23,r19
                adc r24,r24
                mov r19,r24
                rcall daa
                mov r24,r19
                adc r25,r25
                mov r19,r25
                rcall daa
                mov r25,r19

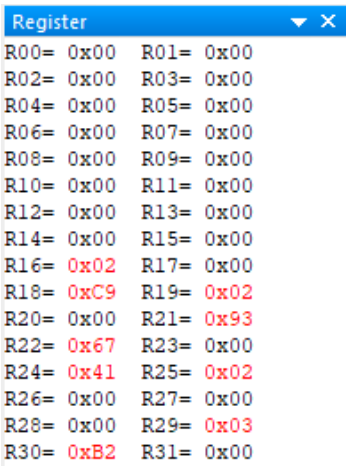
                add r21,r21
                mov r19,r21
                rcall daa
                mov r21,r19
                adc r22,r22
                mov r19,r22
                rcall daa
                mov r22,r19
                adc r23,r23
                mov r19,r23
                rcall daa
                mov r23,r19
                adc r24,r24
                mov r19,r24
                rcall daa
                mov r24,r19
                adc r25,r25
                mov r19,r25
                rcall daa
                mov r25,r19
                pop r5
                ret

daa:  push r16;amankan isi reg. ke stack
                push r17
                push r21
```

```
                push r24
                in r16,sreg; amankan sreg
                in r17,sreg
                ldi r24,0 ; r24 hanya sbg status dr carry flag
                adc r24,r24; status carry flag sebelum koreksi
                out sreg,r17
                brhc lup4; lompat ke lup4 jika H=0
                rcall fk_ln; satuan + 06 jika H=1
lup4:           mov r21,r19
                andi r21,$0f; ambil dgt.satuan
                cpi r21,$0a ; > 9
                brcs lup5; lompat ke lup5 jika < 9
                rcall fk_ln; satuan + 06 jika > 9
lup5:           out sreg,r17; ambil, amankan kembali sreg, uji carry
                brcc lup6; lompat ke lup6 jika C=0
                rcall fk_hn; puluhan + 60 jika C=1
lup6:           mov r21,r19
                andi r21,0xf0; ambil dgt.puluhan
                cpi r21,0xa0
                brcs lup7; lompat ke lup7 jika pul < a
                rcall fk_hn; pul + 60 jika > 9
lup7:           out sreg,r16; mengatur carry flag sblm keluar:
                clc; C = 0
                in r16,sreg ;amankan sreg stlh c=0
                cpi r24,0 ;menguji status
                breq lup8
                out sreg,r16;ambil sreg, set c
                sec; C = 1
                in r16,sreg; amankan sreg stlh c=1
lup8:           out sreg,r16;status akhir
                pop r24;kembalikan isi reg. dr stack
                pop r21
                pop r17
                pop r16
                ret
fk_ln:                  ldi r21,06; koreksi dgt.satuan
                out sreg,r16
                add r19,r21
                in r16,sreg
                brcc tuh
                ldi r21,0x01
                add r17,r21
tuh:            out sreg,r16
                adc r24,r24
```

```
        ret
fk_hn:  ldi r21,$60; koreksi dgt.puluhan
        out sreg,r16
        add r19,r21
        in r16,sreg
        adc r24,r24; amankan carry pd r24
        ret ; akhir dari prog. subrutin.
```

Hasil (R25:R24:R23:R22:R21):



Pembuktian:



4.3 Konversi Oktal ke Desimal

Untuk program mengkonversi bilangan oktal ke desimal digunakan konsep penghitungan secara matematis dimana persamaannya adalah sebagai berikut:

$ABCDEF_{(8)}$

$$\left(\left(\left(\left(\left(\left(\left((A \times 8) + B\right) \times 8\right) + C\right) \times 8\right) + D\right) \times 8\right) + E\right) \times 8\Big) + F$$

Contoh program:

Buatlah program untuk mengubah data Oktal 457346 menjadi Desimal. Simpan hasil konversi pada Register R23:R22:R21.

Code:

```
.include"m8535def.inc"
.org 0x0000
rjmp main
main:
        ldi r16,low(ramend)
        out spl,r16
        ldi r16,high(ramend)
        out sph,r16

        ldi ZL,LOW(DAT<<1)
        ldi ZH,HIGH(DAT<<1)
        lpm r18,Z+
        mov r19,r18
        andi r19,0xf0
        lsr r19
        lsr r19
        lsr r19
        lsr r19
        rcall daa
        lsl r19
        rcall daa
        lsl r19
        rcall daa
        lsl r19
        rcall daa
        mov r21,r19
        adc r22,r5
        andi r18,0x0f
        mov r19,r18
        rcall daa
        add r21,r19
        adc r22,r5
```

```
hush:           rcall kali8
                lpm r18,Z+
                mov r19,r18
                andi r19,0xf0
                lsr r19
                lsr r19
                lsr r19
                lsr r19
                add r21,r19
                mov r19,r21
                rcall daa
                mov r21,r19
                adc r22,r5
                mov r19,r22
                rcall daa
                mov r22,r19
                adc r23,r5
                mov r19,r23
                rcall daa
                mov r23,r19

                rcall kali8
                mov r19,r18
                andi r19,0x0f
                add r21,r19
                mov r19,r21
                rcall daa
                mov r21,r19
                adc r22,r5
                mov r19,r22
                rcall daa
                mov r22,r19
                adc r23,r5
                mov r19,r23
                rcall daa
                mov r23,r19
                inc r29
                cpi r29,2
                brne hush

loop:rjmp loop

DAT:
.DB 0x45,0x73,0x46 ;457346 = 155366
```

```
kali8:          push r5
                add r21,r21
                mov r19,r21
                rcall daa
                mov r21,r19
                adc r22,r22
                mov r19,r22
                rcall daa
                mov r22,r19
                adc r23,r23
                mov r19,r23
                rcall daa
                mov r23,r19

                add r21,r21
                mov r19,r21
                rcall daa
                mov r21,r19
                adc r22,r22
                mov r19,r22
                rcall daa
                mov r22,r19
                adc r23,r23
                mov r19,r23
                rcall daa
                mov r23,r19

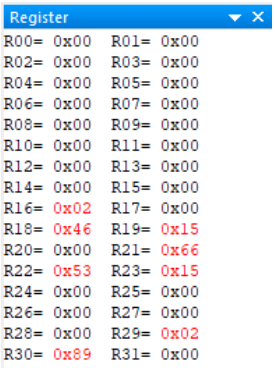
                add r21,r21
                mov r19,r21
                rcall daa
                mov r21,r19
                adc r22,r22
                mov r19,r22
                rcall daa
                mov r22,r19
                adc r23,r23
                mov r19,r23
                rcall daa
                mov r23,r19
                pop r5
                 ret

  daa:  push r16;amankan isi reg. ke stack
```

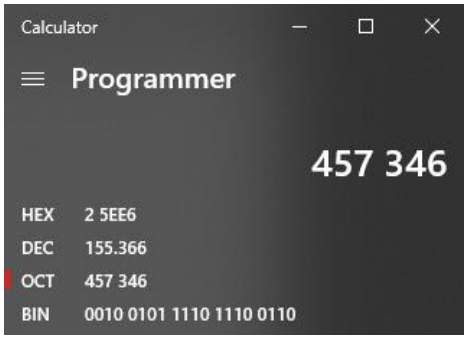
```
                push r17
                push r21
                push r24
                in r16,sreg; amankan sreg
                in r17,sreg
                ldi r24,0 ; r24 hanya sbg status dr carry flag
                adc r24,r24; status carry flag sebelum koreksi
                out sreg,r17
                brhc lup4; lompat ke lup4 jika H=0
                rcall fk_ln; satuan + 06 jika H=1
lup4:           mov r21,r19
                andi r21,$0f; ambil dgt.satuan
                cpi r21,$0a ; > 9
                brcs lup5; lompat ke lup5 jika < 9
                rcall fk_ln; satuan + 06 jika > 9
lup5:           out sreg,r17; ambil, amankan kembali sreg, uji carry
                brcc lup6; lompat ke lup6 jika C=0
                rcall fk_hn; puluhan + 60 jika C=1
lup6:           mov r21,r19
                andi r21,0xf0; ambil dgt.puluhan
                cpi r21,0xa0
                brcs lup7; lompat ke lup7 jika pul < a
                rcall fk_hn; pul + 60 jika > 9
lup7:           out sreg,r16; mengatur carry flag sblm keluar:
                clc; C = 0
                in r16,sreg ;amankan sreg stlh c=0
                cpi r24,0 ;menguji status
                breq lup8
                out sreg,r16;ambil sreg, set c
                sec; C = 1
                in r16,sreg; amankan sreg stlh c=1
lup8:           out sreg,r16;status akhir
                pop r24;kembalikan isi reg. dr stack
                pop r21
                pop r17
                pop r16
                ret
fk_ln:                  ldi r21,06; koreksi dgt.satuan
                out sreg,r16
                add r19,r21
                in r16,sreg
                brcc tuh
                ldi r21,0x01
                add r17,r21
```

```
tuh:    out sreg,r16
        adc r24,r24
        ret
fk_hn:  ldi r21,$60; koreksi dgt.puluhan
        out sreg,r16
        add r19,r21
        in r16,sreg
        adc r24,r24; amankan carry pd r24
        ret ; akhir dari prog. subrutin.
```

Hasil (R23:R22:R21):



Pembuktian:



BAB 5
MENCARI DATA TERKECIL DAN TERBESAR PADA ATMEGA8535

5.1 Teori Pendukung Mencari Data Terkecil dan Terbesar

Instruksi ADD & ADC

Penjumlahan pada Atmega8535 dilakukan dengan memanggil instruksi ADD, serta ADC untuk turut serta menambahkan carry yang terdapat pada flag dalam operasinya. Operand yang dapat digunakan dari kedua intruksi ini adalah R0 sampai dengan R31.

Syntax = ADD Rd, Rr ; Operasi = Rd ← Rd + Rr

Syntax = ADC Rd, Rr ; Operasi = Rd ← Rd + Rr + C

Instruksi LPM & LD

Instruksi LPM (Load Program Memory) digunakan untuk melakukan *load* dari memori dengan tujuannya adalah register pointer. LPM digunakan untuk mendapatkan nilai yang tepat dari setiap alamat program memori yang dideklarasikan.

Instruksi LD (Load from Data) digunakan untuk melakukan *load* dari memori, apabila LPM dari program area, maka LD dari data area.

5.2 Contoh Program Mencari Data Terkecil dan Terbesar

1. Buat program untuk mencari nilai terkecil dari 4 data biner 8-bit yang berada pada reg R6, R7, R8, R9. Simpan nilai data terkecil pada register R20.
2. Buat program untuk mencari nilai terkecil dari 4 data biner 16-bit yang berada pada reg R15:R14, R13:R12, R11:R10, R9:R8. Simpan nilai data terkecil pada register R20:R19.

3. Buat program untuk mencari nilai terbesar dari 16 data biner 8-bit yang berada pada reg R0 sampai R15. Simpan nilai data terbesar pada register R20.
4. Buat program untuk mencari nilai terbesar dari 16 data biner 8-bit yang berada pada program area dengan label DAT. Simpan nilai data terbesar pada register R20.
5. Buat program untuk mencari nilai terkecil dari 10 data biner 8-bit yang berada pada memori area dengan alamat awal $70. Simpan nilai data terkecil pada register R20.
6. Buat program untuk mencari nilai terkecil dari 10 data biner 16-bit yang berada pada program area dengan label DAT. Simpan nilai data terkecil pada register R20:R19.
7. Buat program untuk mencari nilai terkecil dari 10 data biner 16-bit yang berada pada unit memori dengan alamat awal $60. Simpan nilai data terkecil pada register R20:R19.
8. Buat program untuk menjumlahkan 4 data 7-bit yang tersimpan pada program area dengan label awal DAT1 dengan 4 data 7-bit lain pada program area dengan label DAT2. Simpan hasil penjumlahan pada memori unit mulai alamat $60.

JAWAB

1. Program:

```
.INCLUDE "M8535DEF.INC"
.ORG 0
//AMIR HUSEIN (181344003)
RJMP MAIN
MAIN:
LDI R16, LOW(RAMEND)
OUT SPL, R16
LDI R16, HIGH(RAMEND)
OUT SPH, R16

MOV R20, R6
MOV R21, R7
RCALL PINDAH
MOV R21, R8
RCALL PINDAH
MOV R21, R9
RCALL PINDAH

AKHIR: RJMP AKHIR

PINDAH:
CP R20, R21
BRLO BACK
MOV R20, R21
BACK: RET
```

Hasil:

* R6 = 9C; R7 = F5; R8 = 9B; R9 = AA

```
R04= 0x00  R05= 0x00
R06= 0x9C  R07= 0xF5
R08= 0x9B  R09= 0xAA
R10= 0x00  R11= 0x00
R12= 0x00  R13= 0x00
R14= 0x00  R15= 0x00
R16= 0x02  R17= 0x00
R18= 0x00  R19= 0x00
R20= 0x9B  R21= 0xAA
R22= 0x00  R23= 0x00
```

* R6 = AA; R7 = 18; R8 = E1; R9 = C0

```
R04= 0x00  R05= 0x00
R06= 0xAA  R07= 0x18
R08= 0xE1  R09= 0xC0
R10= 0x00  R11= 0x00
R12= 0x00  R13= 0x00
R14= 0x00  R15= 0x00
R16= 0x02  R17= 0x00
R18= 0x00  R19= 0x00
R20= 0x18  R21= 0xC0
R22= 0x00  R23= 0x00
```

```
.INCLUDE "M8535DEF.INC"
.ORG 0

//INPUT (R15:R14)(R13:R12)(R11:R10)(R9:R8)
//OUTPUT (R20:R19)

RJMP MAIN
MAIN:
LDI R16, LOW(RAMEND)
OUT SPL, R16
LDI R16, HIGH(RAMEND)
OUT SPH, R16

//MSB
MOV R22, R15
MOV R21, R13
RCALL CYK
MOV R21, R11
RCALL CYK
MOV R21, R9
RCALL CYK

//PINDAH
MOV R20, R22

//LSB
MOV R22, R14
MOV R21, R12
RCALL CYK
MOV R21, R10
RCALL CYK
MOV R21, R8
RCALL CYK
```

```
//PINDAH
MOV R19, R22

AKHIR: RJMP AKHIR

CYK:
CP R22, R21
BRSH CONV
TER: RET

CONV:
MOV R22, R21
RJMP TER
```

Hasil:

* R15:R14 = 4AFF; R13:R12 = FFFE, R11:R10 = 2A43, R9:R8 = 4AFE

```
R06= 0x00  R07= 0x00
R08= 0xFE  R09= 0x4A
R10= 0x43  R11= 0x2A
R12= 0xFE  R13= 0xFF
R14= 0xFF  R15= 0x4A
R16= 0x02  R17= 0x00
R18= 0x00  R19= 0x43
R20= 0x2A  R21= 0xFE
R22= 0x43  R23= 0x00
R24= 0x00  R25= 0x00
```

3. Program:

```
.INCLUDE "M8535DEF.INC"
.ORG 0

RJMP MAIN
MAIN:
LDI R16, LOW(RAMEND)
OUT SPL, R16
LDI R16, HIGH(RAMEND)
OUT SPH, R16
//INISIALISASI POINTER
LDI R30, 0X0001
LDI R31, 0

MOV R20, R0
RCALL CYB
AKHIR: RJMP AKHIR

CYB:
LD R25, Z+
CP R20, R25
BRSH SKIP
MOV R20, R25
SKIP:
CPI R30, $0F
BREQ AKHIR
RJMP CYB
```

Hasil:

```
* R00= 0xDD  R01= 0xA4
  R02= 0x55  R03= 0x70
  R04= 0x7F  R05= 0xE6
  R06= 0xD1  R07= 0xCD
  R08= 0xC9  R09= 0xC0
  R10= 0x0C  R11= 0x01
  R12= 0x02  R13= 0x0A
  R14= 0x6A  R15= 0x32
  R16= 0x02  R17= 0x00
  R18= 0x00  R19= 0x00
  R20= 0xE6  R21= 0x00
```

```
* R00= 0x32  R01= 0x53
  R02= 0x64  R03= 0x82
  R04= 0xBB  R05= 0xA6
  R06= 0x0C  R07= 0x54
  R08= 0x91  R09= 0x79
  R10= 0x89  R11= 0x99
  R12= 0x90  R13= 0x16
  R14= 0x00  R15= 0xAE
  R16= 0x02  R17= 0x00
  R18= 0x00  R19= 0x00
  R20= 0xBB  R21= 0x00
```

4. Program:

```
.INCLUDE "M8535DEF.INC"
.ORG 0

RJMP MAIN
MAIN:
LDI R16, LOW(RAMEND)
OUT SPL, R16
LDI R16, HIGH(RAMEND)
OUT SPH, R16

//INISIALISASI POINTER
LDI ZL, LOW(2*DAT)
LDI ZH, HIGH(2*DAT)
//INISIALISASI COUNTER
LDI R19,0
LPM R20, Z+
RCALL OPR

AKHIR: RJMP AKHIR

OPR:
LPM R21, Z+
CP R20, R21
BRSH SKIP
MOV R20, R21
SKIP:
INC R19
CPI R19, $0F
BREQ AKHIR
RJMP OPR

.ORG $80
DAT: .DB $43,$22,$F3,$A4,$9C,$36,$37,$34,$2A,$10,$11,$12,$CC,$A1,$E1
```

Hasil:

* Input = 43; 22; F3; A4; 9C; 36; 37; 34; 2A; 10; 11; 12; CC; A1; E1

```
R10= 0x00  R11= 0x00
R12= 0x00  R13= 0x00
R14= 0x00  R15= 0x00
R16= 0x02  R17= 0x00
R18= 0x00  R19= 0x0F
R20= 0xF3  R21= 0x00
R22= 0x00  R23= 0x00
R24= 0x00  R25= 0x00
R26= 0x00  R27= 0x00
```

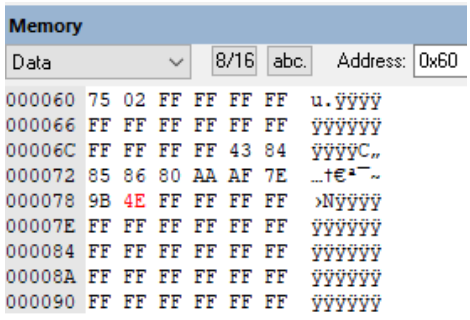
* Input = 43; 4A; 3A; 2A; 11; 4D; 8C; 9A; 9E; 77; 76; 75; 12; 13; 01

```
R10= 0x00  R11= 0x00
R12= 0x00  R13= 0x00
R14= 0x00  R15= 0x00
R16= 0x02  R17= 0x00
R18= 0x00  R19= 0x0F
R20= 0x9E  R21= 0x00
R22= 0x00  R23= 0x00
R24= 0x00  R25= 0x00
R26= 0x00  R27= 0x00
```

5. Program:

Hasil:

*



```
.INCLUDE "M8535DEF.INC"
.ORG 0

RJMP MAIN
MAIN:
LDI R16, LOW(RAMEND)
OUT SPL, R16
LDI R16, HIGH(RAMEND)
OUT SPH, R16

//INISIALISASI POINTER
LDI ZL, LOW($70)
LDI ZH, HIGH($70)
//INISIALISASI COUNTER
LDI R19,0
LD R20, Z+
RCALL OPR

AKHIR: RJMP AKHIR

OPR:
LD R21, Z+
CP R20, R21
BRLO SKIP
MOV R20, R21
SKIP:
INC R19
CPI R19, $0A
BREQ AKHIR
RJMP OPR
```

```
R16= 0x02  R17= 0x00
R18= 0x00  R19= 0x0A
R20= 0x43  R21= 0xFF
R22= 0x00  R23= 0x00
```

*

Memory

Data | 8/16 | abc. | Address: 0x60

```
000060 75 02 FF FF FF FF  u.ÿÿÿÿ
000066 FF FF FF FF FF FF  ÿÿÿÿÿÿ
00006C FF FF FF FF 75 FF  ÿÿÿÿuÿ
000072 32 96 09 FE CF FC  2-.þÏü
000078 16 32 FF FF FF FF  .2ÿÿÿÿ
00007E FF FF FF FF FF FF  ÿÿÿÿÿÿ
000084 FF FF FF FF FF FF  ÿÿÿÿÿÿ
00008A FF FF FF FF FF FF  ÿÿÿÿÿÿ
```

```
R16= 0x02  R17= 0x00
R18= 0x00  R19= 0x0A
R20= 0x09  R21= 0xFF
R22= 0x00  R23= 0x00
```

6. Program:

```
RJMP MAIN
MAIN:
LDI R16, LOW(RAMEND)
OUT SPL, R16
LDI R16, HIGH(RAMEND)
OUT SPH, R16

//INISIALISASI POINTER
LDI ZL, LOW(2*DAT)
LDI ZH, HIGH(2*DAT)
//INISIALISASI COUNTER
LDI R18,0
LPM R19, Z+
LPM R20, Z+
RCALL OPR

AKHIR: RJMP AKHIR

OPR:
CPI R18, $0A
BREQ AKHIR
INC R18
LPM R21, Z+
LPM R22, Z+
//CEK PULUHAN
CP R20, R22
BREQ CEKSAT
BRLO OPR
MOV R20, R22
MOV R19, R21
RJMP OPR

CEKSAT:
CP R19, R21
BRLO OPR
MOV R19, R21
RJMP OPR

.ORG $80
DAT: .DW $4300,$2200,$F303,$A454,$9C63,$3653,$10F0,$3424,$2AFF,$10F4,$11AF,$12B2,$CC3D,$A14B,$E1E3
```

Hasil:

* Input seperti pada program

```
R16= 0x02  R17= 0x00
R18= 0x0A  R19= 0xF0
R20= 0x10  R21= 0xAF
R22= 0x11  R23= 0x00
```

* Input =

```
DAT: .DW $64F0,$2210,$53FC,$1111,$932D,$9999,$1777,$1453,$8110,$A4DE,$184D,$999A,$43FC,$4A8B,$AAAA
```

```
R16= 0x02  R17= 0x00
R18= 0x0A  R19= 0x11
R20= 0x11  R21= 0x4D
R22= 0x18  R23= 0x00
```

7. Program:

```
.INCLUDE "M8535DEF.INC"
.ORG 0
//AMIR HUSEIN
RJMP MAIN
MAIN:
LDI R16, LOW(RAMEND)
OUT SPL, R16
LDI R16, HIGH(RAMEND)
OUT SPH, R16

//INISIALISASI POINTER
LDI ZL, LOW($60)
LDI ZH, HIGH($60)
//INISIALISASI COUNTER
LDI R18,0
LD R19, Z+
LD R20, Z+
RCALL OPR

AKHIR: RJMP AKHIR

OPR:
CPI R18, $0A
BREQ AKHIR
INC R18
LD R21, Z+
LD R22, Z+
//CEK PULUHAN
CP R20, R22
BREQ CEKSAT
BRLO OPR
MOV R20, R22
MOV R19, R21
RJMP OPR

CEKSAT:
CP R19, R21
BRLO OPR
MOV R19, R21
RJMP OPR
```

Hasil:

* Memory

Data | 8/16 | abc. | Address: 0x60

```
000060 43 18 FE B3 64 CC  C.þ³dÌ
000066 1B 02 18 4A 43 BB  ...JC»
00006C 10 10 C8 C9 12 16  ..ÈÉ..
000072 32 96 FF FF FF FF  2-ÿÿÿÿ
000078 FF FF FF FF FF FF  ÿÿÿÿÿÿ
00007E FF FF FF FF FF FF  ÿÿÿÿÿÿ
000084 FF FF FF FF FF FF  ÿÿÿÿÿÿ
00008A FF FF FF FF FF FF  ÿÿÿÿÿÿ
```

```
R16= 0x02  R17= 0x00
R18= 0x0A  R19= 0x1B
R20= 0x02  R21= 0xFF
R22= 0xFF  R23= 0x00
```

* Memory

Data | 8/16 | abc. | Address: 0x60

```
000060 84 77 9B AB AC CD  „w›«¬Í
000066 CF EA 91 71 64 BB  Ïê'qd»
00006C A0 DB AA C9 54 16   Û²ÉT.
000072 32 14 FF FF FF FF  2.ÿÿÿÿ
000078 FF FF FF FF FF FF  ÿÿÿÿÿÿ
00007E FF FF FF FF FF FF  ÿÿÿÿÿÿ
000084 FF FF FF FF FF FF  ÿÿÿÿÿÿ
00008A FF FF FF FF FF FF  ÿÿÿÿÿÿ
```

```
R16= 0x02  R17= 0x00
R18= 0x0A  R19= 0x32
R20= 0x14  R21= 0xFF
R22= 0xFF  R23= 0x00
```

8. Program:

```
.INCLUDE "M8535DEF.INC"
.ORG 0
//AMIR HUSEIN (181344003)
RJMP MAIN
MAIN:
LDI R16, LOW(RAMEND)
OUT SPL, R16
LDI R16, HIGH(RAMEND)
OUT SPH, R16

//POINTER INITIALIZATION
LDI R18, 1
//PROCESS DAT1 FIRST
LDI ZL, LOW(2*DAT1)
LDI ZH, HIGH(2*DAT1)

LPM R20, Z+
RCALL JUMLAH

//THEN PROCESS DAT2
LDI R18, 0
LDI ZL, LOW(2*DAT2)
LDI ZH, HIGH(2*DAT2)
RCALL JUMLAH

//STORE RESULT TO DATA AREA
LDI ZL, LOW(HASIL)
LDI ZH, HIGH(HASIL)
ST Z+, R20
ST Z+, R21
```

```
AKHIR: RJMP AKHIR

JUMLAH:
LPM R19, Z+
ADD R20, R19
ADC R21, R17
INC R18
CPI R18, $4
BRLO JUMLAH
RET

.ORG $90
DAT1: .DB $7F, $4A, $44, $5F

.ORG $96
DAT2: .DB $2D, $55, $77, $10

.ORG $60
HASIL: .DB 0
```

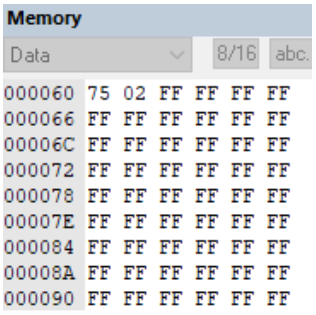
Hasil:

* Input =

```
.ORG $90
DAT1: .DB $7F, $4A, $44, $5F

.ORG $96
DAT2: .DB $2D, $55, $77, $10
```

Output =

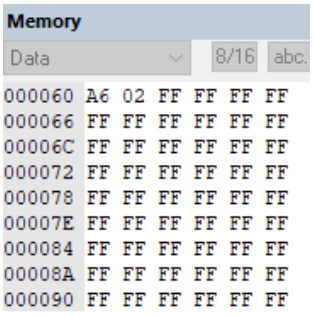


* Input =

```
.ORG $90
DAT1: .DB $1E, $32, $7A, $77

.ORG $96
DAT2: .DB $54, $68, $69, $40
```

Output =



ANALISIS

1. Pada program ini, akan dicari nilai terkecil dari data 8-bit yang ada pada register R6 sampai R9. Langkah awal yang perlu dilakukan ialah memindahkan R6 ke R20, dimana R20 disini akan bertindak sebagai register operasi perbandingan dan akan menyimpan hasil akhirnya juga nanti. R21 akan diisi oleh R7, R8 dan R9 berturut-turut dengan subrutin PINDAH,

dimana didalamnya akan ada perbandingan instruksi CP antara R20 dan R21, bila R20 lebih kecil maka program akan RETURN, sedangkan bila tidak, R20 akan diisi oleh R21. Subrutin ini akan terus dipanggil sampai dengan memindahkan R9 sebagai register input yang terakhir.

2. Pada program ini, data yang diproses merupakan data 4 digit atau 16-bit biner. Konsep untuk menemukan nilai terkecil adalah dengan membandingkan 8-bit MSB dari semua data dahulu. Register yang akan digunakan untuk menampung data perbandingan sementara adalah register R22, dimana proses membandingkan data ada pada subrutin CYK yang akan dipanggil setiap kali ingin membandingkan data sesuai banyaknya data input. Setelah data 8-bit MSB dibandingkan, barulah membandingkan LSB nya.
3. Dalam menyelesaikan program ini, akan digunakan pointer Z sebagai penunjuk alamat register yang akan terus bertambah sampai alamat untuk R15. Batas peningkatan ini akan diatur oleh instruksi CPI pada register R30 yang ketika menyentuh nilai 0FH, maka program akan exit. Pada mulanya, data pada R0 akan dipindahkan ke R20, yang kemudian R20 ini akan dibandingkan dengan R25 yang akan menyimpan data pada register dengan bantuan perintah LD secara terus menerus. Ketika R20 < R25, maka R20 akan diisikan R25, sebaliknya bila tidak, maka program akan loncat ke label SKIP untuk R25 mengambil data dari register yang sudah di point oleh pointer Z.
4. Program ini sebenarnya sama dengan program pada soal sebelumnya yaitu nomor 3. Perbedaannya terletak pada data yang akan dibandingkan bukan dari register, melainkan dari program area dengan label DAT, dengan alamat awal $80. Pointer yang akan digunakan untuk menunjuk ke alamat program area adalah pointer Z. Register R19 akan bertindak sebagai counter untuk membatasi operasi sampai 15 kali. Register yang akan dibandingkan adalah R20 dnegan R21, dimana R21 nilainya akan berganti sesuai jumlah data yang ada. Ketika R20 >= R21, maka program akan loncat ke label SKIP untuk meningkatkan counter +1, sedangkan ketika R20 < R21, maka nilai R21 akan dipindahkan ke R20.

5. Pada program ini, konsep dalam membandingkan data sama seperti soal sebelumnya, apabila pada soal sebelumnya dicari nilai terbesar, maka pada soal ini dicari nilai terkecil. Data yang akan dicek juga berasal dari data area, sehingga diperlukan pointer Z sebagai penunjuk yang dibantu oleh perintah LD (Load Data).
6. Pada program ini, tetap digunakan pointer Z untuk menunjuk pada alamat program area berlabel DAT, dimana R18 bertindak sebagai counter. Hal yang perlu dilakukan dahulu ialah membandingkan nilai MSB 8-bit dahulu, dengan mengambil dari program area melalui instruksi LPM. R20 akan menampung nilai puluhan sementara, dan R19 akan menampung nilai satuan. R22 dan R21 adalah register yang akan terus diisi oleh data input untuk dibandingkan dengan R20:R19. R20 akan dibandingkan dahulu dengan R22, ketika R20 lebih besar, maka nilainya diisi oleh R22 yang kemudian otomatis R19 diisi oleh R21. Apabila nilainya lebih kecil maka program akan mengulang mengisi data ke R22:R21, sedangkan bila R20==R22, akan dilakukan cek satuan pada R19 dengan R21.
7. Program ini persis sama dengan program sebelumnya pada nomor 6. Perbedaannya ada pada data input yang berasal dari data area dengan alamat awal $60, sehingga pointer Z perlu diarahkan pada alamat tersebut, serta proses pengambilan data memanfaatkan instruksi LD, bukan LPM. Secara konsep perbandingan dan lainnya, sama seperti soal sebelumnya.
8. Pada program ini, akan dilakukan proses penjumlahan data pada program area berlabel DAT1 dan DAT2 yang masing-masing memiliki 4 data 7-bit biner, sehingga data terbesar yang memungkinkan adalah 7FH. Penjumlahan yang akan diproses terlebih dahulu adalah data yang ada pada DAT1, sehingga alamatnya ditunjuk diawal oleh pointer Z. Proses penjumlahan ini ada pada subrutin JUMLAH, dimana R21 menyimpan carry dari penjumlahan, dan R20 menyimpan 4-bit LSB, R19 akan selalu mengambil data dari pointer Z, dan dijumlahkan dengan R20. Setelah proses penjumlahan pada DAT1 selesai, barulah pointer Z akan menunjuk alamat DAT2 untuk diambil datanya dan

kemudian dijumlahkan dengan data hasil penjumlahan DAT1. R18 akan bertindak sebagai counter, dimana saat memproses DAT1, R18 diisi 1, karena R20 sudah diisi data dari DAT1 diluar subrutin JUMLAH, dan saat memproses DAT2, R18 diinisialisasi 0. R18 ini akan dicek dengan $4, ketika nilainya telah menyentuh 4, maka program akan dikembalikan ke MAIN.

BAB 6

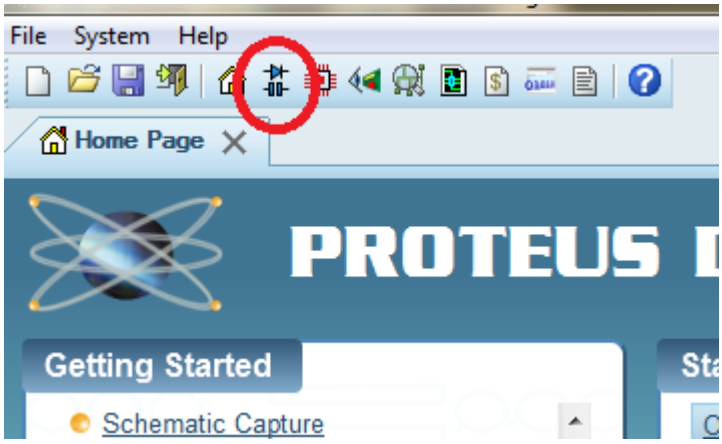
PENGGUNAAN PROTEUS UNTUK SIMULASI ATMEGA8535

6.1 Langkah-langkah Menggunakan Proteus untuk Simulasi

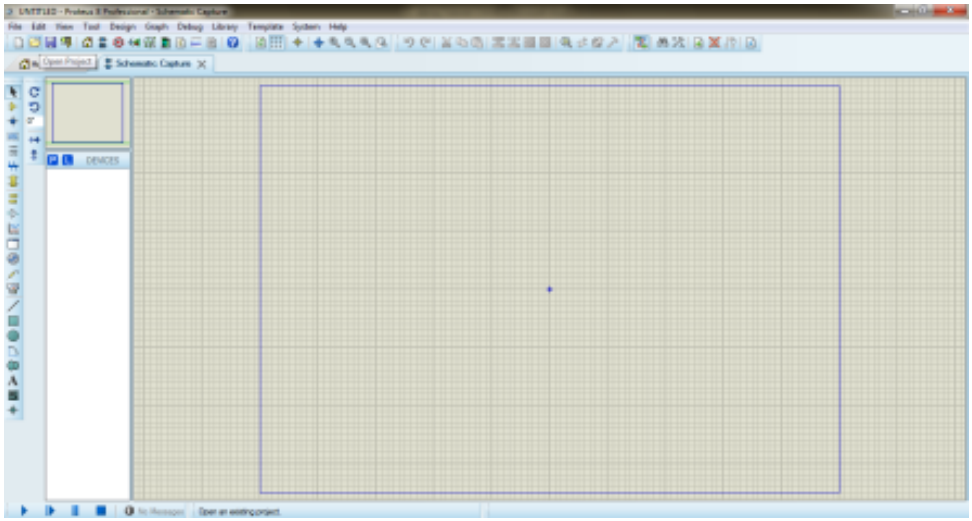
Proteus dan AVR adalah dua buah software yang familiar untuk dunia microcontroler. Proteus sebagai software gambar elektronik sekaligus simulasi dan CodeVision AVR sebagai software programing.

1. Pembuatan Rangkaian Skematik pada Software Proteus

- Pilih menu schematic pada proteus



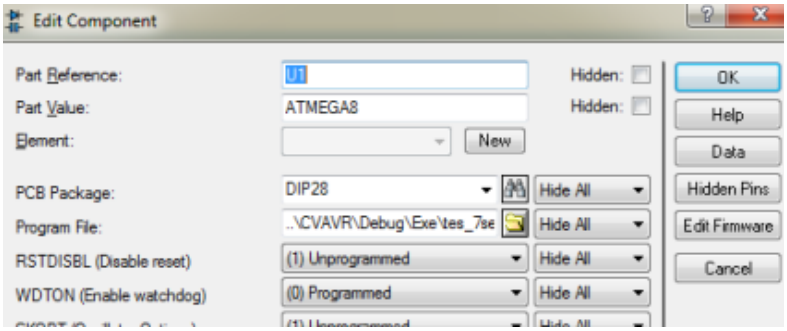
- Setelah itu akan muncul worksheet, disinilah kita bisa memulai membuat rangkaian sesuai yang kita inginkan.



- Gambar sampai schematic selesai dan save

2. Membuat Program pada AVR Atmega 8535
   - Langkah awal adalah membuka aplikasi AVR Studio
   - Klik New Project dan letakan file program pada destinasi yang diinginkan
   - Buat program untuk menyelesaikan masalah-masalah yang diberikan
   - Setelah dibuat, eksekusi program tersebut

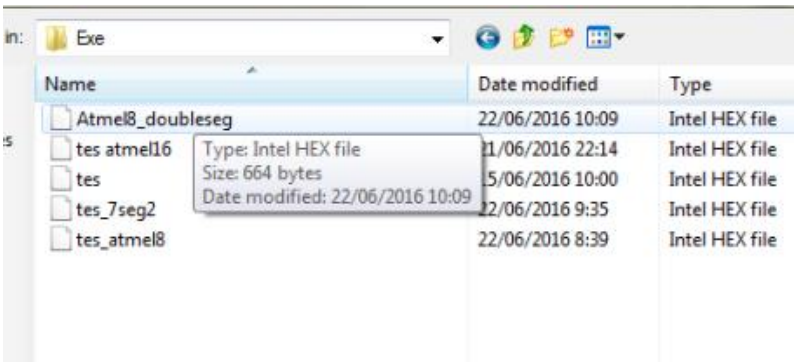
3. Mengaplikasikan Hasil Program AVR pada Rangkaian di Software Proteus
   - Memasukan program CVAVR double klik pada komponen IC schematik dan pada program file klik tanda open/ browse.



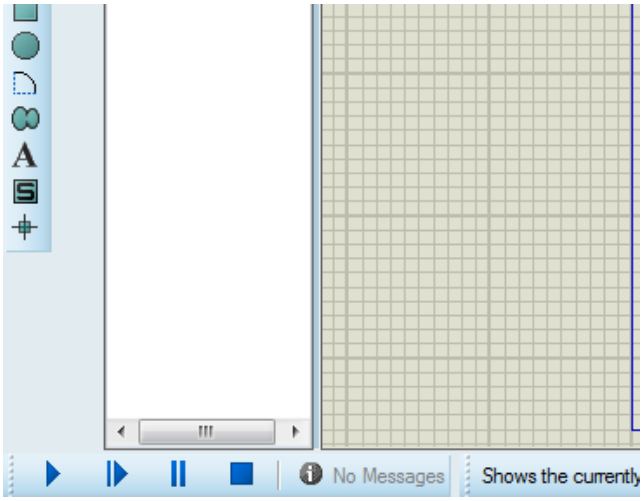
   - Masuk folder tempat menyimpan program AVR



   - Setelah itu pilih file program .hex yang akan diaplikasikan di rangkaian setelah itu tekan open dan terakhir jika sudah kembali ke proteus tekan OK



- Rangkaian beserta programnya telah dibuat, untuk mecoba apakah program sudah berhasil sesuai keinginan tekan play pada kiri bawah software proteus.

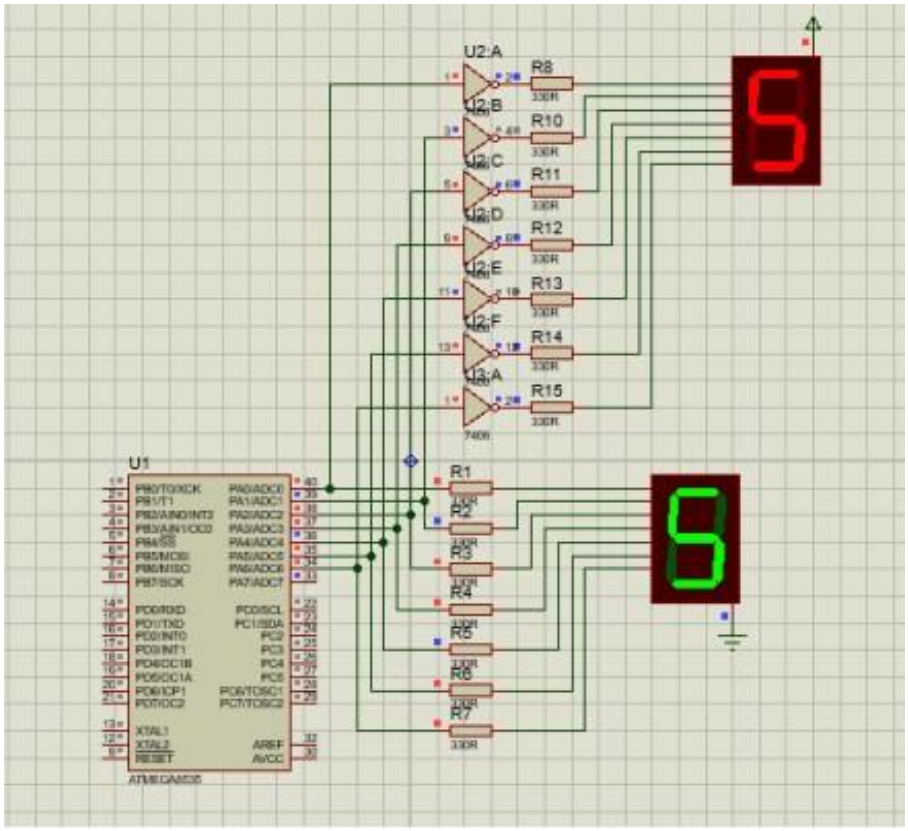


6.2 Contoh Program Penggunaan Proteus untuk Simulasi Atmega8535

Program untuk menampilkan angka lima pada 7 Segment common katoda di aplikasi proteus menggunakan program dari Atmega 8535.

```
.include"m8535def.inc"  //file definisi jenis  microcontroller
.org 0 //set alamat awal (original)
; PORT A TERHUBUNG KE SEGMENT A SAMPAI G DARI 7 SEGMENT
; “COMMON CATHODE”
RJMP MAIN
MAIN: LDI R20, LOW(RAMEND) //lokasi akhir RAM untuk stack(SP)
OUT SPL, R20 //LSB
LDI R20, HIGH(RAMEND)
```

```
OUT SPH, R20 //MSB
LDI R20,0X0FF
OUT DDRA,R20  //port A sebagai output
SANA: LDI R21, $B6  ; MENAMPILKAN ANGKA 5
        OUT PORTA,R21
        RCALL JEDA
        LDI R21,0
        OUT PORTA,R21
        RCALL JEDA
RJMP SANA
AKHIR: RJMP AKHIR
;==================================-=
JEDA: PUSH R20 ; SUBRUTIN DELAY SEKITAR 1 DETIK
        PUSH R21        ; MENGGUNAKAN 3 REGISTER
        PUSH R22
        LDI R20,3
L11:    LDI R21,0
L22:    LDI R22,0
L33:    DEC R22
BRNE L33
DEC R21
BRNE L22
DEC R20
BRNE L11
POP R22
POP R21
POP R21
```

RET
Analisa dari Program ini kita diperintahkan untuk menampilkan 7 Segment common katoda pada proteus menggunakan program dari Atmega 8535. Dengan memasukkan Hexadesimal B6 ke register *R21* lalu memindahkannya ke *OUTPUT Port A* maka akan tampil angka 5 pada output 7 Segment pada proteus. Setelah itu di tambahkan subrutin *JEDA* agar terdapat delay 1 detik pada saat perubahan 7 segment dari tampil angka 5 terhadap 7 segment kondisi mati
Design Rangkaian pada PROTEUS: Sebagai Port Output, dipilih PortA yang terhuhung ke Unit peraga OUTPUT “7 Segment”. Ditunjukkan bagaimana menghubungkan 7 Segment jenis “Common Cathode” (Bawah, warna Hijau), dan 7 Segment jenis “Common Anode” (Atas, warna Merah).   

BAB 7
BINARY COUNTER PADA ATMEGA 8535

7.1 Teori Binary Counter

Counter secara teori maupun praktek dalam melakukan penghitungan dapat bersifat naik dan turun (up-down counter), serta bisa di-reset sesuai dengan yang dikehendaki. Karena merupakan rangkaian yang komprehensif dengan komponen analog lain, maka jenis komponen IC digital yang digunakan adalah merupakan pengembangan dari komponen teknik digital pada pembelajaran elektronika dasar.

1. Counter Up

   Counter Up merupakan suatu rangkaian terpadu yang dirancang untuk digunakan dalam mencacah suatu bilangan decimal secara maju, atau dari bilangan decimal terkecil ke nilai yang terbesar sesuai dengan perintah masukan yang dikehendaki.
2. Counter Down

   Counter Down merupakan suatu rangkaian terpadu yang dirancang untuk digunakan dalam mencacah suatu bilangCan decimal secara mundur, atau dari bilangan decimal terbesar ke nilai yang terkecil sesuai dengan perintah masukan yang dikehendaki.

7.2 Contoh Program Binary Counter

1. Buat program untuk Up-Counter Modulo 12 heksadesimal menggunakan 7 segmen common cathode.

   Program:

```
.INCLUDE"M8535DEF.INC"
.ORG 0X000
RJMP MAIN
MAIN:       LDI R16,LOW(RAMEND)
            OUT SPL,R16
            LDI R16,HIGH(RAMEND)
            OUT SPH,R16

            LDI R16,0XFF
            OUT DDRA,R16 ; PORT A SEBAGAI OUTPUT
            LDI ZL,LOW(KAL<<1)
            LDI ZH, HIGH (KAL<<1)


KEDIP:      LDI R20,0
UP:         MOV R30,R20
            LPM R17,Z
            OUT PORTA, R17
            RCALL DELAY
            ; DIBAWAH ADALAH PROGRAM AGAR 7-SEGMEN BEKEDIP

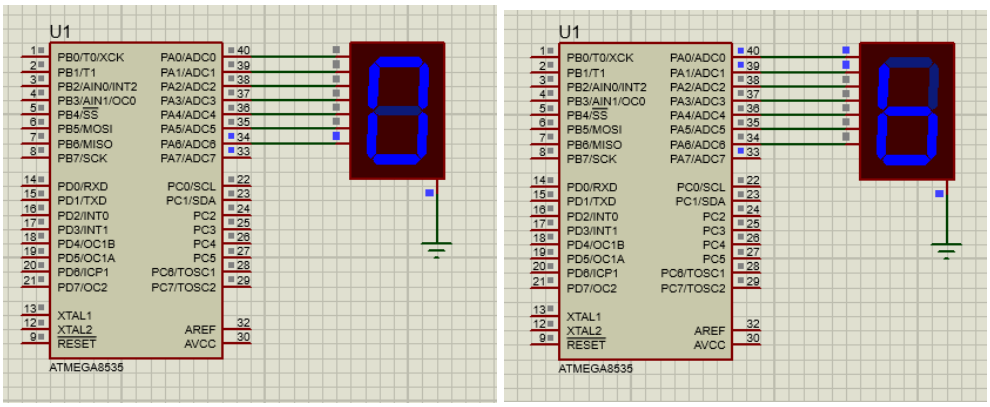
            INC R20
            CPI R20,0X0C
            BRLO UP
            RJMP DELAY

.ORG 0X0100
KAL:
.DB 0X3F, 0X06, 0X5B, 0X4F, 0X66, 0X6D, 0X7D,0X07 ; 0-7
.DB 0X7F, 0X6F, 0X77, 0X7C, 0X39, 0X5E, 0X79, 0X71 ; 8-F


DELAY:      LDI R23, 3
LUP1:       LDI R24, 0XFF
LUP2:       LDI R25,0
LUP3:       DEC R25
            BRNE LUP3
            DEC R24
            BRNE LUP2
            DEC R23
            BRNE LUP1
            RET
```

Program up counter pada AVR studio

Hasil :



Up Counter awal dan counter akhir

2. Buat program untuk Down-Counter modulo 14 heksadesimal dengan menggunakan 7 Segment common anode

Program :

```
.INCLUDE"M8535DEF.INC"
.ORG 0X000
RJMP MAIN
MAIN:       LDI R16,LOW(RAMEND)
            OUT SPL,R16
            LDI R16,HIGH(RAMEND)
            OUT SPH,R16

            LDI R16,0XFF
            OUT DDRA,R16 ; PORT A SEBAGAI OUTPUT
            LDI ZL,LOW(KAL<<1)
            LDI ZH, HIGH (KAL<<1)


KEDIP:      LDI R20,0XD
DOWN:       MOV R30,R20
            LPM R17,Z
            OUT PORTA, R17
            RCALL DELAY

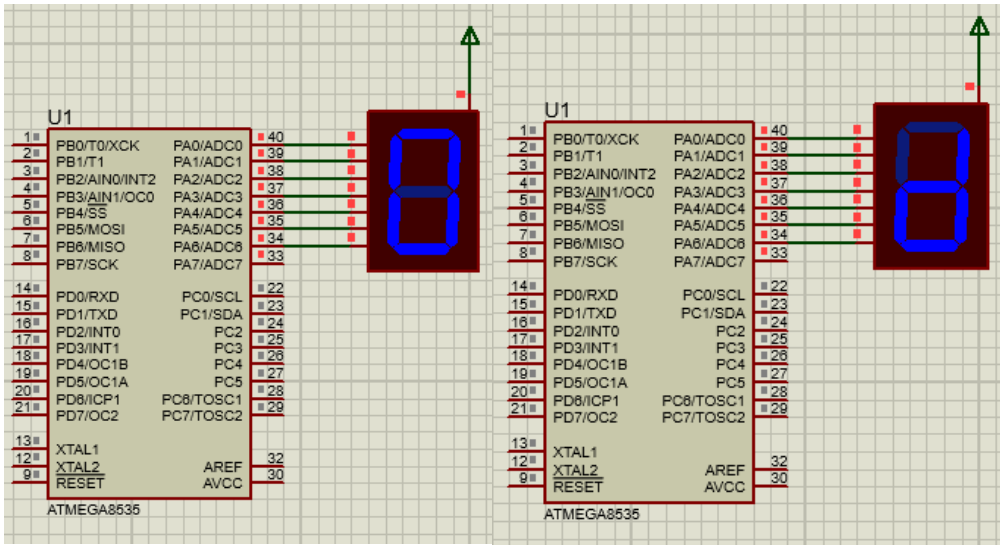
            DEC R20
            BRPL DOWN
            RJMP KEDIP

.ORG 0X0100
KAL:
.DB 0X0C0, 0X0F9, 0X0A4, 0X0B0, 0X99, 0X92, 0X82,0X0F8 ; 0-7
.DB 0X80, 0X90, 0X88, 0X83, 0X0C6, 0X0A1, 0X86, 0X8E ; 8-F


DELAY:      LDI R23, 3
LUP1:       LDI R24, 0XFF
LUP2:       LDI R25,0
LUP3:       DEC R25
            BRNE LUP3
            DEC R24
            BRNE LUP2
            DEC R23
            BRNE LUP1
            RET
```

Program Down counter pada AVR studio

Hasil :



Down Counter awal dan counter akhir

Analisis yang dapat dilakukan pada praktikum saat ini yaitu, program yang telah dibuat ditampilkan kedalam 7 segment yang dihubungkan langsung

dengan ATMega 8535 pada proteus. Data disimpan ada program area bersisi DefineByte yang berfungsi untuk menyimpan kode program untuk menampilkan bilangan heksadesimal 0 s.d F. data yang dimasukan sesuai dengan karaktersistik jenis 7-*segment* diuraikan pada tabel berikut :

Karakter	**7-Segment C-Annode**	**7-Segment C-Cathode**
0	C0	3F
1	F9	06
2	A4	5B
3	B0	4F
4	99	66
5	92	6D
6	82	7D
7	F8	07
8	80	7F
9	90	6F
A	88	77
B	83	7C
C	C6	39
D	A1	5E
E	86	79
F	8E	71

Pada program dibuat *up counter modulo 12* yang akan ditampilkan dalam 7-*segment.* Sehingga data ditampilkan mulai melakukan *counter* dimulai dari nilai 0 sampai dengan nilai B. nilai 0 sebagai *counter* awal dimasukan kedalam R20 kemudian data dipindahkan kedalam R30 sebagai register pointer Z LOW sehingga *counter* dan pointer register Z saling terhubung. Untuk menampilkan output pada PORT A, maka register R17 dimasukan kedalam PORT A dengan perintah "OUT PORTA,R17", perintah RCALL DELAY digunakan untuk memanggil subrutin yang sudah dibuat pada program selanjutnya dengan *define* subrutin DELAY. Untuk mengatur waktu jeda saat program melakukan *counter,* maka diberikan nilai 3 pada DELAY sehingga output akan ditampilkan dengan jeda 3 sebelum output menampilkan data selanjutnya. Untuk mengatur data menyala dan mati dengan selang waktu ½ sesuai dengan program nomor 3, maka ditambahkan beberapa baris program yaitu memberikan nilai 0X00 pada R17 sehingga 7-*segment* akan melakukan delay sebanyak nilai 0X00 yang diberikan.

BAB 8
INTERRUPT PADA ATMEGA 8535

8.1 Teori Interrupt

Interrupt adalah kondisi di mana pada saat program utama dieksekusi/dikerjakan oleh CPU kemudian tiba-tiba berhenti untuk sementara waktu karena ada rutin lain yang harus ditangani terlebih dahulu oleh CPU. Setelah selesai mengerjakan rutin tersebut, CPU Kembali mengerjakan instruksi pada program utama. Port yang berfungsi sebagai interrupt eksternal adalah:

PORTD.2 (PD2) => External Interrupt 0 Input (INT0)

PORTD.3 (PD3) => External Interrupt 1 Input (INT1)

PORTB.2 (PB2) => External Interrupt 2 Input (INT2)

Di dalam mikrokontroler AVR terdapat 21 interrupt baik eksternal maupun internal. Interrupt tersebut dapat dilihat dalam tabel berikut.

No.	Address(2)	Source	Interrupt Definition
1	0x000(1)	RESET	External Pin, Power-on Reset, Brown-out Reset and Watchdog Reset
2	0x001	INT0	External Interrupt Request 0
3	0x002	INT1	External Interrupt Request 1
4	0x003	TIMER2 COMP	Timer/Counter2 Compare Match
5	0x004	TIMER2 OVF	Timer/Counter2 Overflow
6	0x005	TIMER1 CAPT	Timer/Counter1 Capture Event
7	0x006	TIMER1 COMPA	Timer/Counter1 Compare Match A
8	0x007	TIMER1 COMPB	Timer/Counter1 Compare Match B
9	0x008	TIMER1 OVF	Timer/Counter1 Overflow
10	0x009	TIMER0 OVF	Timer/Counter0 Overflow
11	0x00A	SPI, STC	Serial Transfer Complete
12	0x00B	USART, RXC	USART, Rx Complete
13	0x00C	USART, UDRE	USART Data Register Empty
14	0x00D	USART, TXC	USART, Tx Complete
15	0x00E	ADC	ADC Conversion Complete
16	0x00F	EE_RDY	EEPROM Ready
17	0x010	ANA_COMP	Analog Comparator
18	0x011	TWI	Two-wire Serial Interface
19	0x012	INT2	External Interrupt Request 2
20	0x013	TIMER0 COMP	Timer/Counter0 Compare Match
21	0x014	SPM_RDY	Store Program Memory Ready

Register-register yang digunakan untuk interrupt meliputi:

a. MCU Control Register-MCUCR

Bit	7	6	5	4	3	2	1	0	
	SM2	SE	SM1	SM0	ISC11	ISC10	ISC01	ISC00	MCUCR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 3, 2 – ISC11, ISC10: Interrupt Sense Control 1 Bit 1 and Bit 0**

Table 35. Interrupt 1 Sense Control

ISC11	ISC10	Description
0	0	The low level of INT1 generates an interrupt request.
0	1	Any logical change on INT1 generates an interrupt request.
1	0	The falling edge of INT1 generates an interrupt request.
1	1	The rising edge of INT1 generates an interrupt request.

- **Bit 1, 0 – ISC01, ISC00: Interrupt Sense Control 0 Bit 1 and Bit 0**

Table 36. Interrupt 0 Sense Control

ISC01	ISC00	Description
0	0	The low level of INT0 generates an interrupt request.
0	1	Any logical change on INT0 generates an interrupt request.
1	0	The falling edge of INT0 generates an interrupt request.
1	1	The rising edge of INT0 generates an interrupt request.

b. MCU Control and Status Register-MCUCSR

Bit	7	6	5	4	3	2	1	0	
	–	ISC2	–	–	WDRF	BORF	EXTRF	PORF	MCUCSR
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	See Bit Description					

- **Bit 6 – ISC2: Interrupt Sense Control 2**

• ISC2 = 0, a falling edge on INT2 activates the interrupt

• ISC2 = 1, a rising edge on INT2 activates the interrupt

SREG (Status Register)

Digunakan untuk menyimpan informasi dan hasil operasi aritmatika terakhir. Data SREG selalu berubah setiap instruksi atau operasi pad ALU dan datanya tidak otomatis tersimpan apabila terjadi instruksi percabangan baik karena interupsi maupun lompatan.

Bit	7	6	5	4	3	2	1	0	
	I	T	H	S	V	N	Z	C	SREG
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit 7 – I : Global interrupt enable

Bit I digunakan untuk mengaktifkan interupsi secara umum (interupsi global). Jika bit I bernilai ‘1’ maka interupsi secara umum aktif, tetapi jika ‘0’ maka tidak satupun interupsi yang aktif.

d. General Interrupt Control Register-GICR

Bit	7	6	5	4	3	2	1	0	
	INT1	INT0	INT2	–	–	–	IVSEL	IVCE	GICR
Read/Write	R/W	R/W	R/W	R	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

• Bit 7 - INT1:eksternal interrupt request 1 enable

• Bit 6 – INT0 :eksternal interrupt request 0 enable

• Bit 5 – INT2 : eksternal interrupt request 2 enable

8.2 Contoh Program Interrupt

1. Program untuk Heksa-Desimal up-counter 3 digit (menghitung dari $100 sampai $30F) yang outputnya berubah 1/5 detik sekali. Tidak berulang.

   Program:

```
.include"m8535def.inc"
.org 0

RJMP MAIN
MAIN:    LDI R16,LOW(RAMEND)
         OUT SPL,R16
         LDI R16,HIGH(RAMEND)
         OUT SPH,R16

         ;SET PORT OUTPUT
         LDI R20,$FF
         OUT DDRA,R20
         OUT DDRC,R20

         LDI ZL,LOW(2*KAR)
         LDI ZH,HIGH(2*KAR)

         LDI R16,0
         LDI R17,1

S22:     LDI R24,12
         ;TENTUKAN POSISI DIGIT 1
S11:     LDI R19,4
         OUT PORTC,R19
         ;AMBIL DATA KARAKTER
         MOV ZL,R17
         LPM R18,Z
         COM R18
         ;OUTPUTKAN KARAKTER
         OUT PORTA,R18
         RCALL DELAY5ms
```

```
;TENTUKAN POSISI DIGIT 2
LDI R19,2
OUT PORTC,R19
MOV R19,R16
ANDI R19,$F0
SWAP R19
MOV ZL,R19
LPM R18,Z
COM R18
OUT PORTA,R18
RCALL DELAY5ms

;TENTUKAN POSISI DIGIT 3
LDI R19,1
OUT PORTC,R19
MOV R19,R16
ANDI R19,$0F
MOV ZL,R19
LPM R18,Z
COM R18
OUT PORTA,R18
RCALL DELAY5ms

DEC R24
BRNE S11
LDI R27,1
ADD R16,R27
ADC R17,R5
CPI R17,3
BRNE S22
CPI R16,0X10
BRNE S22
```

```
        LDI R27,0XFF
        OUT PORTA,R27

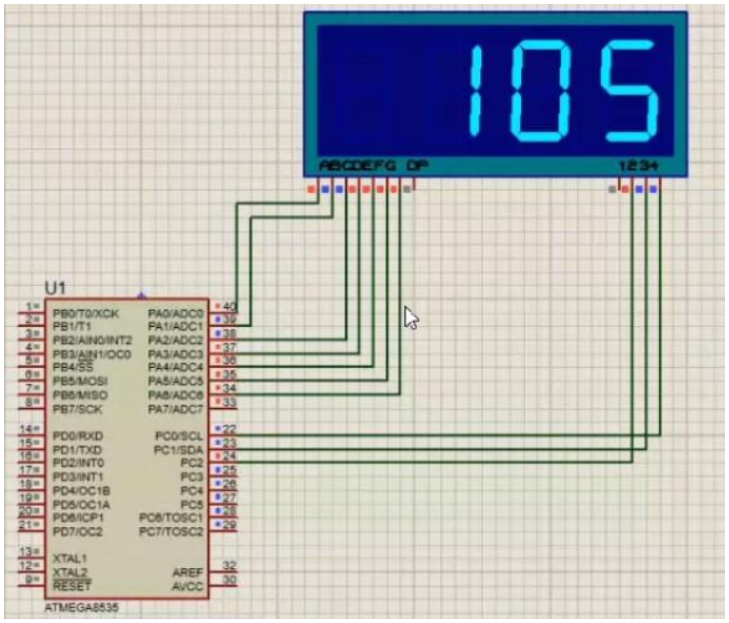
AKHIR:    RJMP AKHIR

DELAY5ms:    PUSH R21
             PUSH R22
             PUSH R23

             LDI R21,1
L21:         LDI R22,0X22
L22:         LDI R23,0
L23:         DEC R23
             BRNE L23
             DEC R22
             BRNE L22
             DEC R21
             BRNE L21

             POP R23
             POP R22
             POP R21
.org $80
KAR: .DB $3F , $06 , $5B , $4F , $66 , $6D , $7D ,$07, $7F, $6F
     .DB $77 , $7C , $39 , $5E , $79 , $71
```

Hasil:



Analisis Untuk membuat Program Heksa decimal up counter 3 digit diperlukan sebuah pointer yang digunakan untuk menginputkan nilai hexa decimal secara berurutan dari $100 sampai $30F. Pada program ini digunakan pointer Z , register R17 di isikan 1 dan R16 diisikan 0 di inputkan nilai untuk Counter, karena yang akan ditampilkan digit yang paling kiri maka R19 harus

diisi dengan 0X04 yang berarti mengaktifkan common 7Segment yang ke – 3 dari kanan setelah itu dikirim ke PORTC (ditentukan posisi digit terlebih dahulu lalu pola karakter yang akan ditampilkan pada 7Segment) ,
lalu digit pertama yang terdapat pada R17 harus diisikan ke ZL dengan menambahkan perintah MOV setelah itu diambil pola karakter dengan menggunakan sintaks LPM R18, Z maka data tersebut akan masuk ke R18, setelah itu harus dikirim ke port A untuk terhubung ke Seven segment menggunakan sintaks OUT PORTA,R18. Kemudian menentukan delay pada subrutin DELAY5ms, lalu mengisikan nilai 2 pada R19 untuk mengaktifakan digit yang ke -2, setelah itu mengambil digit ke 2 (tengah) yang ada pada R16 dengan menggunakan sintaks ANDI R19,0XF0 (sebelumnya R18 telah dipindahkan ke R19), SWAP R19 untuk memindahkan dari sebelah kiri ke sebelah kanan setelah itu dikirim ke ZL (MOV ZL,R19), lalu ambil data tersebut menggunakan LPM R18, Z. lalu dikirim ke PORTA lalu panggil subrutin DELAY5ms.Selanjutnya adalah untuk memasukkan digit ke – 1 , sama halnya dengan digit ke – 2 yaitu dengan mengubah LDI R19, 2 menjadi LDI R19, 1 karena digit yang akan dimasukan adalah yang pertama lalu di AND kan dengan 0x0F , tidak diperlukan SWAP karena posisi sudah benar pada digit paling kanan. Karena 1 digit = 60 Frame , maka program harus diulang sebanyak 60 kali, INC R16 lalu membandingkan dengan R16 dengan 0X00. Jika 00 maka R16 harus dinaikkan 1, jika bukan 0 maka R16 tidak akan dinaikkan. Kemudian jump BRNE S22 , lalu ADD R16, R27 dan ADC R17, R5 untuk memberikan carry ke R17 setelah nilai digit paling kanan (ke – 1) bernilai F maka R17 harus dinaikkan 1 .setelah dinaikkan 1 maka harus di uji pada R17 harus 03 menggunakan CPI R17,3 jika bukan 3 maka BRNE S22 untuk menampilkan kembali mengisikan R24,12 untuk mengulang 12 kali ditampilkan(digit ke-1,ke-2 dan ke-3 masing – masing diulang 12 kali). Jika sudah 12 kali maka akan mencapai 03. Jika R17 bernilai 03 maka setelah itu menguji R16 menggunakan sintaks CPI R16,10. Jika bukan maka jump ke S22 (BRNE S22) . lalu ditambahkan R27 diisikan FF , maka sevent segment

akan mati atau sebagai clear display. Pada Program area KAR berisikan nilai hexa decimal dari 0 sampai F tetapi untuk format seven segment common catode , akan tetapi pada proteus digunakan seven segment common anode, maka yang harus dilakukan adalah dengan menambahkan sintaks COM R18 sebelum sintaks pengiriman ke PORTA yaitu pada sintaks LPM R18,Z.

2. Program untuk Binary up-counter 5 bit yang outputnya berubah 1 detik sekali, menghitung berulang terus menerus. Gunakan unit peraga 7 segment sebagai output.

Program:

```
.INCLUDE"M8535DEF.INC"
.ORG 0
RJMP MAIN

MAIN :  LDI R20 , LOW(RAMEND)
        OUT SPL , R20
        LDI R20 , HIGH(RAMEND)
        OUT SPH , R20

        LDI R20 , $FF
        OUT DDRA , R20
        OUT DDRB , R20

        LDI ZL , LOW(2KAR)
        LDI ZH , HIGH(2KAR)

ULANG:  LDI R16 , 0
F1:     LDI R25 , 20

P0:     MOV R18 , R16
        LDI R17 , 1
        LDI R28 , 5

P1:     OUT PORTA , R17
        CLC
        LSL R17

        LSR R18
        LDI R20 , 0
        ADC R20 , R0
        MOV ZL , R20
        LPM R19 , Z
        OUT PORTB , R19
        RCALL DELAY
        DEC R28
        BRNE P1
```

```
        DEC R25
        BRNE P0

        INC R16
        MOV R18 , R16
        CPI R16 , $20
        BRNE F1

        RJMP ULANG

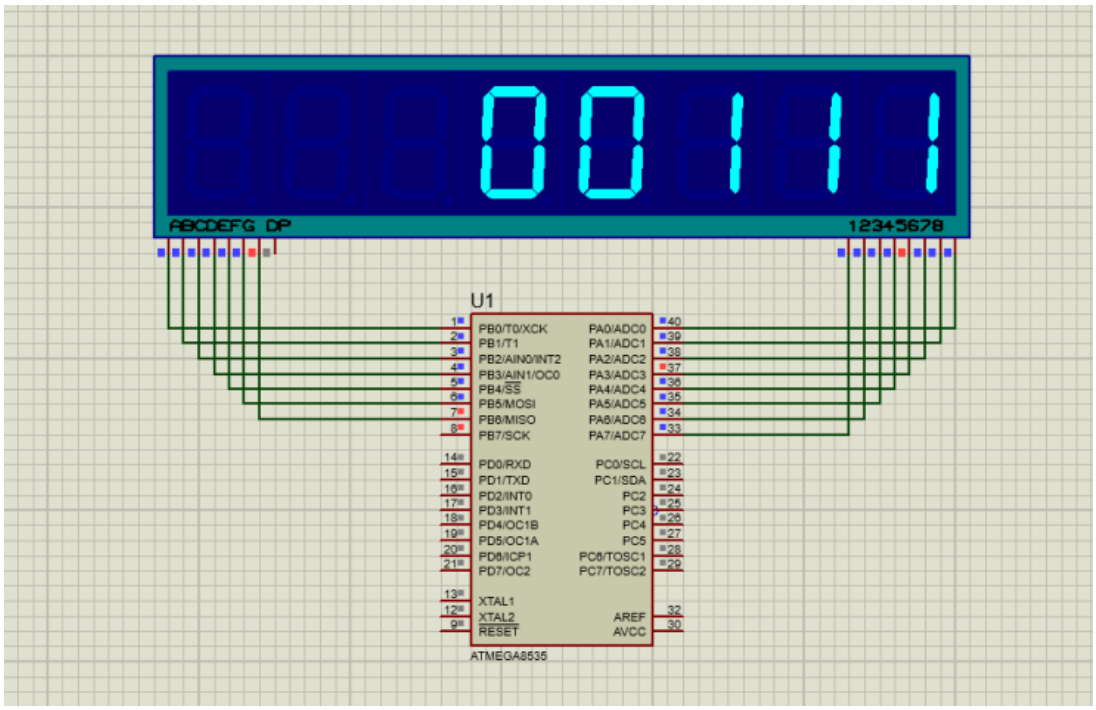
DELAY:      PUSH R21
            PUSH R22
            PUSH R23

            LDI R21,12
L21:        LDI R22,1
L22:        LDI R23,0
L23:        DEC R23
            BRNE L23
            DEC R22
            BRNE L22
            DEC R21
            BRNE L21

            POP R23
            POP R22
            POP R21
            RET

.ORG $80
KAR: .DB $C0 , $F9
```

Hasil:



Analisis untuk membuat up counter binary 5 bit yang berubah terus menerus ,

diperlukan pointer Z untuk mengkontrol proses perubahan binary pada Atmega.

LDI R17,1 sebagai counter setiap bit yaitu yang menentukan seven segment bernilai 0 atau 1.

LDI R28 sebagai counter jumlah 5 bit tersebut apabila telah bernilai 11111 maka akan Kembali ke 00000 untuk mengulangi program terus menerus.

OUT PORTA , R17 untuk output paling kanan setelah itu R17 digeser ke kiri dan R18 digeser kekanan untuk menghasilkan nilai yang berkelanjutan setelahnya.

ADC R20 yang menentukan nilai selanjutnya akan muncul dengan bantuan pointer Z lalu dikeluarkan ke PORTB yang mana akan mendeskripkan nilai 0 atau 1 yang akan muncul.

Setelahnya dipanggil subrutin DELAY agar terdapat jeda diantara perubahan kenaikan Up counter antara satu digit dengan digit lainnya.

DEC R28 counter dari 5 bit , apabila belum bernilai 0 maka akan Kembali jump ke P1, setelah itu mengurangi R25 yaitu counter dari nilai hexa decimal sebagai binary yaitu $20 atau dalam decimal 32 yang merupakan jumlah binary maksimal yang di up counter kan pada program ini.

INC R16 lalu di compare dengan $20 apabila telah mencapai nilai $20 maka jump ke label ULANG yang membuat program terus menerus mengoperasikan Up counter 5 bit tersebut.

3. Program untuk menampilkan 2 kata pada unit peraga 7 segmen (8 karakter) secara bergantian berulang, masing masing selama 4 detik. Kata pertama: “rAJInLAH”. Kata ke dua: “bELAJAr-“

   Program:

```
.INCLUDE"M8535DEF.INC"
.ORG 0
RJMP MAIN

MAIN:
        LDI R20 , LOW(RAMEND)
        OUT SPL , R20
        LDI R20 , HIGH(RAMEND)
        OUT SPH , R20

        LDI R20 , 0XFF
        OUT DDRA , R20
        OUT DDRB , R20

        LDI ZL , LOW(KAR2)
        LDI ZH , HIGH(KAR2)

ULANG:  LDI R16 , 0
        LDI R21 , 0
        LDI R22 , 0

F1:     LDI R25 , 140

P0:     LDI R16 , 7
        ADD R16 , R21
        MOV R18 , R16
        LDI R17 , 1
        LDI R28 , 8

P1:     OUT PORTA , R17
        CLC
        LSL R17

        MOV R20 , R18
        MOV ZL , R20
        LPM R19 , Z
        OUT PORTB , R19
        RCALL DELAY
        DEC R16

        MOV R18 , R16
        DEC R28
        BRNE P1

        DEC R25
        BRNE P0

        LDI R21 , 8
        INC R22
        CPI R22 , 2
        BRNE F1

        RJMP ULANG

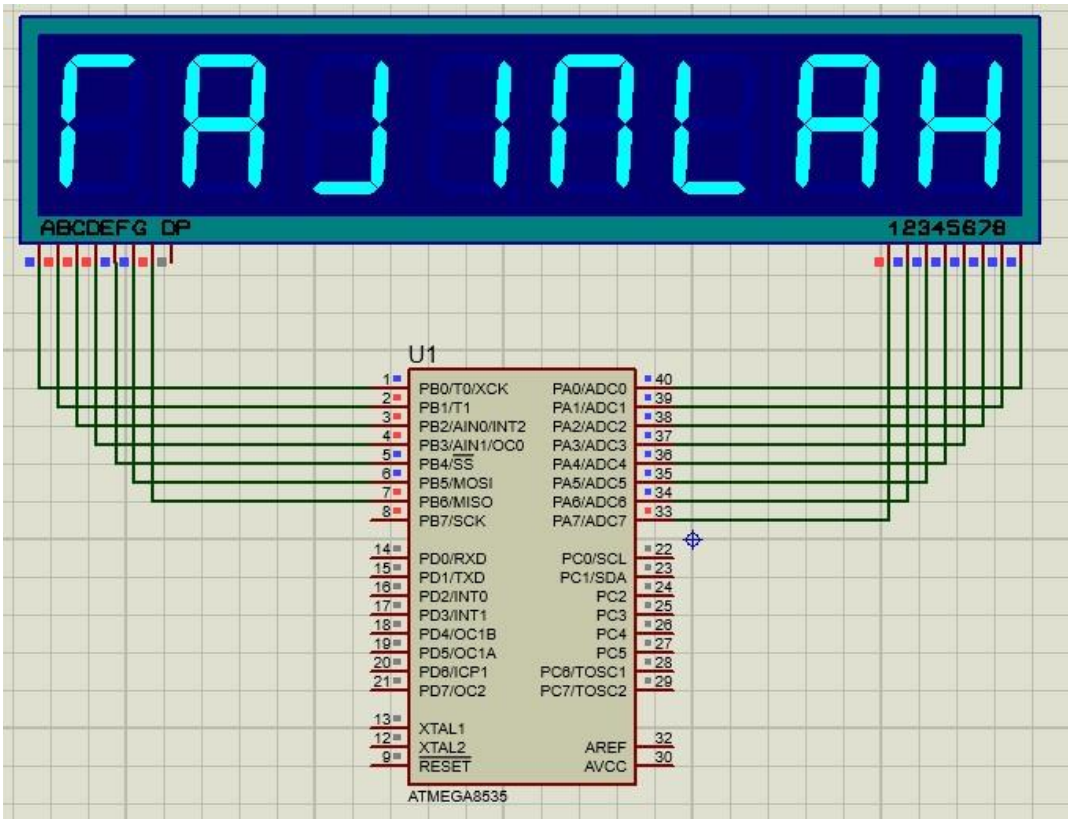
DELAY:      PUSH R21
            PUSH R22
            PUSH R23

            LDI R21,5
L21:        LDI R22,1
L22:        LDI R23,0
L23:        DEC R23
            BRNE L23
            DEC R22
            BRNE L22
            DEC R21
            BRNE L21

            POP R23
            POP R22
            POP R21
            RET

.ORG $80
KAR: .DB $CE , $88 , $F1 , $F9 , $C8 , $C7 , $88 , $89 ;RAJINLAH
     .DB $83 , $86 , $C7 , $88 , $F8 , $88 , $CE , $BF ;BELAJAR-
```

Hasil:



Analisis untuk menampilkan 2 kata pada unit peraga 7 segmen (8 karakter) secara bergantian berulang, masing- masing selama 4 detik. Kata pertama: “rAJInLAH”. Kata ke dua: “bELAJAr-“

Pertama, menentukan output yang akan digunakan pada Atmega8535 yaitu Port A dan Port B.

Port A sebagai Penerjemah bentuk seven segment yang akan ditampilkan.

Port B sebagai posisi agar output setiap bentuk huruf yang dikeluarkan sesuai denga apa yang diminta.

Digunakan Pointer Z sebagai penunjuk program area pada Label KAR yang akan memberikan data hexadecimal yang berisi nilai dari bentuk karakter dan posisi karakter.

Register R16,R21,R22 diberikan nilai 0 agar Ketika proses pengulangan akan berisikan blank.

LDI R25, 140 frame karakter yang akan ditampilkan perdetiknya. Lalu memposisikan agar Batasan perkata pada output berupa 8 karakter saja , maka dari itu diinputkan 8 pada R28.

setelah itu output pada PORTA huruf ke – 1 pada R17, setelah itu pointer akan menunjuk ke program area dan menerjemahkan karakter yang akan dimunculkan. Lalu keluaran portb pada posisi 1 karena Z menunjuk ke 1 lalu pada loop kedua menunjuk ke posisi ke 2 dan seterusnya.

RCALL DELAY diberikan jeda antara kemunculan kata ke - 1 menuju kata ke - 2

LDI R21,8 sebagai counter 8 karakter.

CPI,2 karena hanya menggunakan 2 kata yaitu:

Kata pertama: “rAJInLAH”.

Kata ke dua: “bELAJAr-“

R22 dicompare dengan 2 apabila telah 2 maka akan jump ke label ULANG yang membuat program terus menerus berulang.

BAB 9
POLLING INTERRUPT PADA ATMEGA 8535

9.1 Teori Sistem Polling dan Interrupt Polling

Sistem Polling dan Interupsi Polling sebenarnya bukan suatu fitur , ini adalah sesuatu yang harus dilakukan jika mikrokontroler yang dipilih tidak memiliki interupsi. Polling adalah teknik perangkat lunak dimana kontroler secara terus menerus menanyakan suatu perangkat jika membutuhkan servis.Perangkat membuat suatu tanda ketika data siap untuk ditransfer ke kontroler, dimana kontroler akan melihat pool berikutnya. Beberapa perangkat dapat dipolled dengan sukses, dengan kontroler yang meloncat kepada rutin program yang lain, tergantung pada flag mana yang telah diset.

Dasar dari polling adalah setiap fungsi memakai tipe round-robin untuk menanyakan ketika mereka dalam keadaan yang membutuhkan sebuah servis, kita dapat membuat mereka (prosedure/fungsi) memanggil fungsi mereka sendiri ketika prosedure tersebut membutuhkan penanganan lain. Ini disebut dengan “interupt”, ketika perangkat menginterupsi eksekusi program utama. Prosesor lalu akan mengambil waktu untuk keluar dari eksekusi program normal untuk menguji source interrupt dan mengambil aksi tertentu. Setelah itu, eksekusi program normal dilanjutkan. Sebuah servis interrupt dengan kata lain seperti sebuah sub-rutin, untuk melakukan perintah lain yang sebelumnya tidak dijalankan sehingga dapat diantisipasi oleh prosesor untuk menyesuaikan sebagian waktu, untuk mengeksekusi perintah baru dan menghentikan program utama yang kemudian dijalankan kembali jika tidak ada pemanggilan prosedur lain pada badan program.

9.2 Contoh Program Sistem Polling dan Interrupt Polling

Program dan Analisa untuk untuk menampilkan 4 kata (“rAJInLAH”,”-SELALU”,”SUnGGUH-”, dan ”BELAJAr-”) secara berulang, bergantian dikendalikan oleh Tombol Polling (suatu Push Button) pada unit peraga 7 segment

Common Anoda, yang terdiri dari 8 unit 7 segment. Pada saat kata “rAJInLAH” dan ”-SELALU-” ditampilkan, unit peraga Tidak Berkedip; tapi pada saat kata ”SUnGGUH-” dan ”BELAJAr-” ditampilkan, unit peraga berkedip 2 kali tiap detik.

PROGRAM	ANALISA
INCLUDE "M8535DEF.INC"	file definisi jenis microcontroller yang digunakan pada program untuk atmega8535
.ORG 0	Memulai stack 0 .kode program pada alamat 0x0000
RJMP MAIN	Loncat ke Label MAIN
MAIN:LDI R16,LOW(RAMEND) OUT SPL,R16 LDI R16,HIGH(RAMEND) OUT SPH,R16	Menentukan isi stack dengan address terakhir ram (RAMEND). Untuk Atmega 8535 yaitu 0x025F agar program utama mulai ditulis setelah address terakhir RAM
LDI R16,0B11111111 OUT DDRA,R16 OUT DDRB,R16	Pada statement ini, R16 dimasukan data Biner 11111111 atau pada bentuk hexadecimal adalah FF , lalu selanjutnya dikeluarkan pada DDRA dan DDRB. Maka port A dan Port B berfungsi sebagai PIN OUTPUT.
LDI R16,0B00000011 OUT DDRD,R16	Untuk menghemat Pin pada port D , digunakan biner 00000011 , maka, yang digunakan sebagai output port d adalah pin D0 dan D1
LDI ZL,LOW (2*KAR) LDI ZH,HIGH (2*KAR)	Statement untuk menggunakan Pointer Z sebagai penunjuk ke memori area dengan variabel KAR. Lalu dikali 2 untuk mengubah jenis data didalamnya karena setiap data yang ada masih berupa strings perlu dilakukan inisialisasi terlebih dahulu
L1: LDI R18,0	Statement pada Label L1 di lakukan untuk

LDI R17,80 RCALL DISPLAY	menampilkan kata “rAJInLAH” , R18 disikan 0 karena akan dilakukan pengambilan data pada program area di posisi 0 s/d 7 yang merupakan data hexadecimal seven segment common anoda dari kata “rAJInLAH” (terdapat pada landasan teori) yang di lakukan pada label DISPLAY. R17 diinputkan 80 dikarenakan waktu yg diminta adalah 4 detik sedangkan subrutin DELAY pada program memiliki jeda 2ms
RCALL CHECK CPI R29,0 BREQ L1	Memanggil subrutin CHECK, mengcompare R29 dengan 0 untuk counter yang akan dipicu pada rangkaian proteus Ketika nilai biner 00. Setelah push button di tekan maka nilai binner akan berubah menjadi 01 lalu program akan jump ke statement selanjutnya , jika belum maka program akan selalu looping, jump ke label L1
L2: LDI R18,8 LDI R17,10 RCALL DISPLAY	Statement pada Label L2 di lakukan untuk menampilkan kata “-SELALU-” , R18 disikan 8 karena akan dilakukan pengambilan data pada program area di posisi 8 s/d 15 yang merupakan data hexadecimal seven segment common anoda dari kata “-SELALU-” (terdapat pada landasan teori) yang di lakukan pada label DISPLAY. R17 diinputkan 10 untuk memberikan delay jeda yang pas pada saat tampilan berkedip di kata ke 3 dan ke 4
RCALL CHECK CPI R29,1 BREQ L2	Memanggil subrutin CHECK, mengcompare R29 dengan 1 untuk counter yang akan dipicu pada rangkaian proteus Ketika nilai biner 01. Setelah push button di tekan maka nilai biner akan berubah menjadi 10 lalu program akan jump ke

	statement selanjutnya , jika belum maka program akan selalu looping, jump ke label L2
L3: LDI R18,16 LDI R17,2 RCALL DISPLAY RCALL BLANK	Statement pada Label L3 di lakukan untuk menampilkan kata "-SUnGGUH-", R18 disikan 16 karena akan dilakukan pengambilan data pada program area di posisi 8 s/d 15 yang merupakan data hexadecimal seven segment common anoda dari kata "-SELALU-" (terdapat pada landasan teori) yang di lakukan pada label DISPLAY. R17 diinputkan 5 dikarenakan waktu yg diminta adalah 4 detik sedangkan subrutin DELAY pada program memiliki jeda 2ms dan juga karena terdapat penambahan label BLANK sebagai subrutin untuk mengedipkan kata pada kata "-SUnGGUH-".
RCALL CHECK CPI R29,2 BREQ L3	Memanggil subrutin CHECK, mengcompare R29 dengan 2 untuk counter yang akan dipicu pada rangkaian proteus Ketika nilai biner 10. Setelah push button di tekan maka nilai biner akan berubah menjadi 11 lalu program akan jump ke statement selanjutnya , jika belum maka program akan selalu looping, jump ke label L3
L4: LDI R18,24 RCALL DISPLAY RCALL BLANK	Statement pada Label L4 di lakukan untuk menampilkan kata "bELAJAr-", R18 disikan 24 karena akan dilakukan pengambilan data pada program area di posisi ke 24 s/d 31 yang merupakan data hexadecimal seven segment common anoda dari kata "bELAJAr-", (terdapat pada landasan teori)

	yang di lakukan pada label DISPLAY. Pemanggilan Blank untuk membuat tampilan kata berkedip
RCALL CHECK BRNE L4 RJMP L1	Memanggil subrutin CHECK, pada statement ini nilai R29 telah bernilai 3, nilai biner pada pin D1 D0 adalah 11. Setelah push button di tekan maka nilai biner akan Kembali lagi menjadi 00 lalu program akan jump ke statement awal dan prosesnya akan terus menerus tanpa henti. jika belum maka program akan selalu looping, jump ke label L4
AKHIR: RJMP AKHIR	Statement yang tidak berpengaruh pada program dikarenakan program akan selalu Jump ke label L1. Sehingga proses pada RJMP AKHIR tidak akan tercapai
DISPLAY:MOV R16,R17 MOV R30,R18	Subrutin DISPLAY: untuk menampilkan data kata yang ada pada program area KAR menggunakan pointer Z, yang dipicu oleh data dari Register R17
ULANG1: LDI R23,$8	R23 disikan hexadesimal $8 dikarenakan jumlah data yang akan diambil pada program area adalah 8
LDI R20,$80	R20 diinputkan $80 untuk mengidentifikasi posisi lokasi program area kepada sevent segment untuk dapat mengetahui urutan keluaran data mana yang terlebih dahulu ditampilkan
D1: OUT PORTA,R20 LSR R20	Pada label D1 R20 yang telah disikan posisi data untuk setiap karakter yang ada pada program area. Maka akan outputkan pada Atmega8535 dibagian Pin A01 – A07 ke Common Anode Display 7 Segment , Geser kanan R20

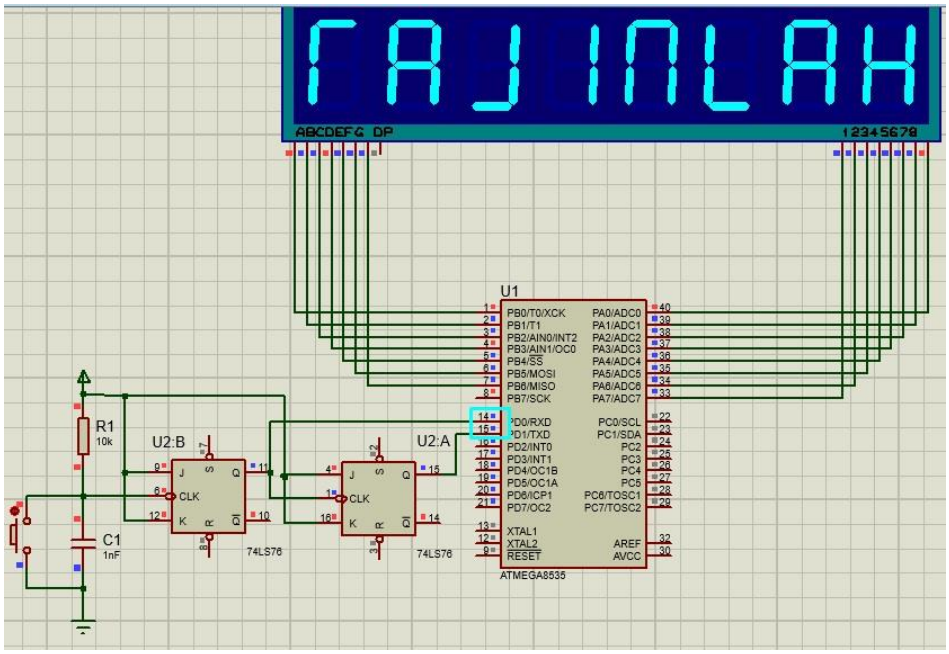
LPM R22,Z+ OUT PORTB,R22	LPM berfungsi untuk membaca satu byte data pada program area yang diambil menggunakan pointer Z lalu di masukkan ke register R22 , setelah itu di Outputkan melalui Pin B berupa logic biner dari pin B01 s/d B07. Selanjutnya diterjemahkan pada IC Seven Segment 8 digit display common anode sebagai Pola Karakter
RCALL DELAY	Memanggil subrutin DELAY untuk jeda Output
DEC R23 BRNE D1	Mengurangi nilai R23 sebagai counter jumlah karakter yang dikerluarkan di setiap katanya yaitu 8 karakter, apabila R23 belum bernilai 0 maka proses penampilan kata perkarakter akan berlanjut
DEC R16 BRNE ULANG1 RET	Register R16 dikurangi 1 sebagai counter lama waktu yang diperlukan untuk mencapai 4 detik jeda , apabila belum bernilai 0 maka jump ke label ULANG1 dan program akan trus menampilkan kata pada rentang waktu 4 detik
BLANK : LDI R16,10	Subrutin BLANK untuk membuat seven segment seakan akan tidak menampilkan apa – apa “blank” , dibuat untuk menampilkan display kata yang di selipkan redipan dalam 4 detik. Menginputkan nilai 10 pada register R16 sebagai counter lama blank terjadi artinya semakin besar nilai R16 maka jarak waktu antara satu kedipan ke kedipan selanjutnya akan semakin lama
ULANG2: LDI R23,$8	R23 disikan hexadesimal $8 dikarenakan jumlah data yang akan diambil pada program area adalah 8

LDI R20,$80	R20 diinputkan $80 untuk mengidentifikasi posisi lokasi program area kepada sevent segment untuk dapat mengetahui urutan keluaran data mana yang terlebih dahulu ditampilkan
D2: OUT PORTA,R20 LSR R20	Pada label D2 R20 yang telah disikan posisi data untuk setiap karakter yang ada pada program area. Maka akan outputkan pada Atmega8535 dibagian Pin A01 – A07 ke Common Anode Display 7 Segment , Geser kanan R20
LDI R22,$FF OUT PORTB,R22 RCALL DELAY	Register R22 di inputkan nilai hexadecimal FF yang mengartikan bahwa tampilan seven segment akan menjadi blank, yang membuat seolah olah segment tersebut tidak menyala
DEC R23 BRNE D2	Mengurangi nilai R23 sebagai counter jumlah karakter yang dikerluarkan di setiap katanya yaitu 8 karakter, apabila R23 belum bernilai 0 maka proses penampilan kata perkarakter akan berlanjut
DEC R16 BRNE ULANG2 RET	Register R16 dikurangi 1 sebagai counter lama waktu yang diperlukan untuk mencapai 4 detik jeda , apabila belum bernilai 0 maka jump ke label ULANG2 dan program akan trus menampilkan kata pada rentang waktu 4 detik
DELAY: PUSH R21 PUSH R22	Subrutin DELAY PUSH dan POP digunakan Untuk pengoperasian transfer data pada stack PUSH untuk menyimpan data pada stack
LDI R22,10 LUP1: LDI R21,200 LUP2: DEC R21	Statement ini digunakan untuk membuang buang waktu atau delay selama 2ms .

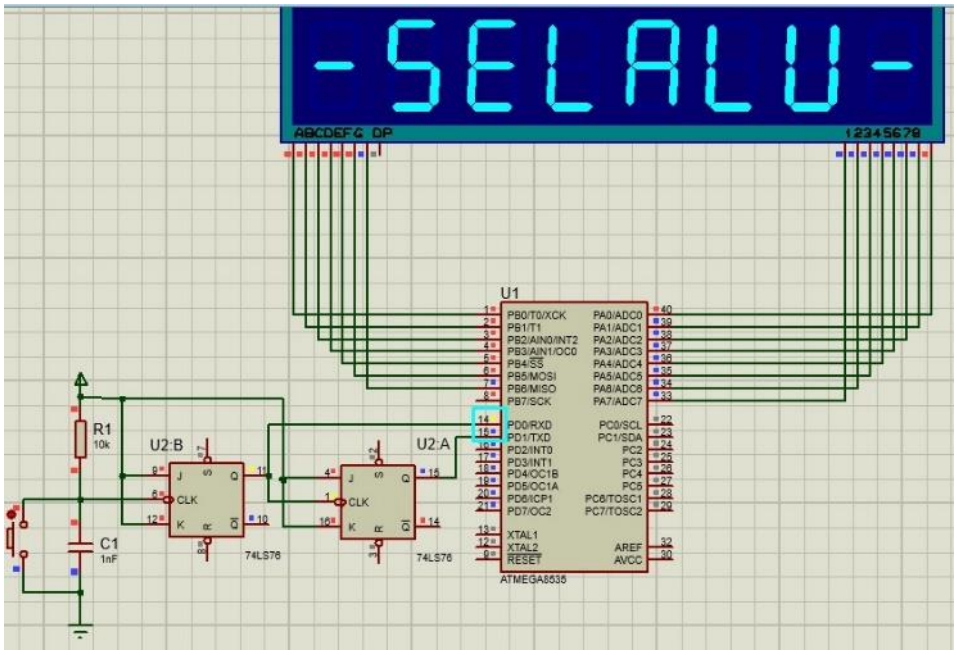
BRNE LUP2 DEC R22 BRNE LUP1	
POP R22 POP R21 RET	POP untuk mengambil data pada stack yang tadi di PUSH sebelumnya
CHECK: IN R29,PIND ANDI R29,3 RET	Subrutin CHECK sebagai polling untuk penentu apabila nilai logika binernya berubah, maka hasil output tampilan seven segmentnyapun akan berubah sesuai dengan urutan biner ANDI yaitu Melakukan operasi logika AND pada setiap isi register R29 dengan konstanta 3 dan menempatkan hasilnya pada register R29
.ORG $80 KAR: .DB $CE , $88 , $F1 , $F9 , $C8 , $C7 , $88 , $89 .DB $BF , $92 , $86 , $C7 , $88 , $C7 , $C1 , $BF .DB $92 , $C1 , $C8 , $82 , $82 , $C1 , $89 , $BF .DB $83 , $86 , $C7 , $88 , $F1 , $88 , $CE , $BF	Program area $80. data Hexadesimal untuk menampilkan karakter pada common anode sesuai dengan kata yang diminta pada soal “rAJInLAH” “-SELALU-“ “SUnGGUH” “bELAJAr-“

Hasil:

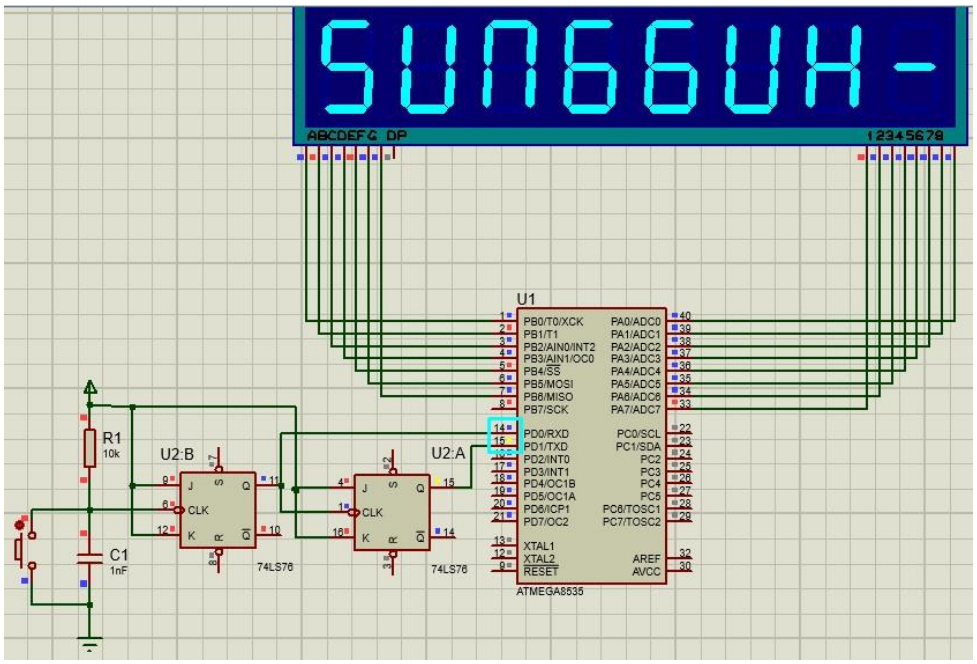
Saat poling berlogic 0B00 maka akan muncul kata “rAJInLAH”:



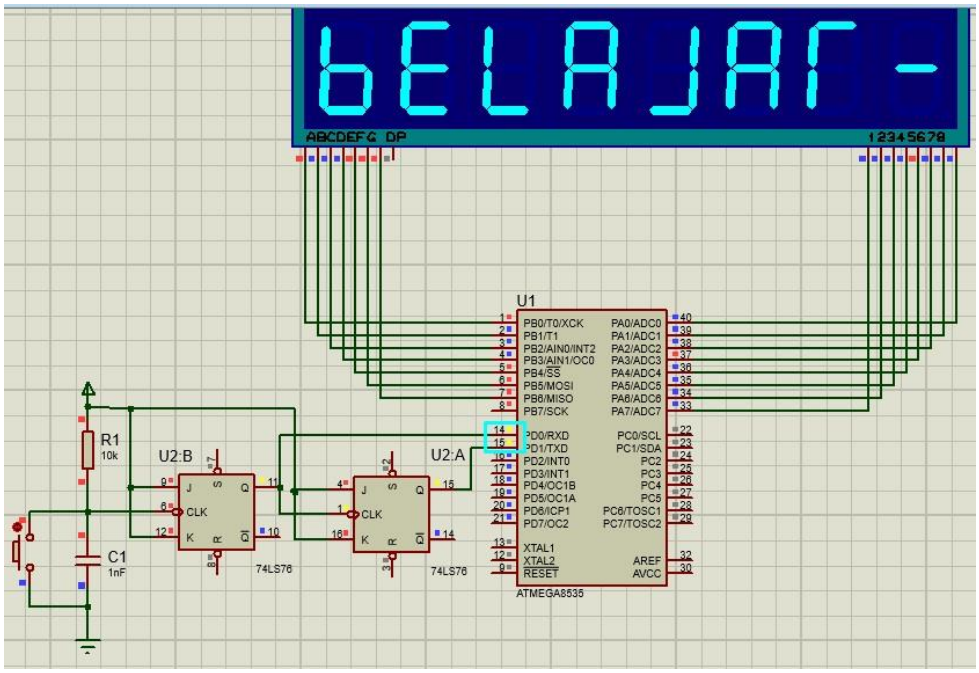
Saat poling berlogic 0B01 maka akan muncul kata “-SELALU-“:



Saat poling berlogic 0B10 maka akan muncul kata “SUnGGUH”:



Saat poling berlogic 0B11 maka akan muncul kata “bELAJAR-”:

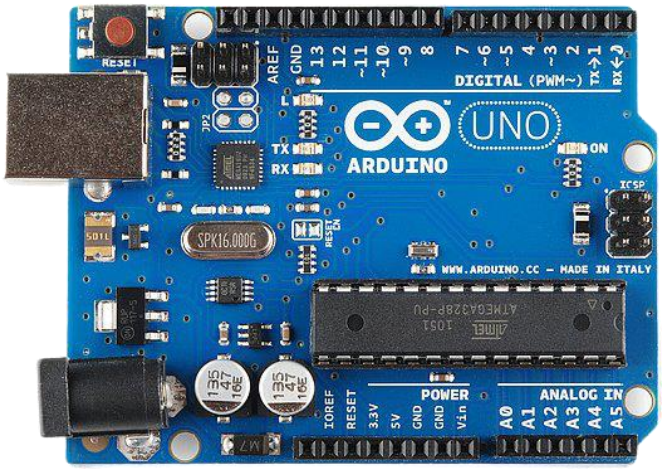


BAB 10
DASAR ARDUINO

10.1 Pengertian Arduino

Arduino Uno merupakan board mikrokontroler berbasis ATmega328 yang memiliki 14 pin input dari output digital yang di mana 6 pin input tersebut dapat digunakan sebagai output PWM dan 6 pin input analog, 16 MHz osilator kristal, koneksi USB, jack power, ICSP header, dan tombol reset. Untuk mendukung mikrokontroler agar dapat digunakan, cukup hanya menghubungkan Board Arduino Uno ke komputer dengan menggunakan kabel USB atau listrik dengan AC yang-ke adaptor-DC atau baterai untuk menjalankannya [5].

Setiap 14 pin digital pada arduino uno dapat digunakan sebagai input dan output, menggunakan fungsi pinMode(), digitalwrite(), dan digitalRead(). Fungsi fungsi tersebut beroperasi di tegangan 5 volt, Setiap pin dapat memberikan atau menerima suatu arus maksimum 40 mA dan mempunyai sebuah resistor pull-up (terputus secara default) 20-50 kOhm [5].



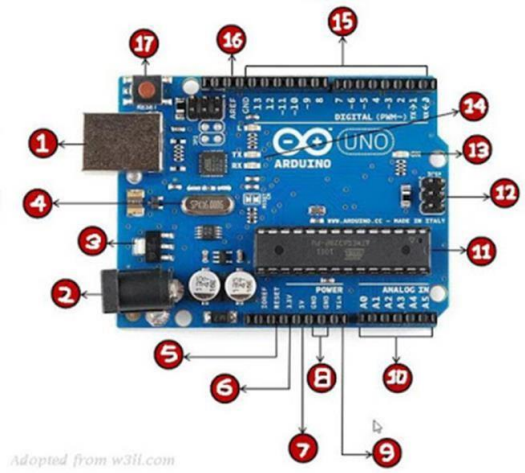
Berikut spesifikasi teknis dari Arduino Uno R3 board [6]:

- Mikrokontroler ATmega328

- Catu Daya 5V
- Teganan Input (rekomendasi) 7-12V
- Teganan Input (batasan) 6-20V
- Pin I/O Digital 14 (dengan 6 PWM output)
- Pin Input Analog 6
- Arus DC per Pin I/O 40 mA
- Arus DC per Pin I/O untuk PIN 3.3V 50 mA
- Flash Memory 32 KB (ATmega328) dimana 0.5 KB digunakan oleh bootloader
- SRAM 2 KB (ATmega328)
- EEPROM 1 KB (ATmega328)
- Clock Speed 16 MHz

Dengan menggunakan mikrokontroler dapat dibuat program untuk mengendalikan berbagai komponen elektronika dan fungsi Arduino Uno dibuat untuk memudahkan dalam melakukan prototyping, memprogram mikrokontroler, membuat alat-alat canggih berbasis mikrokontroler. Arduino Uno board didukung oleh software Arduino IDE yang dapat membantu dalam melakukan pemrograman, melakukan kompilasi program, debugging dan proses download ke Arduino board.

10.2 Pin-pin pada Arduino



Berikut adalah deskripsi dari pin-pin tersebut [7]:

No.	Deskripsi
1	Power USB Menghubungkan papan arduino dengan komputer melalui koneksi USB sebagai supply listrik bagi papan alat atau pemrograman mikrokontrolller.
2	Power Jack Supply untuk arduino dengan tipe jack. Input DC 5-12V.
3	Voltage Regulator IC yang digunakan untuk menstabilkan tegangan eksternal dari Jack No. 2 menuju 5 V, tegangan aman papan Arduino.
4	Crystal Oscillator
	Digunakan sebagai detak jantung pada Arduino. Jumlah cetak menunjukkan 16 MHz yang digunakan sebagai timer atau penghitung.
5 & 17	Reset Digunakan untuk mengulang program Arduino dari awal. Cara pertama dengan menekan tombol reset (17) di papan. Cara kedua dengan menggabungkan PIN reset dengan GND secara singkat.
6,7,8,9	Pin (3.3, 5, GND, Vin) • 3.3V (6) sumber tegangan output 3.3 V. • 5V (7) sumber tegangan output 5 V. • GND (8) merupakan pin untuk ground (pin negatif). • Vin (9) Pin penyuplai listrik sebesar 5 V.
10	Analog Pins papan Arduino Uno memiliki 6 PIN analog A0 sampai A5. Digunakan untuk membaca sinyal atau sensor analog seperti sensor jarak, sensor suhu, dan lainnya. Serta mengubahnya menjadi nilai digital.
11	IC Mikrokontroller Sebagai otak dari papan Arduino yang di program oleh papan Arduino untuk mengatur PIN digital (15) dan PIN analog (10).

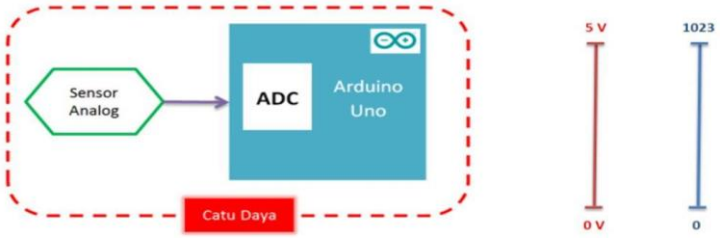
12	ICSP Pin Pin yang dapat digunakan untuk mengembalikan bootloader yang hilang atau rusak. Juga untuk AVR. Dalam Arduino terdapat enam pin yaitu MOSI, MISO, SCK, RESET, VCC, dan GND.
13	LED Power Indicator Lampu ini akan menyala dan menandakan bahwa papan Arduino telah mendapatkan supply dengan baik. Jika lampu tidak menyala maka ada sesuatu yang salah dengan supply maupun papan Arduino tersebut.
14	LED TX dan RX Sebagai indikator bahwa sedang terjadi proses pengiriman dan penerimaan program pada papan Arduino.
15	Pin Digital I/O Terdapat 14 pin digital pada Arduino. Masing-masing berfungsi untuk memberikan nilai logic (1 dan 0). Pin yang memiliki label "-" berfungsi sebagai pin untuk menghasilkan PWM (Pulse Width Modulation). Digital Pin I/O dapat digunakan sebagai fungsi saklar.
16	AREF Digunakan untuk mendapatkan sumber tegangan yang dapat diatur melalui IC. Tegangannya di antara 0 sampai dengan 5 Volt.

IC Mikrokontroller Arduino menggunakan beberapa IC berikut [7]:

Data	**AtMega328**	**AtMega128**	**AtMega8**
Digital I/O Pins	14 (6 PWM)	14 (6 PWM)	14 (3 PWM)
Analog Pins	6 (DIP) atau 8 (SMD)	6 (DIP) atau 8 (SMD)	6
DC Current per I/O Pin Digital	40 mA	40 mA	40 mA
Flash Memory	32 kB	16 kB	8 kB

SRAM	2 kB	1 kB	1 kB
EEPROM	1 kB	512 Bytes	512 Bytes

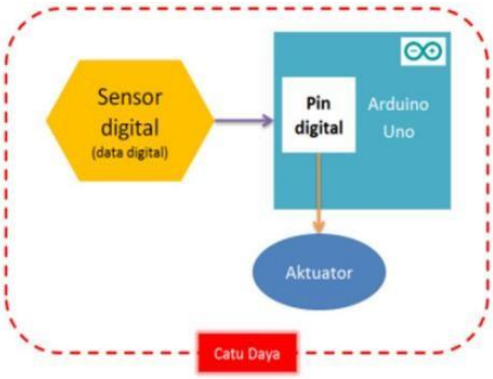
Berikut adalah pin analog pada Arduino [8]:



Pin ini merupakan pin yang digunakan untuk menerima nilai analog keluaran dari sensor analog. Pada pin-pin inilah terdapat fasilitas ADC (Analog to Digital Converter) yang berfungsi mengubah atau meng-konversi nilai analog ke digital. Hal ini perlu dilakukan karena semua proses yang terjadi di dalam arduino merupakan proses digital.

Cara mengubah nilai analog menjadi nilai digital adalah dengan menggunakan konversi kesetaraan. Nilai analog (tegangan) yang diperbolehkan masuk pada pin analog sebesar 0 V hingga +5V. Arduino UNO memiliki resolusi ADC sebesar 10-bit, artinya nilai analog dapat dinyatakan dalam 1023 (210 – 1) nilai diskrit.

Berikut adalah pin digital pada Arduino [8]:



Pin ini digunakan untuk menerima nilai digital keluaran dari sensor digital. Sensor digital mengkonversi nilai analog menjadi digital kemudian dikirimkan kepada arduino melalui pin digital.

Pada pin digital terdapat 2 (dua) kondisi yaitu High (nilai digital 1) dan LOW (nilai digital 0). Pin-pin tersebut dapat diatur sebagai INPUT (masukkan) untuk menerima data digital dari sensor, dan dapat diatur sebagai OUTPUT (keluaran) untuk mengaktifkan atau menonaktifkan aktuator.

10.3 Jenis-jenis Arduino

Berikut adalah jenis-jenis arduino [8]:

1. Entry Level : Arduino/Genuino UNO Arduino PRO Arduino PRO Mini Arduino/Genuino Mini Arduino Nano Arduino/Genuino Starter Kit Arduino Basic Kit Arduino Motor Shield
2. Enhanced Feature: Arduino/Genuino Mega Arduino Zero Arduino DUE Arduino Proto Shield
3. IOT : Arduino YUN Arduino Ethernet Shield Arduino WiFi Shield 101
4. Wearable : Arduino GEMMA Lilypad Arduino USB Lilypad Arduino Main Board Lilypad Arduino Simple Lilypad Arduino Simple Snap
5. 3D Printing: Materia 101

10.4 Skema Proyek Arduino



Sensor dapat berupa transducer, atau perangkat yang mengubah sinyal dari luar suara, cahaya, dll) kedalam bentuk sinyal-sinyal listrik untuk kemudian dikirimkan ke Arduino untuk diolah kedalam proses berikutnya yang akan dikerjakan oleh aktuator. Aktuator adalah peralatan mekanis yang mengontrol sebuah mekanisme/sistem [8].

10.5 Input dan Output Modul

Berikut adalah input dan output yang digunakan pada implementasi Arduino [8]:

LED & Switch

- 7-Segment
- Sensor Suhu
- Sensor Cahaya
- Sensor Ultrasonik
- Buzzer
- Motor Servo
- Sensor Infrared
- Gyro/6-Axis Accelerometer
- Piezo Vibration Sensor
- Bluetooth Comm • I/O Board :
- WiFi Shield
- LED Module
- FND Module
- Modul Sensor Cahaya (Ambient Light Sensor Module)
- Modul Sensor Temperatur (Temperature Sensor Module)
- Ultrasonic Sensor Module
- Motor Module

10.6 Pemrograman Software Arduino IDE

Untuk memprogram board Arduino, dibuthkan aplikasi IDE yang merupakan bawaan dari Arduino yang berguna untuk membuat, membuka, dan mengedit source

code Arduino. Berikut adalah penjelasan mengenai fungsi yang terdapat pada IDE Arduino [8]:



- Verify/Compile digunakan untuk memverivikasi/mengecek terlebih dahulu sketch (source code) yang ada.
- Upload berfungsi untuk mengirimkan sketch ke papan Arduino yang telah terhubung.
- New Sketch digunakan untuk membuat jendela baru untuk sketch baru.
- Open Sketch digunakan untuk membuka sketch yang telah dibuat sebelumnya.
- Save Sketch menyimpan sketch namun tidak meng-compile-nya
- Serial Monitor membuka interface untuk komunikasi visual
- Ket. Aplikasi ini digunakan untuk menampilkan proses yang sedang terjadi
- Console akan menampilkan riwayat apa saja yang telah dilakukan ke program yang ada selama proses. Misal informasi error, dan informasi compile yang berhasil atau tidak.

- Information Port menampilkan port mana yang dipakai oleh Arduino

10.7 Contoh Program Arduino

1. Buatlah rangkaian arduino dengan output 4 buah LED, yang masing-masing LED terhubung pada kaki 8,9,10, dan 11. Hidupkan LED secara bertahap mulai dari LED1 hingga LED4 dengan jeda waktu 1 detik, kemudian matikan LED secara bertahap mulai dari LED1 hingga LED4 dengan jeda waktu 1 detik.

   Program:

```
//inisialisasi jumlah LED
const int numLED = 4;
const int pinLED[numLED] = {8,9,10,11};

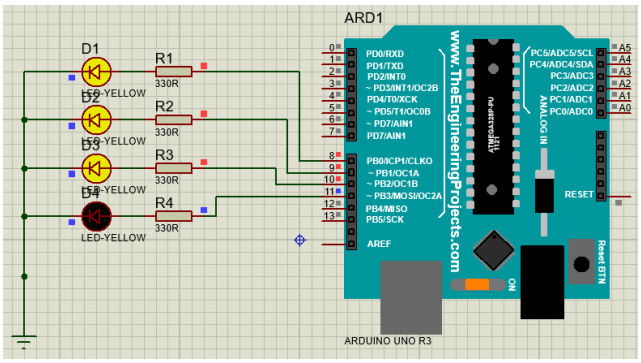
//Inisialisasi semua pin LED sbg Output
void setup()
{
  for(int i=0;i<4;i++)
  {
    pinMode(pinLED[i],OUTPUT);
  }
}

void loop()
{
  //Hidupkan semua LED bertahap dgn jeda 1 detik
  for(int i=0;i<4;i++)
  {
    digitalWrite(pinLED[i],HIGH);
    delay(1000);
  }

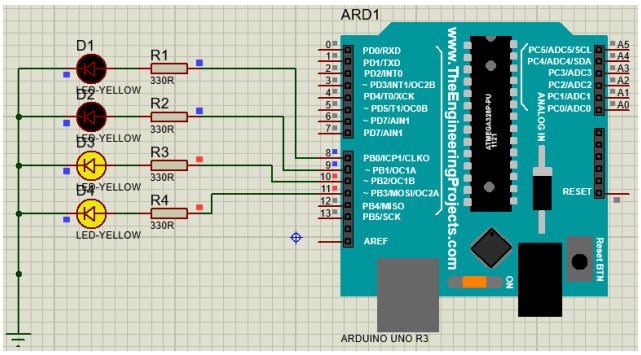
  //Matikan semua LED bertahap dgn jeda 1 detik
  for(int i=0;i<4;i++)
  {
    digitalWrite(pinLED[i],LOW);
    delay(1000);
  }
}
```

   Hasil:

   Ketika menyala secara bertahap dari LED1 hingga LED4:



Ketika mati secara bertahap dari LED1 hingga LED4:



Pada program ini, dibuat program dan rangkaian untuk menyalakan dan mematikan 4 buah LED dengan aturan yang berbeda beda. Penulisan program diawali dengan inisialisasi jumlah LED dan pin-pin LED yang akan dipakai dengan menggunakan perintah const int. Kemudian, dilakukan inisiaslisasi semua pin sebagai output pada void setup dengan menggunakan perulangan for sampai i<4 dengan operator increment yang menggunakan perintah pinMode. Lalu untuk program ini, LED dihidupkan dan dimatikan secara bertahap dari LED1 sampai LED4 sehingga pada void loop, digunakan dua fungsi perulangan for dengan operator increment untuk meghidupkan dan mematikan lampu. Isi perulangan tersebut adalah perintah digitalWrite yang disesuaikan atas HIGH (menyala) atau LOW (mati) nya dan diberikan delay 1000ms atau 1000 detik untuk setiap tahapnya.

2. Buatlah rangkaian arduino dengan output 4 buah LED, yang masing-masing LED terhubung pada kaki 8,9,10, dan 11. Atur agar LED1 menyala selama 1 detik (yang lainnya padam), kemudian bergantian LED2 menyala selama 1

detik (yang lainnya padam), LED3 menyala selama 1 detik (yang lainnya padam), dan terakhir LED4 menyala selama 1 detik (yang lainnya padam).

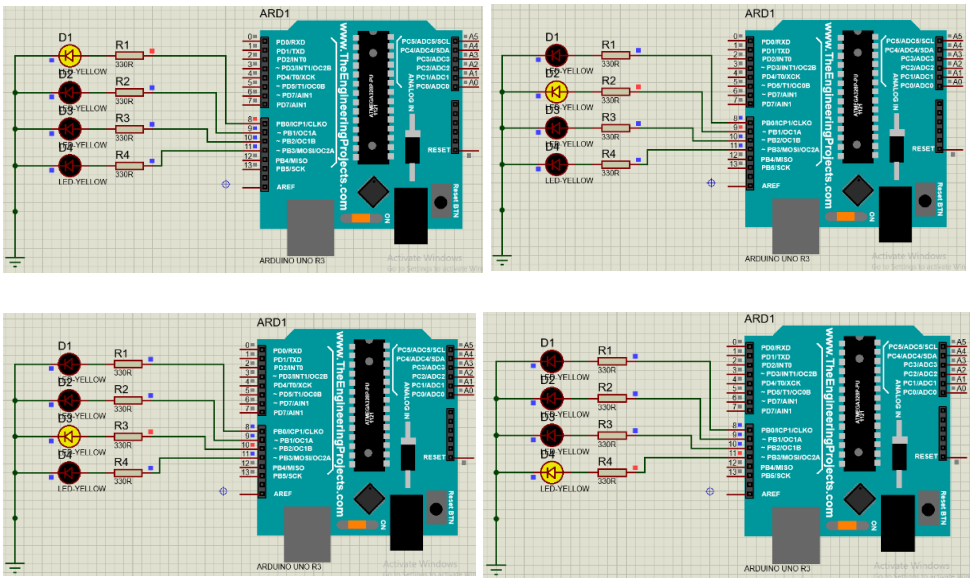
Program:

```
//inisialisasi jumlah LED
const int numLED = 4;
const int pinLED[numLED] = {8,9,10,11};

//Inisialisasi semua pin LED sbg Output
void setup()
{
  for(int i=0;i<4;i++)
  {
    pinMode(pinLED[i],OUTPUT);
  }
}

void loop()
{
  for(int i=0;i<4;i++)
  {
    digitalWrite(pinLED[i],HIGH);
    delay(1000);
    digitalWrite(pinLED[i],LOW);
  }
}
```

Hasil:



Lalu, untuk program ini, tahapan menyala nya adalah setiap LED menyala bergantian dari LED1 sampai LED4, sehingga pada void loop, index pada

variabel pinLED[i] akan diisi dengan nilai pinLED berdasarkan jumlah perulangan dari index 0 sampai 3 dengan operator increment per 1 detik yaitu HIGH terlebih dahulu kemudian LOW.

BAB 11
PWM, PUSH BUTTON, DAN POTENSIOMETER PADA ARDUINO

11.1 Teori PWM

PWM merupakan suatu metode untuk mendapatkan sinyal analog dari sinyal digital. Transisi state antara HIGH dan LOW menjadi lebih halus. PWM digunakan ketika kita ingin mengaktifkan suatu output dengan kekuatan atau intensitas tertentu yang dapat diubah-ubah sesuai keinginan kita. Contoh penerapan PWM : mengatur kecerahan LED, mengatur kecepatan motor DC, mengatur intensitas cahaya pada VLC (*Visible Light Communication*). Berikut ini akan dijelaskan contoh program "Mengontrol kecerahan LED".

11.2 Teori dan Contoh Program Push Button

- Tombol pushbutton (tactile) atau tombol push on, terdiri atas 2 jenis yaitu jenis NO(Normally Open) dan NC (Normally Close) .
- Pada jenis NO, ketika tombol ini ditekan, maka jalur akan tertutup (ON), ketika dilepas jalur akan kembali terbuka (OFF).
- Pada jenis NC (Normally Close), ketika kondisi normal (sebelum ditekan), kaki terminal dalam keadaan terturup / tersambung (Close), tapi ketika ditekan, kaki terminalnya terbuka (tidak tersambung).
- Dalam contoh, akan digunakan jenis pushbutton NO seperti yang ditunjukkan pada gambar di bawah ini:



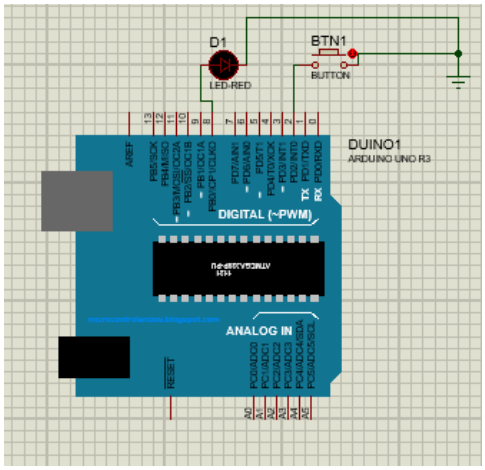
Contoh Program:

```
// pin 2 sebagai input dan pin 8 sebagai output
const int pinButton = 2;
const int pinLED = 8;

void setup() {
  pinMode(pinButton, INPUT);
  pinMode(pinLED, OUTPUT);

  // aktifkan pull-up resistor
  digitalWrite(pinButton, HIGH);
}

void loop() {
  if(digitalRead(pinButton) == LOW){
    digitalWrite(pinLED, HIGH);
  }else{
    digitalWrite(pinLED, LOW);
  }
}
```



- Awalnya LED akan padam. ketika kita menekan tombol pushbutton, LED akan menyala. LED akan kembali padam ketika tekanan tombol dilepas.
- Jika sebuah pin diset sebagai INPUT, maka mikrokontroller akan mengambil data dari pin tersebut. Jika sebuah pin diset sebagai OUTPUT, maka mikrokontroller akan menuliskan data pada pin tersebut. dalam hal ini, mikrokontroller akan mengambil data yang dari pushbutton.
- Pada rangkaian, pinButton (pin 8) dihubungkan ke GND, artinya, ketika pushbutton ditekan maka pinButton (pin 8) akan menjadi 0 (LOW).
- Fungsi utama dari saklar (dalam hal ini adalah pushbutton) adalah mengubah nilai yang awalnya LOW menjadi HIGH, atau sebaliknya. Oleh karena itu,

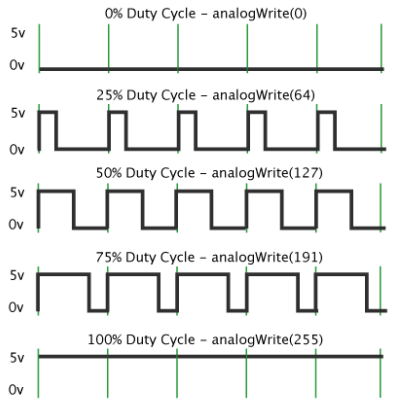
karena ketika pushbutton ditekan akan bernilai LOW (ke GND), maka awalnya harus kita set menjadi HIGH.

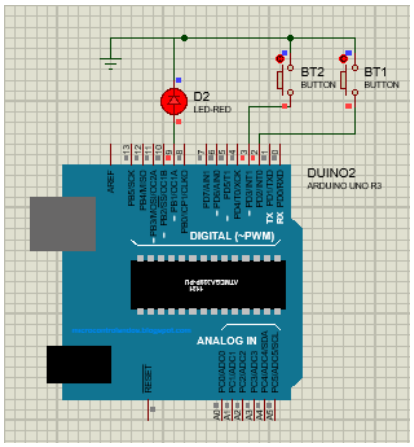
11.3 Teori dan Contoh Program Potensiometer

Untuk percobaan ‘mengontrol kecerahan LED’, kita dapat menggunakan potensiometer 50k ohm jenis trimmer. Dapat juga mencoba dengan menggunakan potensiometer putar. Kelebihan menggunakan potensiometer yaitu lebih mudah karena hanya butuh satu alat untuk membuat LED lebih redup atau lebih terang.

Contoh Program 1:

- Pada contoh berikut, akan ditampilkan contoh program untuk mengatur kecerahan LED menggunakan dua buah pushbutton dengan ketentuan : pushbutton yang pertama untuk menaikkan kecerahan LED hingga paling terang, sedangkan pushbutton yang kedua untuk menurunkan kecerahan LED hingga LED padam. Contoh ini menggunakan konsep PWM.
- Sebagian kaki / pin Arduino dapat men-support PWM, kaki yang support PWM ditandai dengan adanya tanda tilde (~) di depan angka pinnya, seperti 3, 5, 6, dan seterusnya.
- Frekuensi yang digunakan dalam Arduino untuk PWM adalah 500Hz (500 siklus dalam 1 detik). Jadi, Arduino bisa menghidup-matikan LED sebanyak 500 kali dalam 1 detik.
- Untuk menggunakan PWM, kita bisa menggunakan fungsi analogWrite(). Nilai yang bisa dimasukkan pada fungsi tersebut yaitu antara 0 hingga 255.
- Nilai 0 berarti pulsa yang diberikan untuk setiap siklus selalu 0 volt, sedangkan nilai 255 berarti pulsa yang diberikan selalu bernilai 5 volt.
- Jika kita memberikan nilai 127 (kita anggap setengah dari 0 hingga 255, atau 50% dari 255), maka setengah siklus akan bernilai 5 volt, dan setengah siklus lagi akan bernilai 0 volt.





Contoh Program 2:

Buatlah rangkaian serta program arduino untuk mengatur tingkat kecerahan LED menggunakan potensiometer. Gunakan salah satu warna yang ada pada modul RGB LED. Tampilkan nilai potensiometer pada Serial Monitor.

Program:

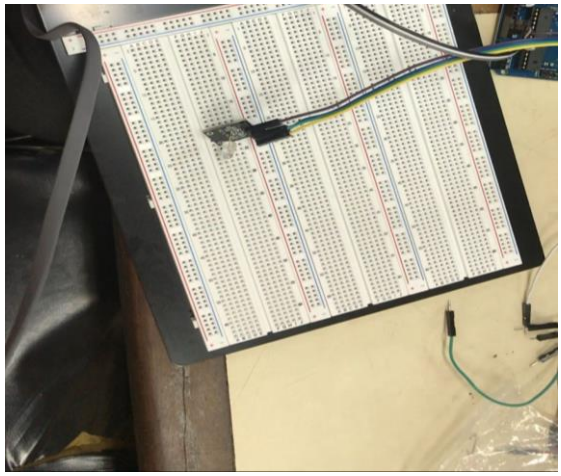
```
const int pinLED1 = 9;
const int pinLED2 = 10;
const int pinLED3 = 11;
void setup() {
  // pin LED sebagai OUTPUT
pinMode(pinLED1,OUTPUT);
pinMode(pinLED2,OUTPUT);
pinMode(pinLED3,OUTPUT);
}

void loop() {
  // perulangan sebanyak 5 kali
  // dari i=0; hingga i=3 atau (i < 5)
for (int i=0; i<4; i++) {
  if (i==1){
    digitalWrite(pinLED1,HIGH);
    digitalWrite(pinLED2,LOW);
     digitalWrite(pinLED3,LOW);
     delay(2000);
  } else if (i==2){
    digitalWrite(pinLED1,LOW);
    digitalWrite(pinLED2,HIGH);
    digitalWrite(pinLED3,LOW);
    delay(2000);
  } else if (i==3){
    digitalWrite(pinLED1,LOW);
    digitalWrite(pinLED2,LOW);
    digitalWrite(pinLED3,HIGH);
    delay(2000);
}else {
 digitalWrite(pinLED1,LOW);
 digitalWrite(pinLED2,LOW);
 digitalWrite(pinLED3,LOW);
}
delay(2000);
}
}
```

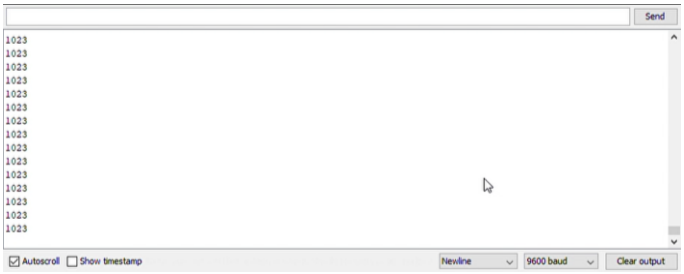
Hasil:

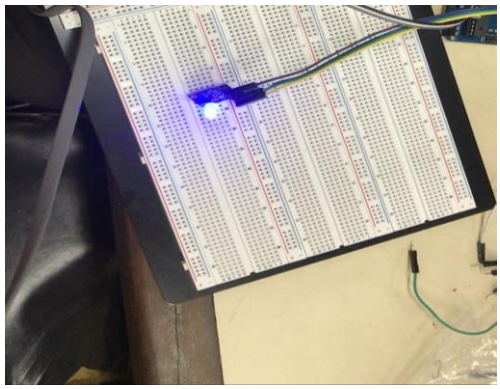
Saat 0 atau minimum:





Saat maksimum:





Pada program yang telah dilakukan terdapat beberapa perintah seperti pengaturan kecerahan menggunakan potensiometer, pengaturan kecerahan dapat digunakan program brightness = map(sensor,0,1023,0,255); yang akan mengatur tingkat kecerahan pada LED. Kemudian untuk memonitori pada angka berapa tingkat kecerahan minimum dan maksimum dapat menggunakan serial monitor pada tools. Dengan kondisi rangkaian, kedua ujung kaki trimmer yang satu sisi (sisi kanan) masing-masing disambungkan ke +5v dan GND. Jika Anda menggunakan potensiometer putar, yang disambungkan ke +5v dan GND adalah pin yang paling pinggi. 4. Pin yang satu (di sebelah kiri) disambungkan ke A0 pada board Arduino. Jika menggunakan potensiometer putar,yang disambungkan ke A0 adalah pin yang tengah pada potensiometer.

Kemudian pada program-program selanjutnya diperintahkan untuk membuat animasi program dengan kondisi LED menyala secara bergiliran, hal tersebut dapat digunakan perintah program if dan else if yang menentukan kondisi dari LED menyala atau tidak. Ketika LED diharapkan menyala, maka dapat digunakan perintah digitalWrite(pinLED1,HIGH); tetapi jika LED diharapkan tidak menyala atau mati dapat digunakan perintah digitalWrite(pinLED2,LOW);

BAB 12
PENGGUNAAN SENSOR PADA ARDUINO

12.1 Teori Sensor

Sensor merupakan alat yang digunakan untuk mendeteksi perubahan besaran fisik seperti tekanan, gaya, besaran listrik, cahaya, gerakan, kelembapan, suhu, kecepatan dan fenomena-fenomena lingkungan lainnya.

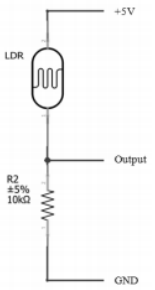
Sensor dapat digolongkan sebagai transduser karena sensor dapat mengkonversi input perubahan fisik yang terdeteksi seperti energi listrik seperti cahaya, tekanan, gerakan, suhu atau energi fisik yang lainnya menjadi output sinyal listrik ataupun resistensi yang kemudian akan dikonversikan lagi ke tegangan ataupun sinyal listrik yang dapat dimengerti oleh manusia.

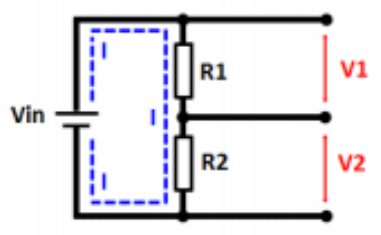
Proses konversi tersebut dilakukan baik melalui perangkat sensor itu sendiri ataupun ditransmisikan secara elektronik melalui jaringan untuk ditampilkan atau diolah menjadi informasi yang bermanfaat bagi pengguna.Terdapat beberapa jenis sensor yang biasa digunakan pada Arduino, antara lain:

12.2 Sensor Cahaya (LDR) / Photoresistor

Light Dependent Resistor (LDR) atau disebut juga Photoresistor merupakan salah satu jenis resistor yang besar hambatan/resistansinya berubah-ubah berdasarkan intensitas cahaya yang diterima. Nilai hambatan dari LDR berbanding terbalik dengan intensitas cahaya yang diterimanya. Nilai hambatan dari LDR akan besar apabila intensitas cahaya yang diterimanya rendah (dalam kondisi gelap). Hambatan dari LDR akan kecil apabila instensitas cahaya yang diterima tinggi (kondisi terang). Dalam kondisi gelap, resistansi LDR bisa mencapai 10 M ohm, tapi dalam kondisi terang, resistansi LDR turun hingga 1 K ohm bahkan bisa kecil lagi. Sifat inilah yang membuat LDR bisa dimanfaatkan sebagai sensor cahaya.

Dalam menggunakan LDR dalam sebuah rangkaian menggunakan prinsip pembagi tegangan seperti pada gambar berikut :





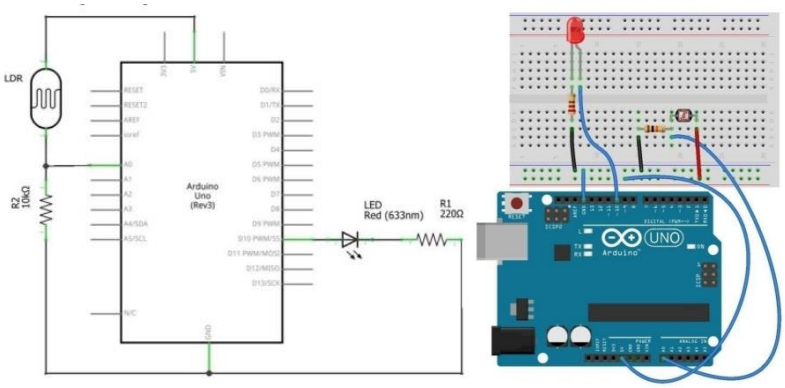
$$V1 = \frac{R1}{(R1 + R2)} .Vin$$

$$V2 = \frac{R2}{(R1 + R2)} .Vin$$

Dengan menggabungkan antara LDR dengan resistor (atau potensiometer), maka kita bisa mendapatkan variasi tegangan (pada V1 atau V2) yang nantinya menjadi inputan pada pin analog Arduino. Ketika kondisi gelap, tegangan yang melewati LDR akan menjadi lebih rendah disebabkan karena resistansi LDR besar, dan tegangan akan lebih tinggi ketika kondisi terang karena resistansi LDR kecil.

Contoh Penggunaan Sensor LDR

Rangkaian:



Program:

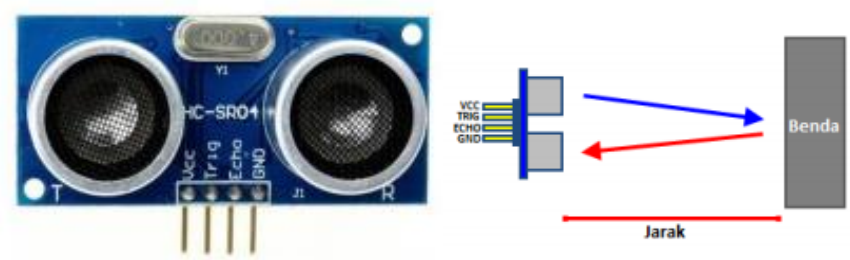
```
//Inisialisasi pin
int ledPin = 13;
int ldrPin = A0;
void setup() {
Serial.begin(9600);
```

```
pinMode(ledPin, OUTPUT);
pinMode(ldrPin, INPUT); }
 void loop() {
//membaca nilai input analog
int ldrStatus = analogRead(ldrPin);
//pengujian kondisi
if (ldrStatus <= 200) {
digitalWrite(ledPin, HIGH);
 Serial.print("Gelap, LED hidup : ");
 Serial.println(ldrStatus);
}
else {
digitalWrite(ledPin, LOW);
Serial.print("Terang, LED mati : ");
Serial.println(ldrStatus); }
 }
```

12.3 Sensor Ultrasonik (HC-SR204)

Sensor ultrasonik adalah sebuah sensor yang berfungsi untuk mengubah besaran fisis (bunyi) menjadi besaran listrik dan sebaliknya. Cara kerja sensor ini didasarkan pada prinsip dari pantulan suatu gelombang suara sehingga dapat dipakai untuk menafsirkan eksistensi (jarak) suatu benda dengan frekuensi tertentu. Disebut sebagai sensor ultrasonik karena sensor ini menggunakan gelombang ultrasonik (bunyi ultrasonik).

Salah satu sensor ultrasonik ialah sensor HC-SR04. Sensor ini juga relatif terjangkau untuk pembelajaran. Gelombang ultrasonik merupakan gelombang yang umum digunakan untuk radar untuk mendeteksi keberadaan suatu benda dengan memperkirakan jarak antara sensor dan benda tersebut.



Gelombang ultrasonik adalah gelombang bunyi dengan frekuensi sangat tinggi yaitu 20.000 Hz. Bunyi ultrasonik bisa merambat melalui zat padat, cair dan gas. Akan tetapi, gelombang bunyi ultrasonik akan diserap oleh tekstil dan busa.

Cara kerja :

Pada sensor ultrasonik, gelombang ultrasonik dibangkitkan melalui alat piezoelektrik dengan frekuensi tertentu. Piezoelektrik ini akan menghasilkan gelombang ultrasonik (umumnya berfrekuensi 40kHz) ketika sebuah osilator diterapkan pada benda tersebut. Alat ini akan menembakkan gelombang ultrasonik menuju suatu area/ target. Setelah gelombang menyentuh permukaan target, maka target akan memantulkan kembali gelombang tersebut. Gelombang pantulan dari target akan ditangkap oleh sensor, kemudian sensor menghitung selisih antara waktu pengiriman gelombang dan waktu gelombang pantul diterima.

Karena kecepatan bunyi adalah 340 m/s, maka rumus untuk mencari jarak berdasarkan ultrasonik adalah :

$$S = \frac{340 \,.\, t}{2}$$

S= jarak sensor ultrasonik dengan benda (bidang pantul)
t = selisih antara waktu pemancaran gelombang oleh transmitter dan waktu ketika gelombang pantul diterima receiver.

HC-SR04 merupakan sensor ultrasonik siap pakai, satu alat yang berfungsi sebagai pengirim, penerima, dan pengontrol gelombang ultrasonik. HC-SR04 digunakan untuk mengukur jarak benda dari 2cm - 4m dengan akurasi 3mm.

Untuk menghitung jarak yang hanya maksimal 4 m maka rumus di atas harus dimodifikasi atau disesuaikan satuannya. Mikrokontroller bisa bekerja pada order mikrosekon (1s = 1.000.000 µs) dan satuan jarak bisa kita ubah ke satuan cm (1m = 100 cm). Oleh sebab itu, rumus di atas bisa diupdate menjadi:

$$S = \frac{340\ \left(\frac{100}{1000000}\right).\ t}{2}$$

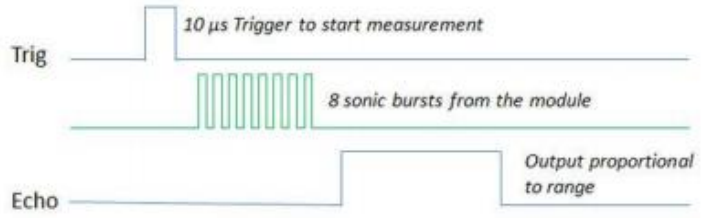
$$S = \frac{0.034\ .\ t}{2}$$

Penggunaan Sensor HC-SR04

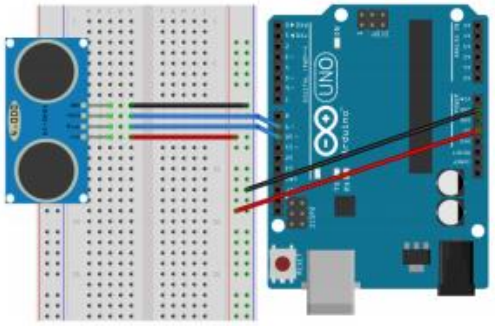
Alat ini memiliki 4 pin, pin Vcc, Gnd, Trigger, dan Echo.

- Pin Vcc untuk listrik positif dan Gnd untuk ground-nya.
- Pin Trigger untuk trigger keluarnya sinyal dari sensor.
- Pin Echo untuk menangkap sinyal pantul dari benda.

Ketika kita memberikan tegangan positif pada pin Trigger selama 10µS, maka sensor akan mengirimkan 8 step sinyal ultrasonik dengan frekuensi 40kHz. Selanjutnya, sinyal akan diterima pada pin Echo. Rumus untuk menghitung jaraknya adalah S = (0.034 *t) /2 cm.



Rangkaian:



Program:

```
// pin 9 trigger
// pin 10 echo
const int pTrig = 9;
const int pEcho = 10;
void setup() {
Serial.begin(9600);
pinMode(pTrig, OUTPUT);
pinMode(pEcho, INPUT); }
long durasi = 0;
 void loop() {
// trigger selama 10us
digitalWrite(pTrig, HIGH);
delayMicroseconds(10);
digitalWrite(pTrig, LOW);
durasi = pulseIn(pEcho, HIGH);
Serial.print("Durasi: ");
Serial.print(durasi);
Serial.print(", Jarak: ");
Serial.println((durasi *0.034)/2);
delay(1000); }
```

Ket:

Fungsi pulseIn() akan memerintahkan sistem untuk menunggu hingga pin Echo bernilai HIGH. Lama proses menunggu akan dianggap sebagai durasi pengiriman + penerimaan sinyal echo yang dipantulkan oleh benda.

fungsi pulseIn() :

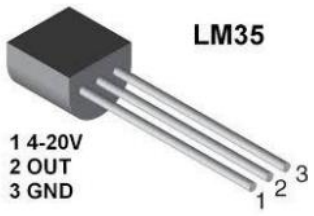
fungsi ini standarnya memiliki 3 parameter, 2 parameter Pin dan Value dan 1 parameter Timeout sebagai parameter tambahan:

```
pulseIn(Pin, Value);
pulseIn(Pin, Value, Timeout);
```

Pada parameter Value, kita bisa memasukkan HIGH atau LOW, jadi Arduino akan menunggu hingga kondisi tersebut dipenuhi. Sedangkan Timeout digunakan ketika dalam waktu tertentu kondisi belum juga terpenuhi.

12.4 Sensor Suhu LM35

Pada proses deteksi suhu, digunakan suatu komponen sensor yang dapat merubah besaran suhu menjadi besaran listrik dalam bentuk tegangan. Salah satu sensor suhu yang sering digunakan adalah sensor Suhu LM35, yang diproduksi oleh National Semiconductor. LM35 memiliki keakuratan tinggi dan kemudahan perancangan jika dibandingkan dengan sensor suhu yang lain.



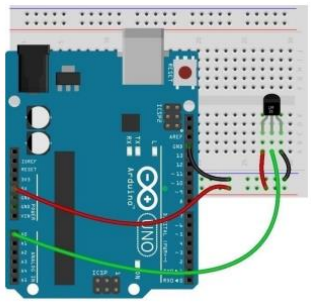
Walaupun tegangan maksimum LM35 20 V, tetapi rekomendasi tegangan yang digunakan adalah 5 V, sehingga tidak memerlukan catu daya dari luar cukup dengan menghubungkannya ke Arduino. Untuk membaca data sensor ini, kita perlu merubah data analog berupa tegangan analog menjadi data digital pada Arduino. LM35 merespon perubahan suhu sebesar 10mV/derejat celcius, untuk itu kita perlu merubah

nilai tegangan yang dihasilkan menjadi milivolt. Dengan menggunakan Arduino yang menggunakan ADC 10bit atau sama dengan 1024 dan tegangan referensi yang digunakan yaitu 5 V, maka didapatkan persamaan berikut ini:

$$Derajat\ Celcius = \frac{ADC}{1024}\frac{5000\ mV}{10\ mV}$$

$$Derajat\ Celcius = ADC\ x\ 0{,}48828125$$

Contoh Penggunaan:



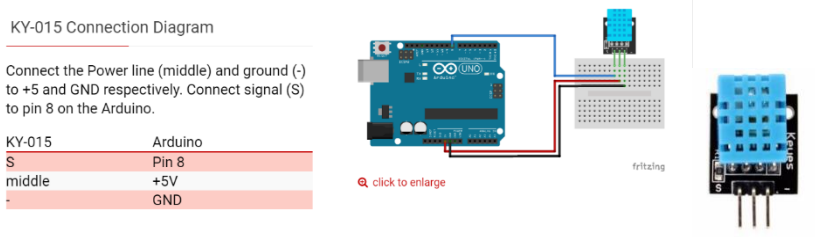
Program:

```
//inisialisasi pin
float temp;
int tempPin = A0;
void setup() {
 Serial.begin(9600);
}
 void loop() {
 temp = analogRead(tempPin);
temp = temp * 0.48828125;
 Serial.print("TEMPRATURE = ");
Serial.print(temp);
 Serial.print("*C");
Serial.println();
delay(1000);
```

}

12.5 Sensor Suhu dan Kelembapan DHT11

- DHT11 adalah sensor yang berguna untuk mengukur suhu sekaligus kelembaban udara.
- Catu daya yang diperlukan sensor: 3V-5V DC
- Pengukuran suhu antara 0^0−50^0 C dengan tingkat presisi +- 2^0 C
- Pengukuran kelembaban udara antara 20%-90% dengan tingkat presisi +- 5%.
- Agar diperoleh hasil yang stabil, jarak pembacaan dilakukan dengan selang waktu setidaknya satu detik.



Untuk pembuatan program yang melibatkan sensor DHT11 ini, pada software Arduino IDE membutuhkan tambahan library DHT11.

Library ini dapat ditambahkan dengan cara:
- masuk di menu Sketch > Include Library > Manage Libraries > Ketik DHT Sensor Library by Adafruit > Pilih Install.

Atau dapat didownload secara terpisah di link:

https://www.arduino.cc/reference/en/libraries/dht-sensor-library/

Setelah library didownload, kemudian klik menu Sketch > Include Library > klik Add .zip Library.

```ino
#include <dht.h>

const int PIN_DHT = A0;

dht DHT;

void setup()
{
  Serial.begin(9600);
  Serial.println("Informasi kelembaban dan suhu udara");
}

void loop()
{
  DHT.read11(PIN_DHT);
  Serial.print("Kelembaban udara = ");
  Serial.print(DHT.humidity);
  Serial.println("%");
  Serial.print("Suhu = ");
  Serial.println(DHT.temperature);

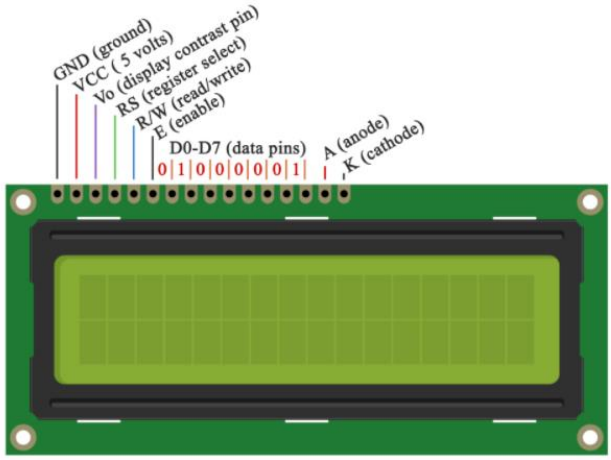
  delay(1000); // Diperlukan oleh DHT11
}
```

BAB 13
PENGGUNAAN LCD PADA ARDUINO

13.1 Teori LCD pada Arduino

LCD (Liquid Crystal Display) adalah suatu jenis media tampil yang menggunakan kristal cair sebagai penampil utama. Di pasaran, terdapat berbagai ukuran LCD, satu di antaranya adalah LCD 16 x 2. Adapun fitur yang disajikan dalam LCD ini adalah [9]:

- Terdiri dari 16 karakter dan 2 baris.
- Mempunyai 192 karakter tersimpan.
- Terdapat karakter generator terprogram.
- Dapat dialamati dengan mode 4-bit dan 8-bit.
- Dilengkapi dengan back light.
- Bekerja pada 5 volt sehingga dapat disambungkan langsung ke pin VCC pada Arduino.



Proses inisialisasi pin arduino yang terhubung ke pin LCD RS, Enable, D4, D5, D6, dan D7, dilakukan dalam baris LiquidCrystal (2, 3, 4, 5, 6, 7), di mana LCD merupakan variable yang dipanggil setiap kali intruksi terkait LCD akan digunakan [9].

Pin LCD nomor 4 (RS) merupakan Register Selector yang berfungsi untuk memilih Register Kontrol atau Register Data. Register kontrol digunakan untuk mengkonfigurasi LCD. Register Data digunakan untuk menulis data karakter ke memori display LCD [10].

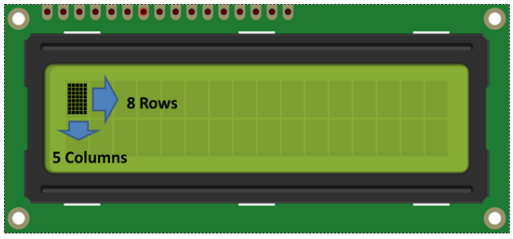
Pin LCD nomor 5 (R/W) digunakan untuk memilih aliran data apakah READ ataukah WRITE. Karena kebanyakan fungsi hanya untuk membaca data dan hanya perlu menulis data saja ke LCD, maka kaki ini dihubungkan ke GND (WRITE) [10].

Pin LCD nomor 6 (ENABLE) digunakan untuk mengaktifkan LCD pada proses penulisan data ke Register Kontrol dan Register Data LCD [10].

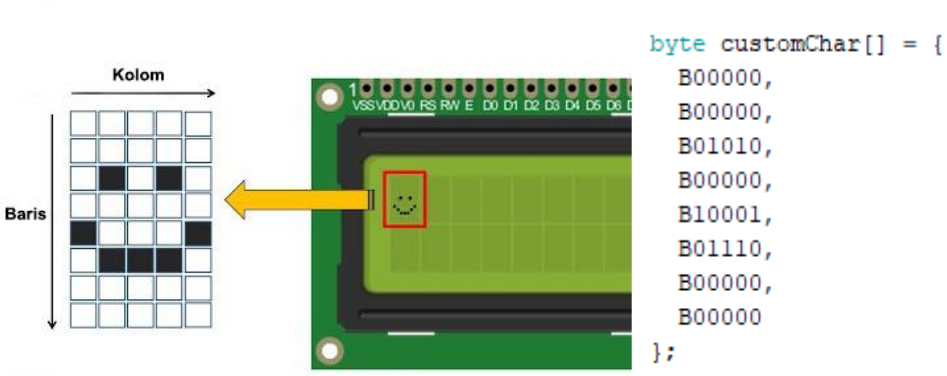
D1 – D3 : jalur untuk transfer 8 bit data. D4 – D7 : jalur untuk transfer 4 bit dan 8 bit data [10].

Pin No	Symbol	Level	Description
1	Vss	0 V	Ground
2	V_{DD}	5.0V	Supply voltage for logic
3	V o	variable	Operating voltagefor LCD
4	RS	H/L	H data/L instruction code
5	R/W	H/L	H/Read (MPU-module) L/write (MPU-module)
6	E	H, H-L	Chip enable signal
7	DB0	H/L	Data bit 0
8	DB1	H/L	Data bit 1
9	DB2	H/L	Data bit 2
10	DB3	H/L	Data bit 3
11	DB4	H/L	Data bit 4
12	DB5	H/L	Data bit 5
13	DB6	H/L	Data bit 6
14	DB 7	H/L	Data bit 7
15	A		Power supply for LED backlight (+)
16	K		Power supply for LED backlight (-)

Berikut adalah contoh penggunaan custom character pada LCD Arduino [11]:



Gambar Jumlah Baris dan Kolom pada LCD Arduino 16x2



Gambar Contoh Penggunaan Custom Character pada Arduino

13.2 Contoh Program LCD pada Arduino

1. Buatlah program untuk menampilkan kalimat pada LCD :

   "Ingat 3M" pada baris pertama, dan "Stay Safe & Stay Healthy" pada baris kedua.

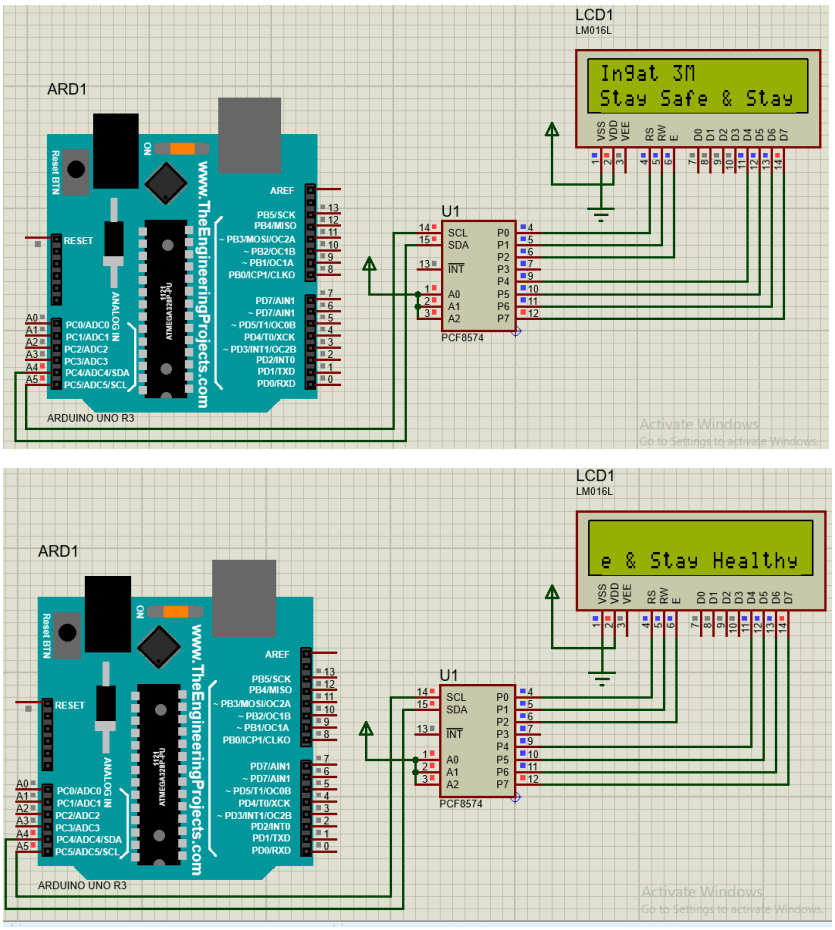
   Program:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup(){
// pilih LCD 16 x 2    (16 kolom, 2 baris)
lcd.begin(16,2);
lcd.print("Ingat 3M");
}

void loop(){
int i=0;
lcd.setCursor(0,1);
lcd.print("Stay Safe & Stay Healthy");
for(i=0;i<16;i++){
  lcd.scrollDisplayLeft();
  delay(200);
}
}
```

   Hasil:



Pada program ini, dilakukan penulisan program untuk menampilkan karakter pada LCD. Pada implementasinya, LCD yang digunakan merupakan LCD 16 x 2 dengan Modul I2C kecuali pada nomor 2 yaitu menggunakan LCD 32 x 2. Untuk penggunaan LCD I2C, digunakan library LiquidCrystal_I2C dan diinisalisasi pin pada LCD dengan perintah LiquidCrystal_I2C lcd(0x27, 16, 2); sedangkan untuk penggunaan LCD tanpa modul I2C, digunakan library LiquidCrystal dan diinisialisasi pin dari RS, E, D4, D3, D2, D1 secara berurutan. Lalu pada program ini, dalam fungsi void setup diberikan perintah untuk menginisialisasi LCD yang digunakan yaitu 16 x 2 dan diberikan perintah lcd.print untuk menampilkan kalimat “Ingat 3M”. Kemudian, fungsi void loop berisi perintah untuk menampilkan kalimat “Stay Safe & Stay Healthy pada baris kedua kolom pertama sebagai karakter berjalan dari kanan ke kiri dengan mengunakan perintah

lcd.scrollDisplayLeft(); pada fungsi for dengan batas sampai dengan index ke 15 menggunakan operator increment.

2. Buatlah program untuk menampilkan ‘custom caracter’ pada baris pertama LCD yang bergerak dari kanan ke kiri.

Program:

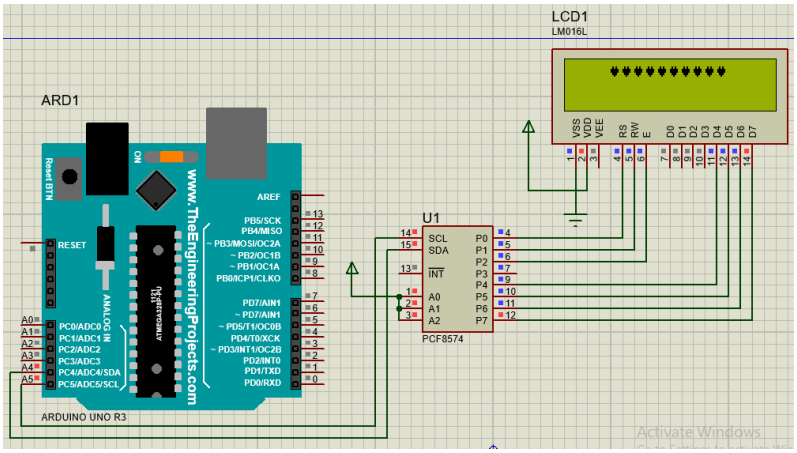
```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);

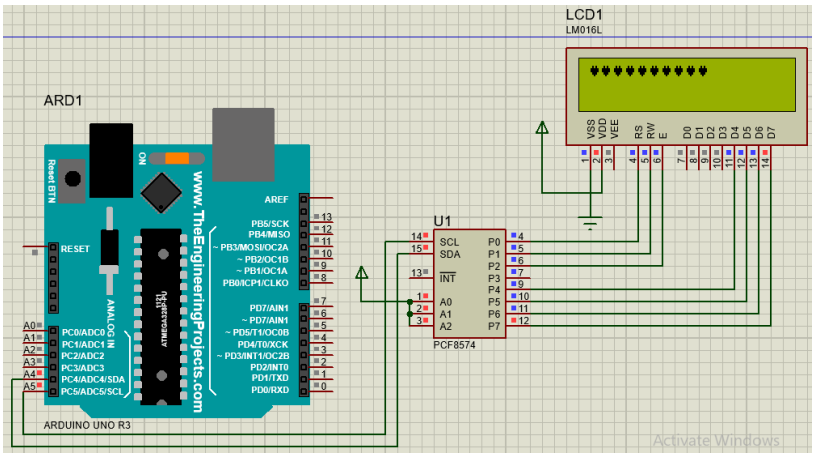
byte customChar[] = {
  B00000,
  B01010,
  B11111,
  B11111,
  B01110,
  B00100,
  B00000,
  B00100
};

void setup(){
  lcd.begin(16,2);
  lcd.createChar(0,customChar);
  for(int i=15;i>=6;i--){
  lcd.setCursor(i,0);
  lcd.write((byte)0);
}
}

void loop(){
  for(int i=0;i<16;i++){
  lcd.scrollDisplayLeft();
  delay(100);
}
}
```

Hasil:





Pada program ini, penulisan program diawali dengan inisialisasi custom karakter yang merupakan 8 kombinasi deretan 5 bit. Dengan nilai 0 berarti posisi tersebut LOW dan nilai 1 berarti posisi tersebut HIGH. Pada fungsi void setup, diberikan perintah lcd.createChar(0,customChar) untuk melakukan pointing dari array customChar ke integer 0. Lalu, terdapat perulangan for dari 15 sampai dengan 6 dengan operator decrement yang kemudian indeks tersebut akan digunakan pada parameter kolom pada perintah setCursor(I,0) sehingga penulisan custom karakter dimulai dari kolom ke 15. Untuk menuliskan custom karakter, digunakan perintah lcd.write(byte(0)). Kemudian, pada fungsi void loop, diberikan perintah agar custom karakter berjalan dari kanan ke kiri sebanyak 16 kali menggunakan perintah lcd.scrollDisplayLeft.

BAB 14
AKTUATOR (RELAY DAN MOTOR DC) PADA ARDUINO

14.1 Pengertian Relay

Relay merupakan jenis golongan saklar yang beroperasi berdasarkan prinsip elektromagnetik yang dimanfaatkan untuk menggerakan kontaktor guna menyambungkan rangkaian secara tidak langsung. Tertutup dan terbukanya kontaktor disebabkan oleh adanya efek induksi magnet yang dihasilkan dari kumparan induktor yang dialiri arus listrik.

Perbedaan dengan saklar yaitu pergerakan kontaktor pada saklar untuk kondisi on atau off dilakukan manual tanpa perlu arus listrik sedangkan relay membutuhkan arus listrik. Komponen elektronika ini menggunakan prinsip elektromagnetik untuk menggerakan saklar sehingga dengan arus listrik yang kecil (low power) dapat menghantarkan listrik yang bertegangan lebih tinggi.

14.2 Fungsi Relay

Beberapa fungsi relay saat di aplikasikan ke dalam sebuah rangkaian elektronika antara lain:

1. Mengendalikan sirkuit tegangan tinggi dengan menggunakan bantuan signal tegangan rendah.
2. Menjalankan logic function atau fungsi logika.
3. Memberikan time delay function atau fungsi penundaan waktu.
4. Melindungi motor atau komponen lainnya dari korsleting atau kelebihan tegangan.

Kontak point relay terdiri dari 2 jenis yaitu:

1. Normally Close (NC) yaitu kondisi awal sebelum diaktifkan akan selalu berada pada posisi close (tertutup).
2. Normally Open (NO) yaitu kondisi awal sebelum diaktifkan akan selalu berapa pada posisi open (terbuka).

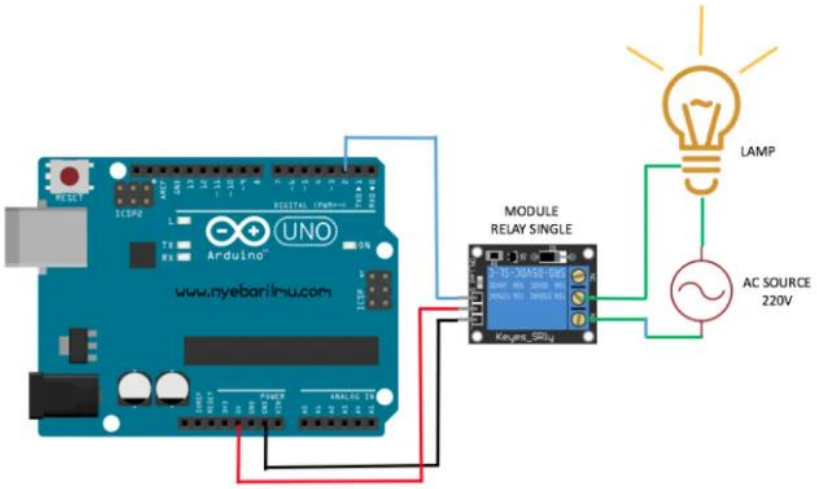
14.3 Cara Kerja Relay

Pada sebuah relay terdapat 4 bagian penting yaitu: Electromagnet (coil), Armature, Switch Contact Point (saklar) dan Spring.

Iron core(besi) yang dililitkan oleh kumparan coil berfungsi untuk mengendalikan iron core tersebut. Ketika kumparan coil di berikan arus listrik, maka akan timbul gaya elektromagnet sehingga akan menarik Armature berpindah posisi yang awalnya NC(tertutup) ke posisi NO(terbuka) sehingga menjadi saklar yang dapat menghantarkan arus listrik di posisi NO.

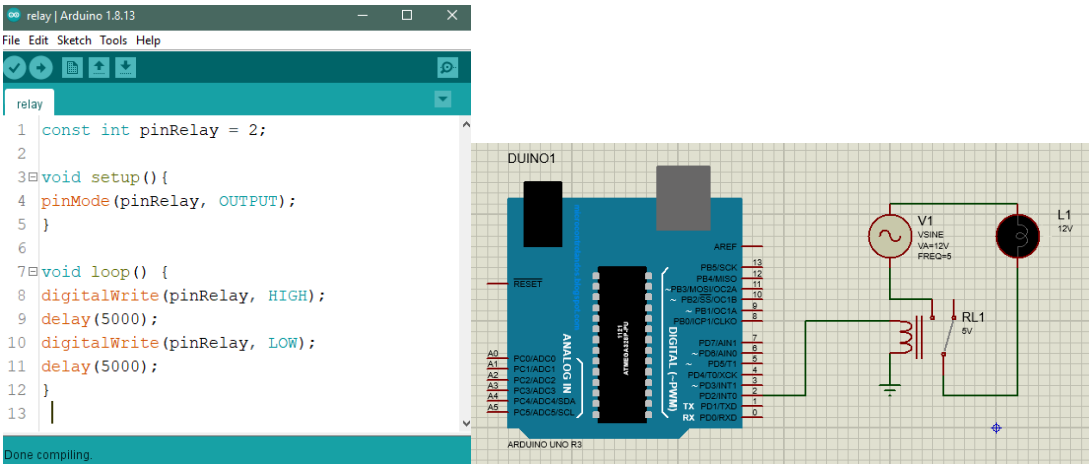
Posisi Armature yang tadinya dalam kondisi CLOSE akan menjadi OPEN atau terhubung. Armature akan kembali keposisi CLOSE saat tidak dialiri listrik. Coil yang digunakan untuk menarik Contact Point ke posisi CLOSE umunnya hanyak membutuhkan arus llistrik yang relatif kecil.

14.4 Interface Relay 1 Chanel pada Arduino

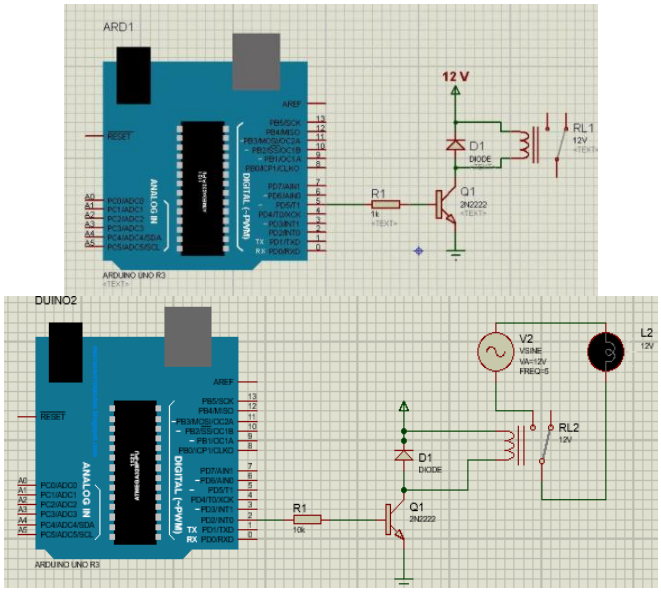


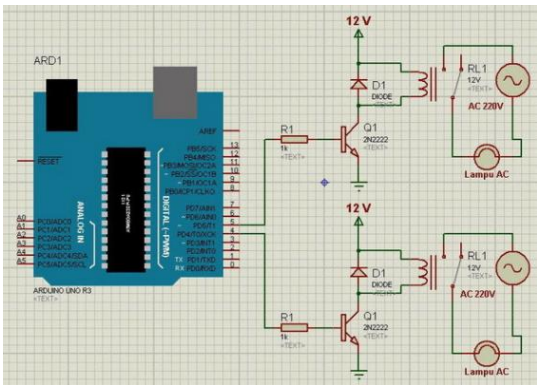
14.5 Simulasi Relay 1 Chanel pada Arduino

Contoh 1: pada contoh ini, relay menggunakan tegangan 5V sehingga tidak membutuhkan driver / transistor. Komponen lain yang dibutuhkan yaitu Lamp (12V) dan VSINE (Amplitudo 12V dan Freq 5 Hz)



Contoh 2: pada contoh ini, relay menggunakan tegangan 12V sehingga tidak bisa langsung dihubungkan ke arduino. Untuk itu dibutuhkan driver berupa transistor. Berikut ini contoh rangkaian driver relay yang terhubung pada arduino. Input driver diberi logika ‘HIGH’ untuk membuat relay ‘ON’ dan diberi logika ‘LOW’ untuk ‘OFF’.





14.6 Pengertian Motor DC



Motor DC (Direct Current) merupakan piranti elektronika yang mengubah energi listrik menjadi energi mekanik. Motor DC merupakan jenis motor paling sederhana yang memiliki 2 kabel, yaitu catu daya dan ground. Pemberian catu daya dapat dibolak-balik untuk memberikan efek arah putaran yang berbeda. Motor akan terus berputar jika catu daya diberikan dan berhenti jika catu daya diputus.

Aplikasi : biasa digunakan pada kipas angin atau roda pada robot/mobil mainan.

Motor DC memiliki 2 bagian penting yaitu Stator (bagian yang tidak berputar) dan Rotor (bagian yang berputar). Pada rotor inilah poros diletakkan. Di ujung poros dapat diletakkan objek yang ingin diputar.

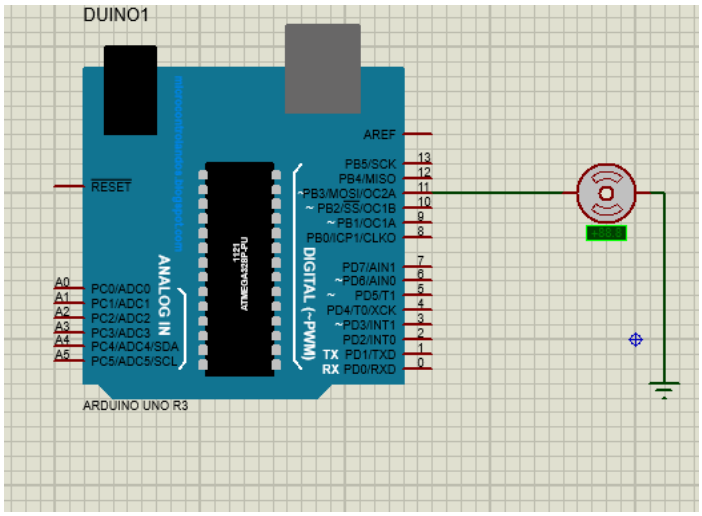
14.7 Simulasi Motor DC

Contoh 1:

```
// ---------------------------------------

// Contoh untuk mengatur putaran motor DC
```

```
// ---------------------------------------
const int PIN_PWM = 11;        // gunakan pin yang available untuk PWM
void setup()
{
  pinMode(PIN_PWM, OUTPUT);
}
void loop()
{
  // Kecepatan tinggi
  analogWrite(PIN_PWM, 255);    //255 merupakan nilai kecepatan tertinggi
  delay(5000);
    // Kecepatan sedang
  analogWrite(PIN_PWM, 128);
  delay(5000);
      // Motor dimatikan
  analogWrite(PIN_PWM, 0);
  delay(5000);
}
```



Atur tegangan pada motor DC sebesar 5V.

Contoh 2:

```
// ---------------------------------------

// Contoh untuk mengatur putaran motor DC

//    menggunakan potensiometer 5K

// ---------------------------------------

const int PIN_PWM = 11;

const int PIN_POTENSIO =A0;

void setup()

{

  pinMode(PIN_PWM, OUTPUT);

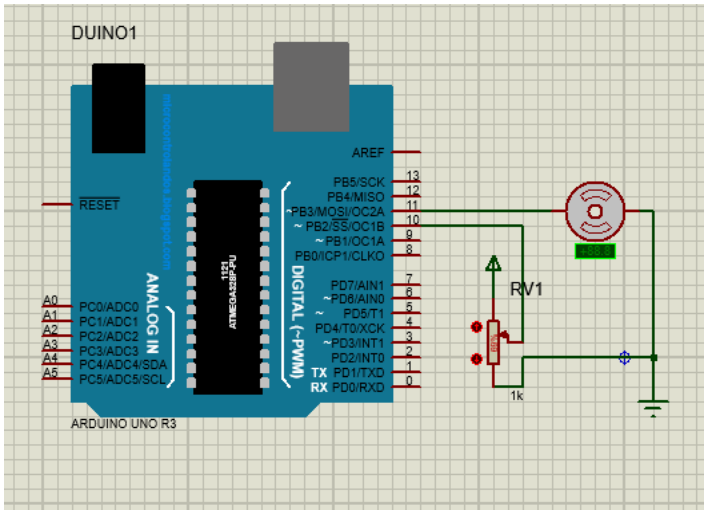
}

void loop()

{
```

```
  // Baca nilai yang diatur melalui potensiometer
  int nilai = analogRead(PIN_POTENSIO);


  // Atur kecepatan motor DC
  analogWrite(PIN_PWM, nilai / 4);    //atau menggunakan fungsi map
}
```

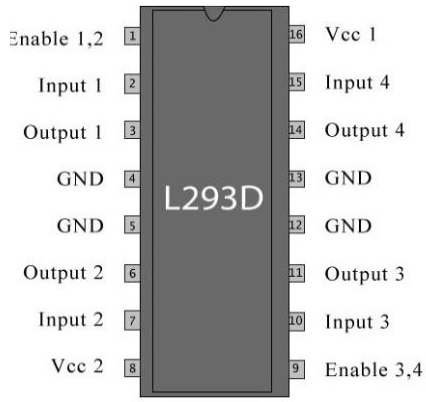


Pengaturan Motor DC dengan Menggunakan Motor L293D:

Motor Driver atau bisa disebut juga H-Bridge merupakan suatu rangkaian yang digunakan untuk menggerakan motor DC. Motor Driver menerima perintah dari mikrokontroler untuk menggerakan motor DC.

Terdapat 3 pengaturan yang dapat dilakukan:

- Mengontrol motor berputar searah jarum jam.
- Mengontrol motor berputar berlawanan jarum jam.
- Mengatur kecepatan putar
- Mengontrol kecepatan putar motor.



IN 1	IN 2	KONDISI MOTOR DC 1
0	0	STOP
1	0	PUTAR BERLAWANAN JARUM JAM (CCW)
0	1	PUTAR SEARAH JARUM JAM (CW)
1	1	STOP

IN 3	IN 4	KONDISI MOTOR DC 2
0	0	STOP
1	0	PUTAR BERLAWANAN JARUM JAM (CCW)
0	1	PUTAR SEARAH JARUM JAM (CW)
1	1	STOP

Contoh 3:

```
// ---------------------------------------

// Contoh pengaturan arah motor

//    menggunakan H-bridge L293D

// ---------------------------------------

int PIN_I1 =  8;  // Pin untuk port I1

int PIN_I2 =  9;  // Pin untuk port I2

int PIN_EA = 11;  // Pin untuk port EA / PWM

const int KECEPATAN = 128; // Antara 0 dan 255

void setup()

{
```

```
  pinMode(PIN_I1, OUTPUT);
  pinMode(PIN_I2, OUTPUT);
  pinMode(PIN_EA, OUTPUT);

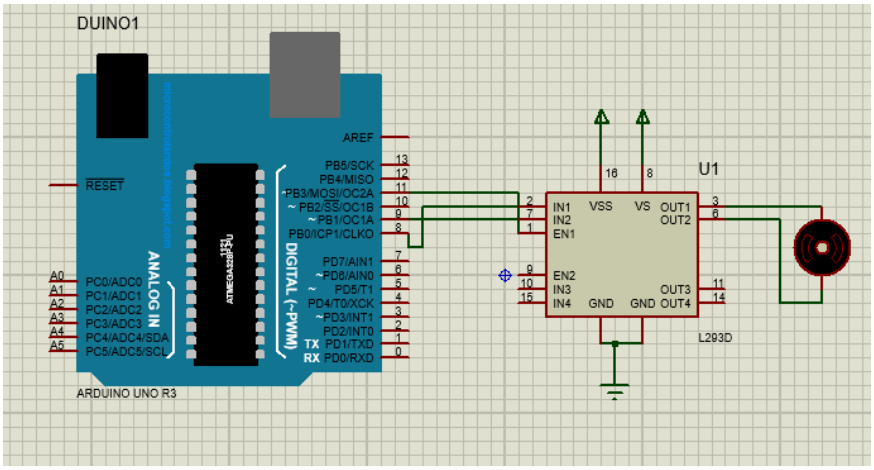
  // Atur kecepatan
  analogWrite(PIN_EA, KECEPATAN); // Atur kecepatan
}
void loop()
{
  // Berlawanan arah jarum jam
  digitalWrite(PIN_I1, LOW);
  digitalWrite(PIN_I2, HIGH);
  delay(2000);
    // Stop putaran
  digitalWrite(PIN_I1, LOW);
  digitalWrite(PIN_I2, LOW);
  delay(2000);
    // Searah jarum jam
  digitalWrite(PIN_I1, HIGH);
  digitalWrite(PIN_I2, LOW);
  delay(2000);
    // Stop putaran
```

digitalWrite(PIN_I1, LOW);

digitalWrite(PIN_I2, LOW);

delay(2000);

}



Contoh 4:

// Coding: //

int EN1=11;

int EN2=12;

void setup() {

pinMode(EN1,OUTPUT);

pinMode(EN2,OUTPUT);

pinMode(5,OUTPUT);

pinMode(6,OUTPUT);

pinMode(9,OUTPUT);

pinMode(10,OUTPUT);

```
}

void loop() {

digitalWrite(EN1,HIGH);

digitalWrite(EN2,HIGH);

forward();

}

void forward() {

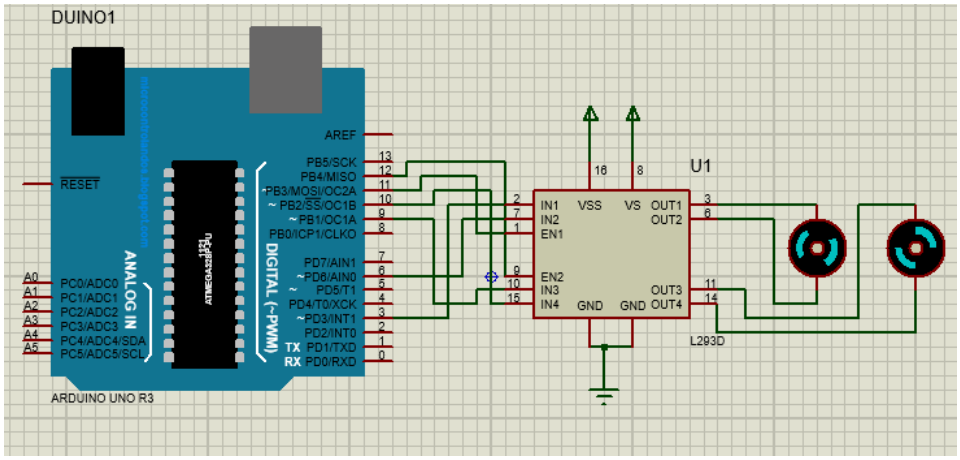
analogWrite(5,100); // pengaturan arah putaran dan kecepatan motor kiri

analogWrite(6,0);

analogWrite(9,0); //pengaturan arah putaran dan kecepatan motor kanan

analogWrite(10,200);

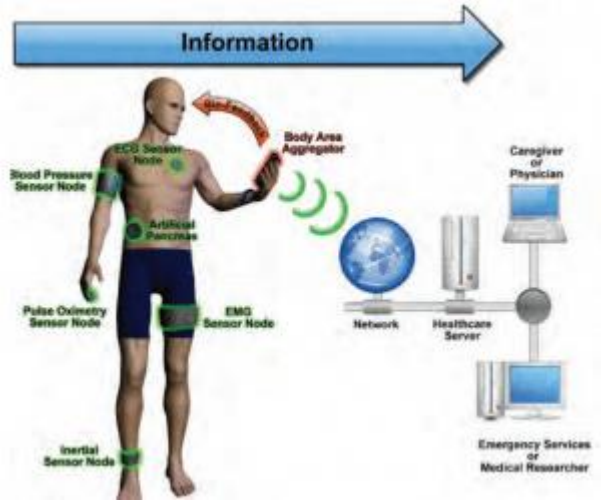
 }
```



BAB 15
WSN DAN IOT

15.1 Teori WSN

Wireless sensor network adalah sebuah kumpulan node yang dapat berupa sensor yang akan melakukan pengambilan data pada parameter ukur dan kemudian dikirimkan pada sebuah node sentral atau sebuah server untuk dilakukan pengolahan data (Firdaus, 2014). Node-node yang ada pada WSN merupakan sensor yang diletakkan pada titik-titik pada sebuah area yang ingin diketahui besaranya (Ilyas. M., Mahgoub. I., 2005). Misalnya pada sebuah ladang pertanian, ingin diketahui kelembapan tanahnya, maka sensor pengukur kelempan akan diletakkan ditanah pada area pertanian tersebut, dan jumlahnya tidak hanya satu namun puluhan sensor. Node-node tersebut masing-masing memilki sumber daya sendiri yang dapat berupa baterai, dan memiliki perangkat transmitter data untuk dapat mengirimkan data ke node sentral atau server.



Gambaran WSN sebagai sistem informasi kesehatan (Fischione. C., 2014).

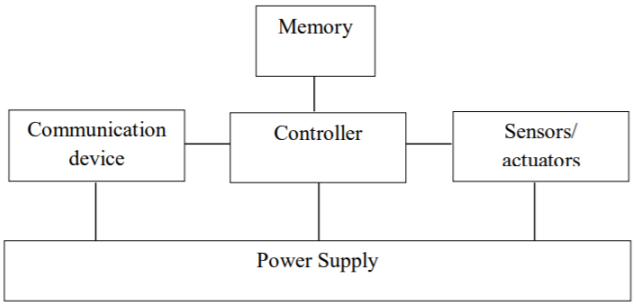
WSN sudah mulai digunakan pada banyak bidang. WSN digunakan pada bidang kesehatan, perumahan, kemananan. Contoh pada pembukaan awal artikel ini adalah

contoh penerapan WSN pada bidang pertanian. WSN juga mulai populer digunakan pada bidang kesehatan. WSN digunakan sebagai pemantau parameter kesehatan seorang pasien yang dapat dilakukan oleh pasien maupun petugas kesehatan agar dapat melakukan perawatan dan pemataуan rutin.

1. Sensor Node

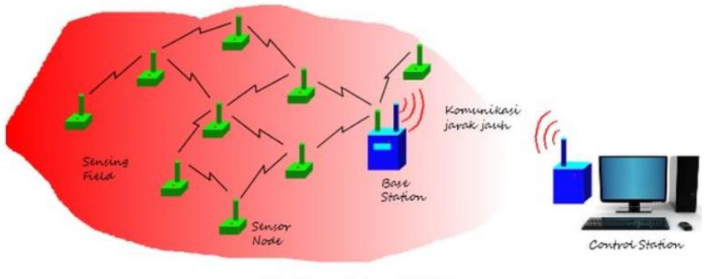
   Teknologi sensor digunakan untuk mengukur berbagai fenomena di sekitarnya, seperti temperatur, kelembaban udara, kadar gas, posisi, dan lain sebagainya. Tidak hanya itu, penemuan sensor-sensor baru terutama jenis bio sensor terus dilakukan untuk mengamati fenomena dalam tubuh tanaman, hewan, dan manusia. Sensor memiliki kemampuan mengubah besaran yang diukur dari suatu fenomena menjadi besaran listrik yang selanjutnya dapat diproses oleh komputer. Namun untuk beberapa kondisi, keluaran sensor tidak langsung dihubungkan dengan komputer. Beberapa di antaranya kondisi tersebut yakni kecilnya sinyal listrik yang dihasilkan oleh sensor, lokasi sensor yang terpisah dari komputer, dan jumlah sensor yang cukup banyak. Karena itu, sensor perlu dilengkapi perangkat lain sebelum dihubungkan dengan komputer [12].

   Untuk dapat terkoneksi dengan komputer, sensor-sensor ini membutuhkan perangkat pemroses data (*processor*) atau *controller*, pemancar dan penerima (*transceiver*) atau *communication device*. *Processor* atau *controller* mengubah sinyal listrik menjadi data digital dengan format yang dibutuhkan oleh komputer. Sementara, *transceiver* mengirim data digital ke komputer melalui media komunikasi yang dipilih. Keseluruhan unit yang terdiri dari sensor/aktuator, *controller*, *transceiver*, *memory* serta *power supply* ini disebut sebagai *Sensor Node* (SN) .



2. Jaringan

Untuk penerapan pada ruang yang lebih luas, jumlah SN yang digunakan umumnya lebih dari satu. Dalam operasinya, semua SN yang berada dalam *sensing field* mengirim data ke sebuah *Base Station* (BS) yang terletak pada lokasi yang sama dengan SN. Setelah mengumpulkan data dari SN, BS kemudian meneruskan data ke *Control Station* (CS) yang umumnya terpisah dalam jarak yang jauh. Pada bagian terakhir inilah, data dikumpulkan untuk dianalisa lebih lanjut menjadi informasi. Jaringan yang dibentuk oleh SN, BS, dan CS ini dikenal sebagai *Sensor Network*. Jika komunikasi antar unit dalam *sensor network* dilakukan tanpa menggunakan kabel, maka sistem ini disebut sebagai *Wireless Sensor Network* (WSN).



Konfigurasi dasar WSN

3. Komunikasi dan Konsumsi Daya

Dalam melakukan komunikasi antar unit dalam WSN, media komunikasi yang digunakan sangat tergantung pada lingkungan di mana WSN ditempatkan. Pada lokasi di darat, gelombang radio, LASER atau infra merah (IR) umum digunakan, Untuk WSN yang diletakkan dalam tanah

(*underground*), media komunikasi yang dipilih adalah gelombang radio dengan frekuensi tertentu yang mempunyai karakteristik mampu merambat dalam tanah. Untuk WSN yang terletak di bawah permukaan air (*underwater*) menggunakan gelombang suara sebagaimana yang digunakan oleh ikan paus dan lumba-lumba, karena mampu merambat dalam jarak yang jauh. Untuk mendapatkan hasil ukur yang teliti, SN umumnya diletakkan di lokasi yang dekat dengan fenomena yang diukur. Namun kebutuhan ini juga memiliki konsekuensi tersendiri. Instalasi di medan sulit seperti di dalam hutan lebat, tebing curam, mulut kawah gunung berapi, dan lain sebagainya, dapat mengundang kesulitan atau resiko bagi petugas instalasi. Oleh karenanya, SN dirancang untuk mempunyai ukuran kecil, mudah dipasang, mudah dipindahkan, dan minim atau tanpa perawatan. Untuk mencapai persyaratan ini, sumber listrik SN biasanya hanya mengandalkan tenaga baterai.

Karena SN umumnya diinginkan dapat bekerja dalam waktu yang lama tanpa perawatan (misalnya satu atau dua tahun), tenaga baterai harus dapat dihemat. Untuk itu, SN dirangkai dari sensor/aktuator, *controller / processor*, *transceiver*, serta *memory* yang mempunyai daya rendah. Akibatnya, *controller / processor* yang dipilih mempunyai kemampuan proses rendah, sementara jangkauan pancaran *transceiver* juga sangat terbatas. Untuk lebih menghemat konsumsi daya baterai, jeda waktu pengiriman data juga dapat diperpanjang. Dalam masa jeda yang panjang tersebut, aktifitas *controller / processor* dikurangi dan transceiver dimatikan. Jeda waktu pengiriman bisa diperpendek kembali jika hasil pengukuran menunjukkan perubahan besar dalam waktu yang cepat. Keadaan ini dianggap sebagai indikasi adanya situasi *emergency*, di mana pelaporan data harus sering dilakukan untuk memperkirakan datangnya bencana. Mekanisme seperti ini umumnya digunakan pada sistem peringatan dini.

4. Base Station (BS)

Berbeda dengan SN yang berukuran kecil dan mempunyai kemampuan terbatas, BS mempunyai ukuran yang lebih besar dengan spesifikasi *processor* dan *transceiver* yang lebih baik. Ini karena BS harus melakukan pekerjaan yang lebih banyak dan kompleks daripada yang dilakukan oleh SN. Misalnya, saat menerima data dari beberapa SN, BS tidak meneruskan langsung tiap data kepada CS, melainkan perlu merangkum dan mengubahnya menjadi format data yang sesuai dengan media komunikasi antara BS dan CS. Selain itu, berbeda dengan SN yang dapat "ditidurkan" untuk periode yang panjang, BS harus tetap "terbangun" untuk menunggu kiriman data dari SN-SN lain yang periode aktifnya berbeda. Juga, BS harus tetap menjaga jaringan komunikasinya dengan CS. Karena itu, BS perlu dilengkapi baterai dengan kapasitas yang lebih besar. Seringkali, perangkat panel surya atau pembangkit tenaga listrik lainnya ditambahkan untuk mengisi tenaga baterai secara berkala.

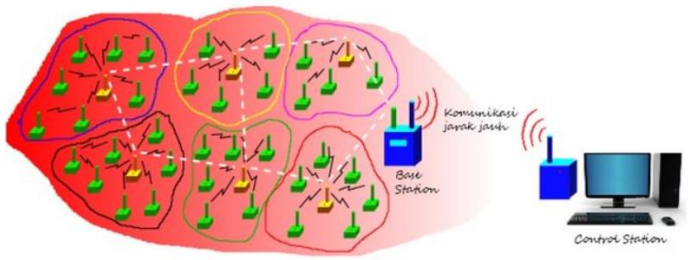
5. Koneksi Wireless

Jangkauan komunikasi radio SN sangatlah dekat, biasanya berkisar beberapa puluh meter hingga lima ratusan meter, dikarenakan daya pancar *transceiver* yang rendah. Jarak jangkau ini akan lebih rendah jika SN ditempatkan dalam hutan, karena transmisi sinyal radio akan terganggu oleh lebatnya tanaman dan pohon dalam hutan. Jangkaun transmisi radio yang pendek akan menjadi masalah jika SN terpisah jauh dari BS, karena SN tidak dapat mengirim data ke BS. Untuk mengatasi masalah ini, elemen-elemen dalam WSN dirancang untuk mampu melakukan komunikasi *multihop*. Dengan metode ini SN-SN lain yang jaraknya jauh dari BS, dapat terhubung ke BS melalui SN yang dekat dengan BS. Paket data dari SN yang bersangkutan di kirim ke BS secara berantai oleh SN-SN perantara. Sambil tetap mengirim paket datanya sendiri, SN-SN perantara juga berfungsi sebagai *relay station* Komunikasi *multihop* ini merupakan salah satu ciri

utama dari WSN. Dengan cara ini, cakupan sebuah jaringan WSN dapat diperluas *(scalable).*

6. Multi-tier

Perebutan jalur komunikasi antar SN yang berjumlah ratusan dan ribuan untuk mengirim atau me-*relay* paket data akan mengakibatkan tabrakan (*collision*) dan kemacetan (*congestion*), yang pada akhirnya dapat melumpuhkan jaringan. Untuk mengatasi masalah ini, struktur jaringan dasar yang merupakan jaringan *single-tier* dikembangkan menjadi jaringan *multi-tier*. Sebuah jaringan *multi-tier* dapat mengelompokkan SN menjadi beberapa *cluster*. Setiap *cluster* dipimpin oleh sebuah *cluster head* (CH), yang berfungsi mengumpulkan data dari seluruh SN yang ada dalam clusternya. Setelah dikumpulkan, data dirangkum dan dikirimkan ke BS. Karena SN hanya mengirim data ke CH di dalam clusternya, maka jumlah aliran data dari tiap individu SN ke BS menjadi berkurang. Akibatnya, *collision* dapat dikurangi, sementara *congestion* dapat dicegah.



Jaringan WSN *multi-tier*

7. WSN-IOT

WSN dapat dikembangkan menjadi IoT, atau dengan kata lain WSN adalah pondasi terbentuknya IoT.

15.2 Teori IOT

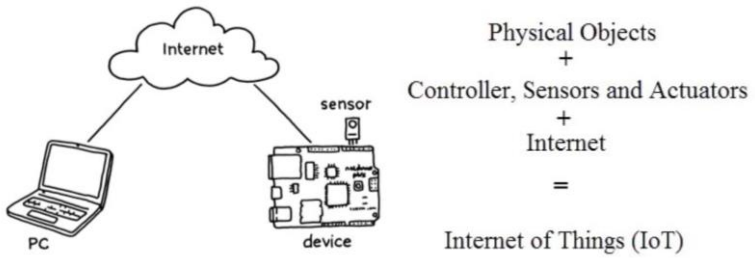
IoT yang pertama kali dibuat adalah sebuah pemanggang roti. Pada tahun 1989, John Romkey dan Simon Hackett mengkoneksikan sebuah toaster ke internet. Toaster

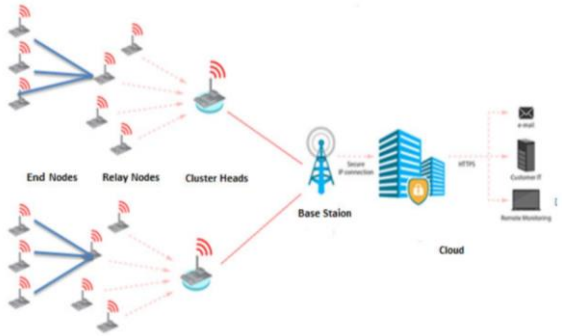
atau pemanggang roti tersebut bisa dimatikan dan dinyalakan melalui jaringan internet. Meskipun terdengar sederhana, alat ini sudah termasuk sangat canggih pada masa itu [5]. Pada tahun 1994, Steve Mann menciptakan IoT lainnya dalam bentuk WearCam. Di tahun 1997, Paul Saffo menemukan teknologi sensor yang bisa digunakan pada berbagai perangkat. Pada tahun 1999, Kevin Ashton yang merupakan direktur Auto IDCentre dari MIT membuat konsep Internet of Things yang terkenal sampai sekarang. kutipan dari Kevin Ashton yang di tulis pada tahun 2009 untuk jurnal RFID :

"If we had computers that knew everything there was to know about things—using data they gathered without any help from us—we would be able to track and count everything, and greatly reduce waste, loss and cost. We would know when things needed replacing, repairing or recalling, and whether they were fresh or past their best."

"We need to empower computers with their own means of gathering information, so they can see, hear and smell the world for themselves, in all its random glory."

Dari kutipan diatas dapat memberikan ide tentang ideologi yang melatarbelakangi dari pengembangan IoT. IoT adalah konsep yang menghubungkan semua perangkat ke internet dan memungkinkan perangkat IoT berkomunikasi satu sama lain melalui internet. IoT adalah jaringan raksasa dari perangkat yang tehubung – semua yang mengumpulkan dan membagikan data tentang bagaimana suatu oerangkat tersebut digunakan dan lingkungan dimana perangkat tersebut di operasikan.





IoT hierarchical architecture: WSN connected to cloud services through Gateways

1. Arsitektur IOT

- ***Sensor Connectivity Dan Network Layer***: Ini terdiri dari sensor / reader (pembaca) yang bertanggung jawab untuk mengumpulkan data dari lingkungan, jaringan untuk mengumpulkan dan mentransfer data sensor / reader dan aktuator untuk mencapai target sesuai data sensor. Jaringan sensor dapat berupa Wi-Fi, Ethernet, XBee, Bluetooth atau jaringan kabel.

- ***Gateway Dan Network Layer***: Layer atau lapisan ini menyimpan sebagian besar data dari sensor, reader, tag, dll. Dan mentransfer data ini ke ***Management Service Layer***, yang merupakan lapisan berikutnya. Ia juga bertanggung jawab untuk berbagai protokol jaringan untuk berbagai perangkat IOT. Gateway dapat berupa mikrokontroler, embedded OS, modul Komunikasi Radio atau Signal processor dan modular. Jaringan dapat berupa Wi-Fi, Ethernet, LAN atau WAN.

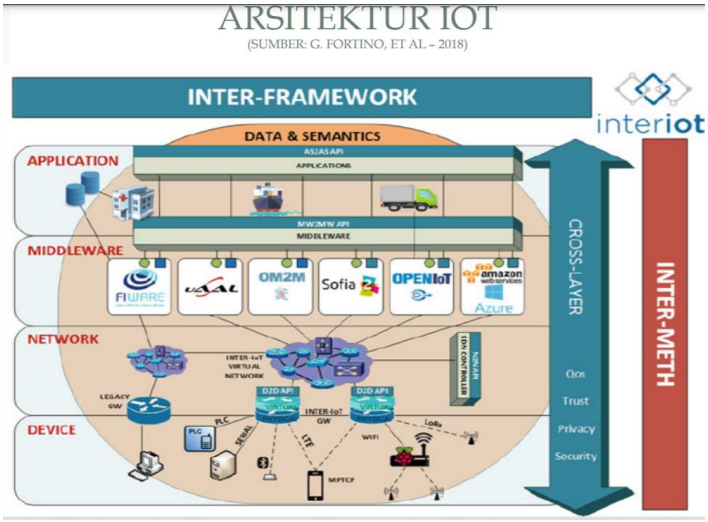
- ***Management Service Layer***: Lapisan ini mengamankan analisis perangkat IOT, analisis informasi, dan manajemen perangkat. Ini terdiri dari layanan dukungan operasional untuk memodelkan, mengkonfigurasi dan mengelola perangkat IOT, Billing Support System untuk mendukung penagihan dan pelaporan, dan layanan aplikasi IOT / M2M untuk mengontrol dan mengenkripsi data, mengelola aturan dan proses bisnis, dll. Dengan kata sederhana, lapisan ini memungkinkan ekstraksi informasi yang diperlukan dari data sensor yang dikumpulkan.

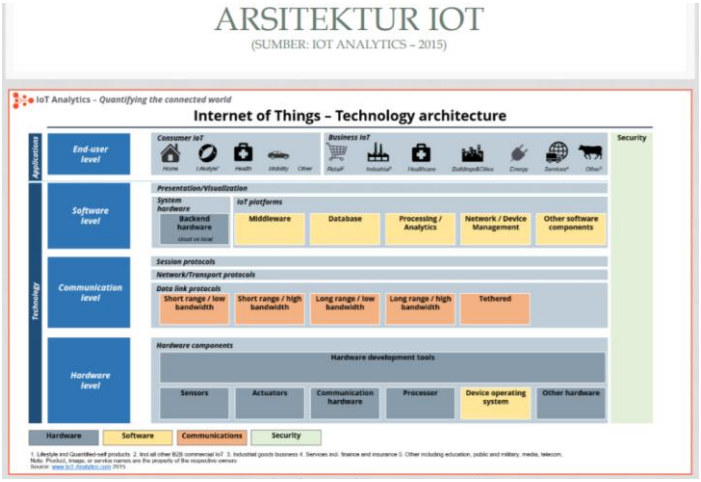
- ***Application Layer***: Lapisan ini menggunakan data yang dikumpulkan untuk melayani tujuan yang diperlukan. Ini mencakup otomatisasi rumah, perawatan kesehatan, pengawasan, retail, tracking, dan otomasi industry.

2. Cara kerja IOT

Cara Kerja IOT:

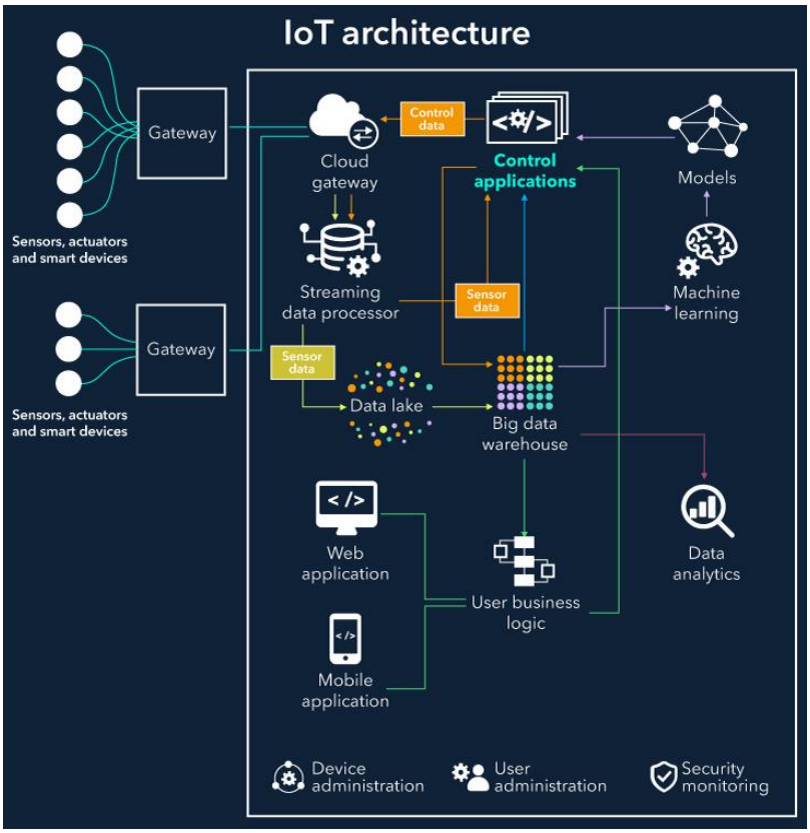
- Data dikumpulkan dari lingkungan atau environment oleh sensor / device.
- Data yang terkumpul kemudian dikirim ke cloud (internet storage) melalui saluran konektivitas seperti Wi-Fi, Satelit, Bluetooth, LAN, dll.,
- Setelah data diterima oleh cloud, ia akan diproses oleh program perangkat lunak. Ini melibatkan penggalian informasi yang diperlukan dari data yang diterima.
- Setelah data diproses oleh perangkat lunak, hasil akhirnya dikomunikasikan kepada pengguna melalui email, teks atau pemberitahuan.





3. Intrastruktur IOT

   Lima hal yang membentuk infrastruktur IoT adalah platform IoT, teknologi akses, penyimpanan & pemrosesan data, analitik data, dan keamanan. Bagian penting dari setiap perangkat yang terhubung ke internet, lima pilar inilah yang memungkinkan pertumbuhan untuk solusi IoT di masa depan.



DAFTAR PUSTAKA

[1] Minangsari, "Mikrokontroler ATMega 8535," pp. 5-7, 2015.

[2] Tkrakitan, "MACAM – MACAM REGISTER DI MIKROKONTROLER AVR ATMEGA 8535," 28 Januari 2016. [Online]. Available: https://tkrakitan1411890.wordpress.com/2016/01/28/macam-macam-register-di-mikrokontroler-avr-atmega-8535/. [Accessed 8 Desember 2022].

[3] Admin, "DASAR-DASAR PEMROGRAMAN AVR," 5 November 2007. [Online]. Available: https://maswie2000.wordpress.com/2007/11/05/dasar-dasar-pemrograman-avr/. [Accessed 8 Januari 2022].

[4] S. Sudirman, "PERCABANGAN, STACK, SUBRUTIN, DAN DELAY," 2019. [Online]. Available: https://slideplayer.info/slide/12472932/. [Accessed 8 Januari 2022].

[5] Pradana, "Arduino Uno," *2017*, pp. 3-4, 2017.

[6] Febrianto, "Apa itu Arduino Uno?," Ndoware, 9 April 2014. [Online]. Available: https://ndoware.com/apa-itu-arduino-uno.html. [Accessed 8 Januari 2022].

[7] Sugeng, "Mengenal Bagian - Bagian dan Features pada Papan Arduino UNO beserta Fungsinya," TECHNOLOGY AND ELECTRONICS, 18 Desember 2017. [Online]. Available: http://electronicsbot.blogspot.com/2017/12/mengenal-bagian-bagian-dan-features.html. [Accessed 8 Januari 2022].

[8] R. Hasanah, "ARDUINO," pp. 7-8, 2021.

[9] Husein, "MINIATUR PINTU GESER OTOMATIS BERBASIS ARDUINO," 2017. [Online]. Available: https://eprints.akakom.ac.id/3871/3/3_133310011_BAB_II.pdf. [Accessed 8 Januari 2022].

[10] M. A. Prasetyo, "Running Text di LCD 16x2 dengan Arduino," Boarduino, Desember 2014. [Online]. Available: http://www.boarduino.web.id/2014/12/running-text-di-lcd-dengan-arduino.html?m=0. [Accessed 8 Januari 2022].

[11] A. Putu, "Cara Membuat Custom Text atau Custom Karakter di LCD dan I2C Arduino," 17 Agustus 2020. [Online]. Available: https://mr-leong.com/cara-membuat-custom-text-lcd-arduino/. [Accessed 8 Januari 2022].

[12] F. Yusuf, "Mengenal Wireless Sensor Network," UGM, 2018 Agustus 2018. [Online]. Available: https://sensornetwork.mipa.ugm.ac.id/2018/08/22/125/. [Accessed 2008 Januari 2022].